

基于渐进信息集与边际价值评估的微网储能序贯调度

摘 要

高比例光伏接入下，微网购电与储能调度受到能量跨期耦合、预测误差、信息分时到达及动态电价的影响。若仅依据点预测求解单一最优轨迹，预测偏差将通过高额紧急购电与计划调整成本非对称地传导至实际运行费用。本文构建基于渐进信息集的序贯优化框架，将确定性调度、两阶段随机规划与多时点滚动重优化统一于同一预测-决策经济评价体系中。

针对问题一，将储能的跨时段能量转移刻画为确定性混合整数线性规划问题，在满足功率平衡及储能运行边界的条件下求解最优购电策略。最优全天计划购电量为 **59,482.70 kWh**，购电费用为 **35,126.95 元**，较无储能基准降低 **26.90%**。LP 松弛检验显示模型整数间隙为零，且未出现同时充放电现象。

针对问题二，严格遵循因果序贯评价协议，采用周周期负荷预测与七日同刻光伏季节均值作为预测基线，并基于历史负荷-光伏成对整日残差生成联合场景，构建两阶段随机 MILP 确定日前购电计划，同时建立严格的物理执行边界与跨日储能推演。2025 年 2 月至 12 月连续 334 天均获得最优解，实际总费用为 **14,839,462.00 元**，紧急购电量为 **398,161.90 kWh**。结果表明，在 5 倍惩罚下，预测误差会非对称传导至成本，模型有效性应由下游调度经济性而非单一统计指标决定。

针对问题三，构建提前期分段的融合预测与序贯滚动重优化模型，并以边际信息价值 (VOI) 衡量新增预报的经济贡献。四组信息扩张策略中， $S_2 = \{0, 6, 12\}$ 的年度费用最低，为 **14,505,500.73 元**；6:00 和 12:00 更新分别产生 **95,120.80 元** 和 **127,349.06 元** 的边际收益，而 18:00 更新的实现边际价值为 **-2,217.41 元**。Bootstrap 重采样检验 (95% 置信区间 [546.36, 792.54] 元/天) 进一步验证了 S_2 相较 S_0 (日均节省 666.08 元) 具有显著且稳定的日级经济收益。

针对问题四，将附件 4 给出的 10 min 动态电价作为外生时变参数嵌入上述随机-滚动框架。动态价格下，日前随机调度总费用为 **15,547,956.36 元**，滚动策略 S_2 的费用降至 **15,196,922.47 元**，较前者降低 2.26%，紧急购电量下降 23.74%。各预报节点的边际信息价值排序与固定电价情形保持一致，表明所建序贯决策框架对价格机制变化具有较好的稳定性。

综上，本文将四问统一为由确定性最优-随机日前决策-序贯信息更新-动态价格响应逐级扩展的运筹优化问题。研究表明，微网储能调度的关键不在于单纯追求更低的预测误差，而在于将预测不确定性、储能物理状态与市场结算规则共同转化为可执行的经济决策；预测信息的有效性最终取决于其能够贡献的**边际决策价值**。

关键字： 微网能量管理 联合残差场景 两阶段随机规划 滚动重优化 信息价值

一、 问题重述

1.1 问题背景

随着分布式光伏渗透率不断提高，微网已成为促进可再生能源就地消纳、提升供电灵活性和降低用能成本的重要形式 [1]。对于与外部电网相连的非孤岛微网，可在光伏富余或电价较低时用储能充电，负荷或电价较高时放电，实现能量跨时段转移 [8]。因此，如何在满足负荷需求和储能运行约束的前提下，协调外网购电与储能充放电以降低购电费用，是微网能量管理的重要优化问题。

实际调度还受储能状态跨期耦合与源荷预测不确定性的双重影响：SOC 使当前决策直接影响后续调节能力，光伏与负荷的随机波动又使日前计划面临预测偏差，因此决策应利用概率预测刻画不确定性，而非仅依赖单一点预测 [7]。本题中供电不足部分需按当时电价的 5 倍紧急购电，不同方向的预测误差经济后果差异显著；而 Smart “Predict, then Optimize” 框架也表明，统计误差更低未必对应更低的决策损失 [12]。因此，仅用 MAE、RMSE 衡量预测质量还不够，还需考察其对调度经济性的实际影响。

此外，微网预报信息呈现明显的序贯到达特征：系统在每日 0:00、6:00、12:00、18:00 持续获得新的光伏预测，并可付出调整成本修改尚未执行的购电计划。新增信息虽能降低不确定性，其经济价值却受剩余决策时域、既有购电承诺和调整成本制约，且随电价实时波动，储能的跨时段机会成本也随之变化。因此，本题核心在于：在预测不确定、信息逐步揭示与动态价格共同作用下，建立兼顾物理可行性与经济性的微网序贯调度模型，并评价新增预测信息的边际决策价值。

1.2 问题提出

为在保证微网供电可靠性的基础上充分发挥储能的跨时段调节能力，并研究预测不确定性、信息更新和动态电价对购电决策的影响，现结合题目所给负荷、光伏预测与实际出力、电价及储能参数建立数学模型，分析以下问题：

- **问题一：**在每日电价、负荷规律和当日光伏预测均已知的确定性条件下，以微网购电费用最小为目标，在满足负荷供给、储能容量、充放电功率、效率以及 0:00 与 24:00 储电量相同等约束下，建立日前购电与储能充放电协同优化模型，给出全天购电策略及储能运行方案，并给出指定时段购电量、全天购电量和购电费用。
- **问题二：**当负荷和光伏实际功率随日期发生变化，每日 0:00 只能依据当时可获得的信息制定当天购电计划，若实际供电不足则缺额部分按交易时刻电价的 5 倍进行紧急购电。试在避免未来信息泄漏的前提下，对负荷和光伏出力进行预测并刻画其不确定性，建立日前随机调度模型，在计划购电成本与紧急购电风险之间进行权衡，

给出 2025 年 2 月至 12 月的连续购电策略。

- **问题三：**在每日 0:00、6:00、12:00 和 18:00 均可获得新的未来 24 小时光伏预报的条件下，需在冻结已执行决策的基础上，根据逐步更新的信息滚动调整后续购电策略，并考虑计划购电量减少部分按交易电价 50% 结算、增加部分按交易电价 1.5 倍结算以及紧急购电等费用。试比较不同预报更新时点的经济贡献，并分析是否有必要进一步引入其他时刻的预报。
- **问题四：**当外网电价由固定日内价格扩展为实时波动价格后，利用附件 4 给出的动态电价数据，在保持问题二和问题三原有负荷、光伏及信息更新条件不变的基础上，分别重新建立日前随机购电模型和多时点滚动调整模型，分析动态电价对购电费用、储能行为及预测信息价值的影响，并生成完整的购电策略。

二、 问题分析

2.1 问题一的分析

问题一在确定电价、负荷与光伏预测下，核心是利用储能实现电能跨时段转移。由于当前充放电行为通过储能荷电状态（State of Charge, SOC）影响后续调节能力，问题具有跨时段耦合特征，需全天统一决策而非逐时寻优。

为此，本文以购电费用最小为目标，将购电量、储能充放电及弃光量设为决策变量，通过功率平衡、SOC 递推、容量/功率边界及充放电互斥约束建立确定性混合整数线性规划（MILP）模型。进一步删除互斥二元变量构造线性规划（LP）松弛模型，结合条件性交换论证与 LP-MILP 数值交叉验证考察整数约束作用；并基于储能往返效率推导跨时段套利的局部价格判据，结合效率、容量、最大充放电功率的灵敏度分析解释调度结果。

2.2 问题二的分析

问题二取消负荷、光伏完全已知的假设，每日 0:00 须依据当时信息制定全天计划。因缺电部分按 5 倍电价紧急购电，预测偏差具有非对称经济后果，问题由确定性优化转为预测不确定性下的日前随机决策问题。

本文采用严格序贯因果评价，按“历史信息预测-制定计划-实际运行-评价更新”逐日推进，避免未来信息泄漏：依据负荷、光伏周期结构分别采用周周期负荷预测与七日同刻光伏季节均值预测；再对历史残差中的负荷-光伏做成对整日重采样，生成保留日内相关性与源荷联合变化特征的随机场景。

由 5 倍紧急购电规则可知不同方向预测误差损失不对称，说明仅用 MAE、RMSE 等统计指标不足以评价预测价值。据此建立两阶段随机 MILP，将预测不确定性直接传递至购电与储能决策，并通过实际购电费用和紧急购电量评价预测方案。

2.3 问题三的分析

问题三在每日 0:00、6:00、12:00 和 18:00 持续获得新的未来 24 小时光伏预报，使决策信息随时间逐步增加。同时，购电计划的修改需要承担调增、调减等额外结算费用，因此问题的核心转化为新增预测信息与调整成本之间的动态权衡。

本文将不同预报时点对应的信息表示为渐进信息集

$$\mathcal{I}_0 \subseteq \mathcal{I}_6 \subseteq \mathcal{I}_{12} \subseteq \mathcal{I}_{18}.$$

首先分析官方光伏预报误差随提前期的变化规律，并构建提前期分段的官方预报-季节基线融合预测；随后在每个信息更新时点冻结已经执行的决策，仅重新优化尚未执行的购电与储能计划，从而建立多时点序贯滚动重优化模型。

为评价新增预报的经济贡献，设置 $S_0 = \{0\}$ 、 $S_1 = \{0, 6\}$ 、 $S_2 = \{0, 6, 12\}$ 和 $S_3 = \{0, 6, 12, 18\}$ 四组信息策略，并以相邻策略成本差定义实现边际信息价值。进一步构造使用未来真实负荷与光伏的完全信息离线下界，用于量化现实调度方案与理想信息条件之间的成本差距；同时区分理论信息价值与实际路径上的实现信息价值，并采用 Wilcoxon 符号秩检验和 Bootstrap 重采样对滚动更新收益的日级稳定性进行检验。

2.4 问题四的分析

问题四引入实时波动电价，使储能充放电同时受源荷不确定性、跨期 SOC 状态与时变价格机会成本共同影响。本文不重新建模，而是将附件 4 给出的 10 min 动态电价分别引入问题二的随机优化框架与问题三的序贯滚动框架：问题 4-2 将计划购电与紧急购电费用按对应时段动态电价计算；问题 4-3 进一步将计划购电、调增、调减及紧急购电结算价格全部替换为相应时刻动态价格，以研究价格波动对滚动信息价值的影响。

此外，为检验模型稳定性，进一步改变电价波动强度与预测误差规模，考察二者对总费用、紧急购电量及滚动策略经济性的影响，评价所建序贯调度框架的鲁棒性。

针对以上问题，本文采用图 1 所示整体思路框架进行系统性研究和分析。

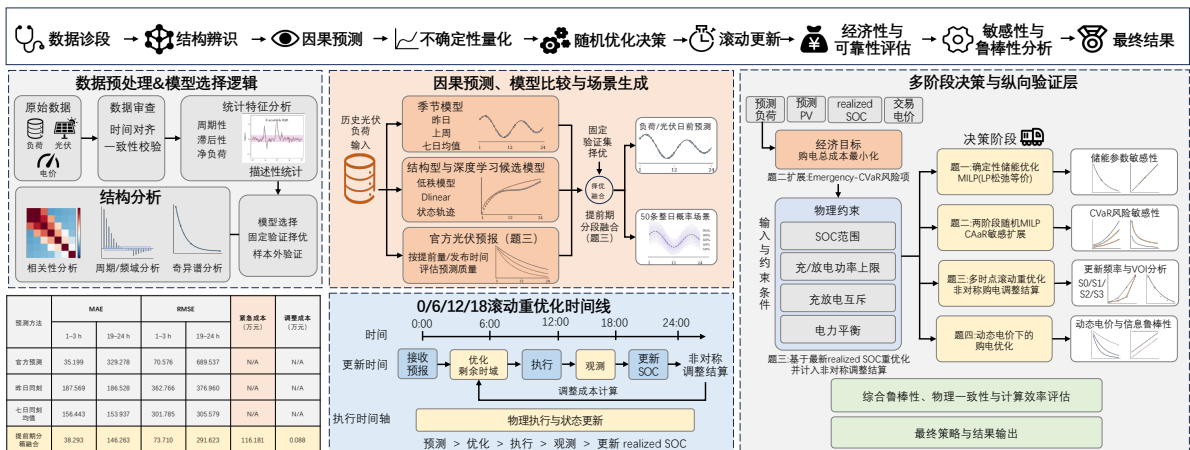


图 1 微网储能序贯调度的总体研究框架

三、 模型假设

1. 将微网视为单母线系统，忽略线路损耗及其他未给出的网络约束。
2. 以 10 min 为统一决策时间尺度，并认为各时段内负荷、光伏功率及电价保持不变。
3. 储能充、放电效率均取 90%，研究期内容量、功率上限及效率参数保持不变，暂不考虑自放电与电池老化。
4. 微网仅允许从外网购电，不考虑反向售电；当光伏无法被负荷或储能完全消纳时允许弃光。
5. 各时刻决策严格遵循信息到达顺序，未来真实负荷与光伏出力不参与当前决策；问题四中附件 4 电价视为外生给定价格序列。

四、 符号说明

符号	含义	单位
t, d, s	日内时段、运营日、成对场景索引	—
τ, h	日内发布时间及预测提前期	h
Δt	10 分钟时间步长, $\Delta t = 1/6$	h
$L_t, \hat{L}_t$	实际负荷与负荷预测	kW
$P_t, \hat{P}_t$	实际光伏与光伏预测	kW
$N_t, \hat{N}_t$	实际净负荷与净负荷预测	kW
$L_{t,s}, P_{t,s}$	场景 s 的负荷与光伏	kW
p_t	外网普通购电价格	元/kWh
$p_t^{(\gamma)}$	保持均价不变、按 γ 缩放波动后的电价	元/kWh
E_t, E_t^{act}	计划与实际储能电量	kWh
$E_{\min}, E_{\max}$	储能电量上下界	kWh
$c_t, d_t, P_{\max}$	充电功率、放电功率及其上限	kW
η_c, η_d, u_t	充电效率、放电效率、二元充电模式	—
$g_t, x_{t,s}, e_{t,s}$	计划购电、场景调用计划电、场景紧急购电功率	kW
$w_t, w_{t,s}$	确定性与场景弃光功率	kW
$g_t^{(0)}, \Delta g_t^+, \Delta g_t^-$	零点购电承诺、增购功率、减购功率	kW
$V_d(E)$	日末储能的跨日延续价值	元
λ_r, α	风险权重（主方案为 0）、CVaR 置信水平（0.90）	—
$\zeta_d, \text{CVaR}_{\alpha,d}$	VaR 辅助阈值、日紧急购电费用 CVaR	元
S_0, S_1, S_2, S_3	更新时间表: $\{0\}, \{0, 6\}, \{0, 6, 12\}, \{0, 6, 12, 18\}$	h
$C(S_i), \text{VOI}(S_i)$	实际总费用、相对 S_0 的信息价值	元
ΔVOI_i	相邻更新时间表的增量信息价值	元
γ, κ	问题四的价格波动系数、成对预测残差幅度系数	—

五、 问题一：确定性最优调度

5.1 能量流与储能状态模型

附件 1 给出完整单日的负荷、光伏和电价，故本问不引入预测误差。将全天划分为 $\mathcal{T} = \{0, 1, \dots, 143\}$ 共 144 个 10 分钟区间， $\Delta t = 1/6$ h。以 g_t, c_t, d_t, w_t 分别表示外网购电、充电、放电和弃光功率， E_t 表示区间 t 起点的储能电量（kWh），而非比例 SOC。每一时段满足

$$g_t + P_t + d_t = L_t + c_t + w_t, \quad (1)$$

其中 $0 \leq w_t \leq P_t$ ，即弃光只能来自当期光伏。储能状态按实际充、放电效率递推：

$$E_{t+1} = E_t + \eta_c c_t \Delta t - \frac{d_t \Delta t}{\eta_d}. \quad (2)$$

题设参数为 $\eta_c = \eta_d = 0.9$ 、 $P_{\max} = 5000$ kW，且 $1200 \leq E_t \leq 10800$ kWh。问题一采用单日闭环条件 $E_0 = E_{144} = 6000$ kWh，以排除通过透支期末电量降低当日费用的方案。

5.2 确定性储能–购电 MILP

引入二元变量 u_t 表示充电状态； $u_t = 1$ 时允许充电， $u_t = 0$ 时允许放电。问题一的正式模型为

$$\begin{aligned} \min \quad & \sum_{t \in \mathcal{T}} p_t g_t \Delta t \\ \text{s.t.} \quad & \begin{cases} g_t + P_t + d_t = L_t + c_t + w_t, & t \in \mathcal{T}, \\ E_{t+1} = E_t + \eta_c c_t \Delta t - d_t \Delta t / \eta_d, & t \in \mathcal{T}, \\ E_{\min} \leq E_t \leq E_{\max}, & t = 0, \dots, 144, \\ 0 \leq c_t \leq P_{\max} u_t, & t \in \mathcal{T}, \\ 0 \leq d_t \leq P_{\max} (1 - u_t), & t \in \mathcal{T}, \\ u_t \in \{0, 1\}, & t \in \mathcal{T}, \\ g_t \geq 0, \quad 0 \leq w_t \leq P_t, & t \in \mathcal{T}, \\ E_0 = E_{144} = 6000. \end{cases} \end{aligned} \quad (3)$$

目标函数只计外网购电支出，不含售电收益、充放电补贴、循环奖励或电池退化成本。二元变量仅用于排除同一 10 分钟区间内的同时充、放电；正式结果采用 CBC 求解该 MILP。

5.3 充放电互补约束的条件性精确松弛

为解释互补约束在本模型中的作用，考虑删除 u_t 并分别保留 $0 \leq c_t, d_t \leq P_{\max}$ 所得的 LP 松弛。记 $\alpha = \eta_c \eta_d < 1$ 。

命题 1（可行交换条件下的互补精确松弛）若该 LP 存在一个最优解，使其中每个 $c_t > 0, d_t > 0$ 的时段均满足

$$g_t + P_t - w_t \geq (1 - \alpha) \min \left\{ c_t, \frac{d_t}{\alpha} \right\}, \quad (4)$$

则至少存在一个满足 $c_t d_t = 0$ 的 LP 最优解，因而该条件下 MILP 与 LP 的最优目标值相同。

证明概要：对任一同时充、放电时段，令 $\delta = \min\{c_t, d_t/\alpha\}$ 、 $r = (1 - \alpha)\delta$ 。作如下变换：

$$c'_t = c_t - \delta, \quad d'_t = d_t - \alpha\delta. \quad (5)$$

由于 $\alpha/\eta_d = \eta_c$ ，式(5)保持该时段的储能增量不变，故全部后续 E_t 及终端条件均不变；同时至少有 c'_t, d'_t 之一为 0。该变换在功率平衡中释放 r 的供给余量。由式(4)，可将其分解为减少购电 a_t 和增加弃光 b_t ，其中 $a_t + b_t = r$ 、 $0 \leq a_t \leq g_t$ 、 $0 \leq b_t \leq P_t - w_t$ 。于

是功率平衡、变量边界和功率上限仍成立，而目标函数变化为 $-p_t a_t \Delta t \leq 0$ 。逐时重复即可得到互斥可行解；再结合 LP 是 MILP 的松弛，得到两者最优值相等。完整交换论证见附录1.1。

非负电价、无售电收益和无循环补贴保证上述交换不会增加费用；式 (4)则是保持外网非负与弃光上界所需的可行性条件。因此，该命题是条件性结论，并不声称所有 LP 可行解或最优解都会自然互斥。

5.4 LP-MILP 数值交叉验证

对附件 1 实例，正式 MILP 与仅删除二元模式变量及两条互斥约束的 LP 使用完全相同的功率平衡、储能递推、功率与电量边界、弃光约束和首末电量。表1中的数据直接来自同口径审计结果。两者目标值均为 35126.948591 元，绝对与相对松弛间隙均为

表 1 附件 1 单日算例的 MILP 与 LP 松弛同口径对照

指标	正式 MILP	LP 松弛
目标值/元	35126.948591	35126.948591
外网购电量/kWh	59482.699	59482.699
充电量/kWh	20740.666	20740.666
放电量/kWh	16799.940	16799.940
弃光量/kWh	0	0
储能电量范围/kWh	1200–10800	1200–10800
最大同时充放电功率/kW	0	0
同时充放电区间数	0	0
单次求解用时/s	0.211	0.067

两次求解使用相同输入和物理约束；绝对松弛间隙为 0 元。用时为已保存的单次测量，不表示重复实验的均值或置信区间。

0；LP 最大同时充放电功率为 0，求解用时由 0.211 s 降至 0.067 s，单次计时比约为 3.1。两解仅在 5 个等价时段的功率分配存在数值差异，其中 3 个时段的差异超过 10^{-3} ，但全天购电、充电、放电、弃光及 SOC 边界完全一致。由此，理论结构解释与数值证书共同表明：LP 松弛在附件 1 这一确定性单日算例上精确，但该结论不推广至问题二至四的随机、多阶段或滚动模型；正式主模型仍为 MILP。

5.5 效率修正的局部套利判据

考虑仅由价差驱动的两个理想化时段 i 和 j 。若时段 i 从外网额外购买 1 kWh 用于充电，则进入储能 η_c kWh，未来能够向负荷释放 $\eta_c \eta_d$ kWh。忽略功率和电量边界等耦合约束时，正的纯价格套利空间要求

$$p_j \eta_c \eta_d > p_i, \quad \text{即} \quad \frac{p_j}{p_i} > \frac{1}{\eta_c \eta_d}. \quad (6)$$

在 $\eta_c = \eta_d = 0.9$ 时, 阈值为 $1/0.9^2 \approx 1.235$ 。这是理想化、局部的效率修正套利判据, 并非全局最优策略的充分必要条件; 实际决策还同时受 SOC 边界、功率上限、光伏出力、负荷需求和终端 SOC 约束影响。图2显示充电主要位于低价或光伏富余时段, 放电主要位于较高价格时段, 实际轨迹与上述经济机制总体一致。

5.6 求解结果与灵敏度

无储能对照按 $g_t = (L_t - P_t)_+$ 购电、 $w_t = (P_t - L_t)_+$ 弃光。表2在相同负荷、光伏和电价下比较经济收益与物理可行性。正式 MILP 达到 **Optimal**, 购电费用为 35126.948591 元, 较无储能对照的 48052.047 元减少 12925.098 元, 节费率为 26.898%; 外网购电量为 59482.699 kWh, 原有 6247.963 kWh 弃光被全部消纳, 储能电量始终处于 1200–10800 kWh。

表 2 问题一经济收益与物理核验

指标	无储能对照	正式 MILP	改善或核验
购电费用/元	48052.047	35126.948591	节省 12925.098
节费率	—	—	26.898%
外网购电量/kWh	61789.935	59482.699	减少 2307.236
弃光量/kWh	6247.963	0	全部消纳
储能电量/kWh	不适用	1200–10800	首末均为 6000
最大功率平衡残差/kW	—	4.0×10^{-5}	通过
最大电量递推残差/kWh	—	4.7×10^{-4}	通过

题目指定区间购电量 $q_t = g_t \Delta t$ 见表3, 六个 4 小时区间的充放电量见表4, 所有电量单位均为 kWh。

表 3 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.000	12:00–12:10	486.403	14:00–14:10	0.000
16:00–16:10	394.931	18:00–18:10	636.983	20:00–20:10	0.000
全天购电量	59482.699 kWh	全天购电费	35126.949 元		

全天累计充电 20740.666 kWh, 累计放电 16799.940 kWh; 二者差异来自充、放电效率损耗。六个 4 小时区间的储能吞吐量如下。

采用单因素分析, 效率取 $\eta_c = \eta_d \in \{0.80, 0.85, 0.90, 0.95\}$, 容量及功率上限各取基准的 0.50, 0.75, 1.00, 1.25 倍, 其余条件不变; 容量变化时, 电量下界、上界和首末值分别保持为容量的 10%、90%、50%。12 个情形均达到最优并通过物理核验, 结果见图3。单程效率从 0.80 增至 0.95, 费用下降 11.66%。容量每增加 3000 kWh, 费用依次减少 2850.82、2042.64、1570.20 元; 功率每增加 1250 kW, 依次减少 1043.97、219.92、53.93 元, 均呈边际收益递减。从基准向相邻较高设定的有限步长归一化敏感度看, 效率损耗

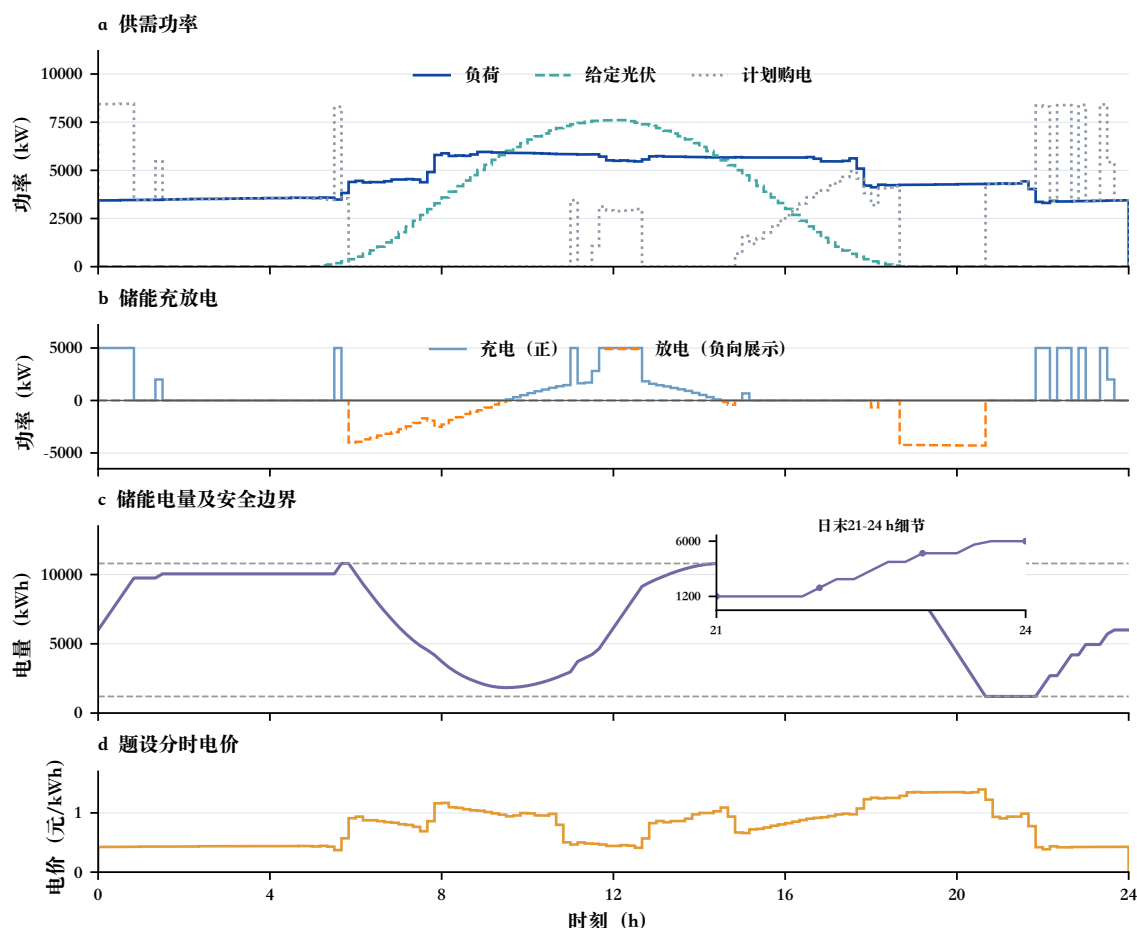


图 2 附件 1 确定性最优轨迹：供需功率、充放电、储能电量与电价。放电以负向展示以区分动作；虚线为电量边界，嵌图放大 21–24 时的原轨迹；各曲线均为单次确定性解，无误差带。

表 4 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4500.000	0.000	4:00–8:00	833.333	6365.841
8:00–12:00	4787.964	1702.997	12:00–16:00	5286.035	91.101
16:00–20:00	0.000	5780.132	20:00–24:00	5333.333	2859.868
0:00 储电量	6000.000 kWh	24:00 储电量	6000.000 kWh		

影响最大、容量次之，5000 kW 以上继续增加功率的收益已很小。该判断限于本日输入和所测参数区间，不等价于投资最优结论。

问题一建立的变量–约束式储能调度骨架将在后续问题中保持不变，并通过增加不确定场景、多时点结算及动态价格机制逐级扩展。

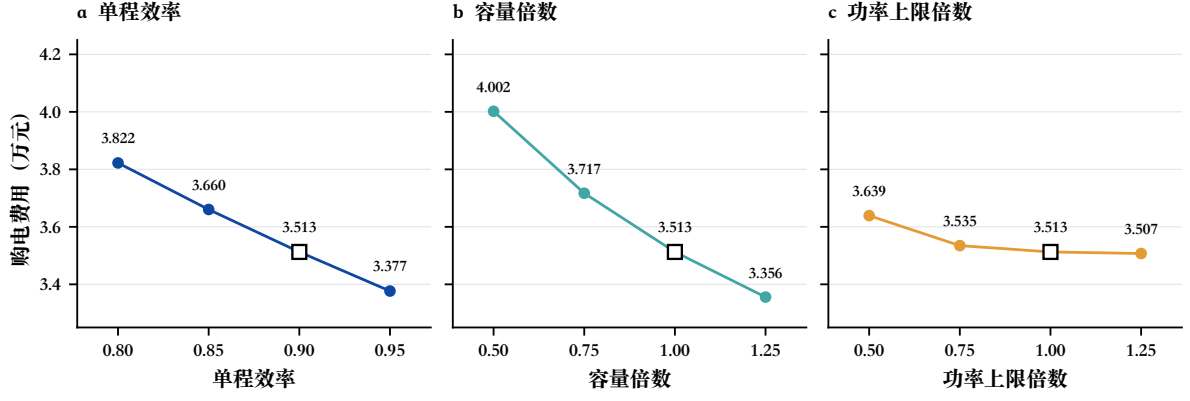


图3 单因素灵敏度。圆点为已求解的确定性情形，空心方框为基准配置，连线仅辅助比较；纵轴为明示的局部费用尺度，不设误差棒。

六、 问题二：不确定条件下的日前调度

6.1 预测为何必须服务于决策

令净负荷 $N_t = L_t - P_t$ ，预测误差 $e_t = \hat{N}_t - N_t$ 。MAE、RMSE 对误差方向近似对称，但本题结算并不对称： $e_t < 0$ 可能造成购电不足，缺口须按 $5p_t$ 紧急购电； $e_t > 0$ 则主要造成偏保守的日前购电。因此

$$\text{预测误差小} \not\Rightarrow \text{实际购电成本低}, \quad \ell(e_t) \neq \ell(-e_t).$$

这与决策导向的 Predict-then-Optimize 观点一致：预测误差与预测所诱导的优化损失并不等价 [12]。本文并未训练 SPO+ 或端到端决策模型，而是采用“因果预测-残差场景-随机优化-实际回放”闭环，同时评价预测精度与经济后果。

上述不对称性可由一个单时段局部模型严格刻画。在净负荷与计划购电均非负的供电缺口区间，暂忽略储能的跨时段作用，令计划购电恰为 $\hat{N}_t$ ，未用完的计划电仍按 p_t 支付，供能缺口按 $5p_t$ 补购。全信息费用为 $C_t^* = p_t N_t$ 。若 $e_t \geq 0$ ，计划偏高，费用遗憾为 $p_t e_t$ ；若 $e_t < 0$ ，则

$$p_t \hat{N}_t + 5p_t (N_t - \hat{N}_t) - p_t N_t = 4p_t (-e_t).$$

记 $(x)_+ = \max\{x, 0\}$ ，得到局部决策遗憾

$$R_t(e_t) = p_t (e_t)_+ + 4p_t (-e_t)_+. \quad (7)$$

因此，在绝对误差相同的情况下，低估净负荷的局部边际暴露是高估的 4 倍。式 (7) 只解释题设结算规则的方向性：实际全年调度还受储能、SOC、场景对冲和跨时段耦合影响，不能把真实总费用逐时分解为该式。

6.2 严格因果预测与模型选择

问题二在 0:00 给出目标日 144 个光伏点预测。模型选择仅使用 2025 年 1 月 18–31 日：按验证 RMSE 最小、MAE 并列判据确定模型；2025 年 2 月 1 日至 12 月 31 日的 334 天只作冻结检验。任一预测日的输入、标准化量和残差均来自该日 0:00 以前，正式期实测值不反向调参。比较昨日同刻、上周同刻、近 7 日同刻均值、相位、SSA、DLinear[5] 及两种已有结构融合方法；表5直接由保存结果生成。

表 5 光伏预测的因果比较：一月选型，2–12 月检验

模型	验证 MAE	验证 RMSE	正式期 MAE	正式期 RMSE
昨日同刻	125.244	272.937	185.346	375.175
上周同刻	143.422	291.266	214.688	419.948
近 7 日同刻均值	101.382	213.551	153.400	303.858
相位模型	116.045	242.959	170.571	341.373
SSA	219.383	334.147	260.834	393.833
DLinear	151.795	261.481	423.397	692.692
结构/残差混合	872.672	1397.551	1400.663	1993.198
相位/结构融合	116.045	242.959	170.571	341.373

误差单位均为 kW。验证期为 1 月 18–31 日；模型仅按验证 RMSE 最小、MAE 并列判据选择。

近 7 日同刻均值 $\hat{P}_{d,t} = 7^{-1} \sum_{j=1}^7 P_{d-j,t}$ 以验证 RMSE 213.551 kW 入选；正式期 MAE、RMSE 分别为 153.400、303.858 kW。更复杂模型没有提高统一因果验证精度，故不将 Hankel/Vandermonde 或神经网络作为正式预测器。进一步以运营日为配对单位，对保存的 334 个日 MAE 差作双侧 Wilcoxon 符号秩检验，并对预先确定的 5 项比较作 Benjamini–Hochberg 校正 [9, 10]。五项均在 5% FDR 下显著；图4给出效应方向和离散程度，点为日差值中位数，线段为四分位距，不是置信区间。

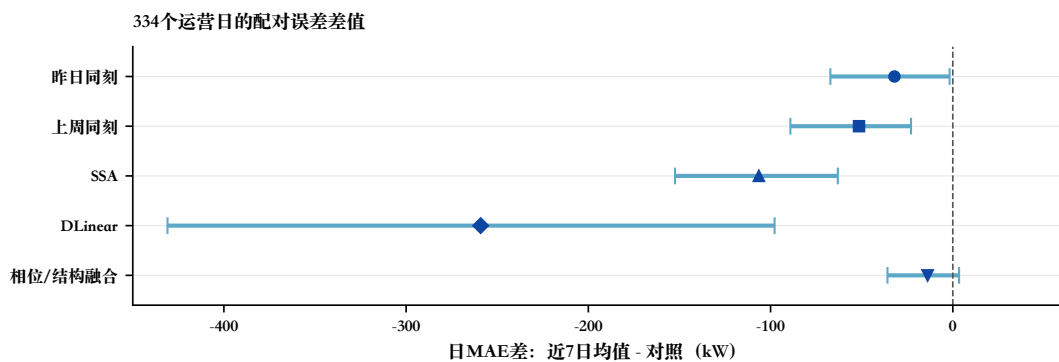


图 4 近 7 日同刻均值相对五种对照的配对日 MAE 差。负值表示本文正式预测较优；点为 334 日中位数，横线为第 25–75 百分位，统计单位为日。

尽管附件提供全年实测数据，逐日决策严格遵循信息到达顺序。某日真实负荷和光伏仅在相应区间发生后用于执行、结算、状态更新与评价，不参与该日 0:00 计划。历史残差池也只纳入当时已经观测完成的日误差，这一逐步评价原则与 prequential 思想一致 [6]。

6.3 成对残差场景与两阶段随机 MILP

点预测无法表达误差方向、连续性及负荷–光伏联合变化。对每一正式日，从该时点以前已经完成的历史日中成对抽取负荷和光伏整日残差，叠加于当日点预测并截取非负值，构造 $S = 50$ 条等概率场景；同一 s 始终对应同一历史残差日，保留日内相关性和联合依赖。场景构造不假设正态、时段独立或负荷与光伏独立。这一场景层正是将式(7)所揭示的下行暴露传递至决策端：它不仅保留误差幅度，还保留整日时间依赖和负荷–光伏联合依赖，使优化器能够权衡计划购电与紧急补购风险。

日前变量 g_t, c_t, d_t, E_t, u_t 在场景间共享；场景 s 的计划电调用、紧急购电和弃光分别为 $x_{t,s}, e_{t,s}, w_{t,s}$ 。令 $V_d(E_{145})$ 为日末延续价值，基础模型为

$$\begin{aligned} \min \quad & \sum_t p_t g_t \Delta t + \frac{1}{S} \sum_s \sum_t 5 p_t e_{t,s} \Delta t - V_d(E_{145}) \\ \text{s.t.} \quad & \begin{cases} x_{t,s} + e_{t,s} + P_{t,s} + d_t = L_{t,s} + c_t + w_{t,s}, \\ 0 \leq x_{t,s} \leq g_t, \quad e_{t,s} \geq 0, \quad 0 \leq w_{t,s} \leq P_{t,s}, \\ E_{t+1} = E_t + \eta_c c_t \Delta t - d_t \Delta t / \eta_d, \\ E_{\min} \leq E_t \leq E_{\max}, \\ 0 \leq c_t \leq P_{\max} u_t, \quad 0 \leq d_t \leq P_{\max} (1 - u_t), \quad u_t \in \{0, 1\}. \end{cases} \end{aligned} \quad (8)$$

其中 $t = 1, \dots, 144$ 、 $s = 1, \dots, 50$ ，储能参数沿用问题一。 $g_t - x_{t,s}$ 是已经购买但场景中未调用的计划电，不是物理注入，也不等同于弃光。

为避免单日视界末端无代价放空， V_d 采用三段边际价值递减的凹分段线性函数。第一断点为 $E_{\min}$ 加“场景净负荷相对点预测的正误差日能量”80%分位数，第二断点为其与 $E_{\max}$ 的中点，第三断点为 $E_{\max}$ ；三段边际价值依次为 $\eta_d Q_{0.9}(p)$ 、 $\eta_d Q_{0.5}(p)$ 、 $\eta_d Q_{0.1}(p)$ 。该项只用于决策，不计入费用结算；问题二不强制每天回到 6000 kWh。

6.4 实际执行、费用与风险扩展

计划确定后，以附件 2 实测序列顺序回放。实际充放电不超过计划动作，并受当期可消纳功率、可用电量和容量边界限制；缺口最后由紧急购电补足。实际状态满足

$$E_{t+1}^{\text{act}} = E_t^{\text{act}} + \eta_c c_t^{\text{act}} \Delta t - d_t^{\text{act}} \Delta t / \eta_d, \quad E_{d+1,1}^{\text{act}} = E_{d,145}^{\text{act}}.$$

首日初值为 6000 kWh，后续传递上一日实际终值，不存在未吸收放电或隐藏能量汇。334/334 个日模型均最优，期望口径与实际结算见表6。主方案 $\lambda_r = 0$ 的实际紧急购电量为 398161.900954 kWh，实际总费用为 14839462.002357 元。最大实际功率平衡残差为 1.00×10^{-6} kW，实际电量递推残差为 9.84×10^{-6} kWh；实际 E_t 处于 1200–10800 kWh，跨日衔接通过审计。

表 6 问题二期望成本主方案的费用与能量核验

指标	模型内期望	实际结算
计划购电费/元	13323201.223427	13323201.223427
紧急购电费/元	1051144.022760	1516260.778930
总购电费/元	14374345.246186	14839462.002357
紧急购电量/kWh	277524.659704	398161.900954
未使用计划电量/kWh	1304219.205	1513828.453
实际储能电量/kWh	—	1200–10800
未吸收放电/kWh	—	0

334/334 日求解最优；风险权重为 0。期望与实际为两种评价口径，并非互为基准的两种策略。延续价值不计入结算。

风险敏感扩展仅把场景紧急购电费用 $C_{d,s}^{\text{emg}} = \sum_t 5p_{t,s}e_{t,s}\Delta t$ 作为风险损失。取 $\alpha = 0.90$,

$$\text{CVaR}_{\alpha,d} = \zeta_d + \frac{1}{(1-\alpha)S} \sum_s \xi_{d,s}, \quad \xi_{d,s} \geq C_{d,s}^{\text{emg}} - \zeta_d, \quad \xi_{d,s} \geq 0,$$

并在式(8)目标中加入 $\lambda_r \text{CVaR}_{\alpha,d}$ 。计划购电费不进入 CVaR 损失； $\lambda_r = 0$ 仍为主结果。图5表明,提高风险权重降低紧急尾部暴露但提高期望运行费用。作为温和示例, $\lambda_r = 0.05$ 使期望运行费用增加约 0.093%，紧急电量和 CVaR 分别下降约 13.75%、13.39%；实际费用事后下降 0.119%，该事后结果不用于宣称其事前最优。CVaR 线性化参照 [11]。

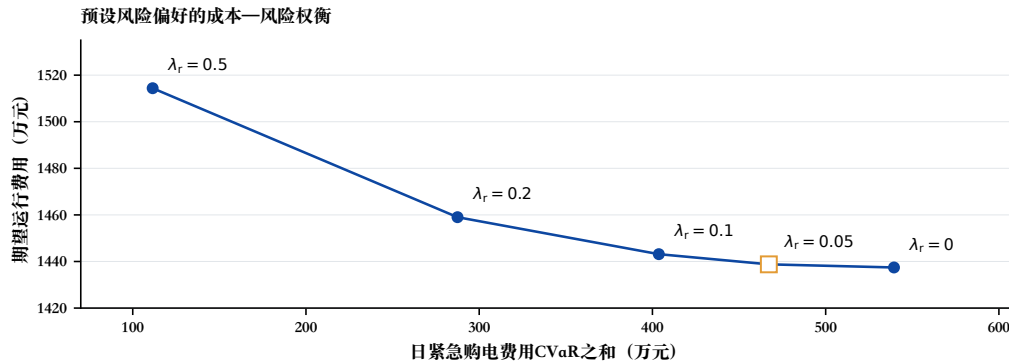


图 5 预设风险偏好的成本—风险权衡。横轴为 334 个日模型紧急购电费用 CVaR 之和，纵轴为期望运行费用；方框标出温和示例 $\lambda_r = 0.05$ ，无统计误差棒。

题面表 3 所需的指定日计划购电量与紧急购电量按 144 个 10 分钟区间完整输出；详见附件结果文件，正文不重复展开高频明细。

七、 问题三：信息逐步到达下的滚动调度

7.1 多发布时间预测与因果融合

附件 3 每天在 0:00、6:00、12:00、18:00 发布未来 24 个整点光伏预测。若发布时间为 τ ，第 h 个节点的目标时刻为 $t^{\text{tar}} = \tau + h$ ， $h = 1, \dots, 24$ 。原始数据包含 1460 个完整批次、35040 个节点；年末 36 个目标超出附件 2 实测范围，评价时记录为越界而不强行

配对。每一批次独立用 PCHIP 映射到 10 分钟网格，保持小时节点、单调性和局部极值，不跨批次使用未来实测值 [3]。

图6a 显示，官方预测 MAE 从 1–3 h 的 34.52 kW 增至 19–24 h 的 319.51 kW。因此使用 2025 年 1 月 8–16 日拟合、1 月 17–30 日选型，把官方预测与近 7 日同刻均值按提前期分箱融合：

$$\hat{P}_{\tau,h} = \theta_b \hat{P}_{\tau,h}^{\text{off}} + (1 - \theta_b) \hat{P}_{\tau,h}^{\text{sea}}, \quad 0 \leq \theta_b \leq 1.$$

五个分箱的官方权重依次为 0.871、0.450、0.232、0.033、0.089。短提前期偏重最新官方信息，长提前期偏重稳定季节项。冻结后在 2–12 月样本外评价，融合 MAE、RMSE 为 121.73、252.99 kW；图6b 按相同样本比较官方与融合 RMSE。权重只由已经获得真值的目标更新，未来实测扰动不会改变已发布预测。

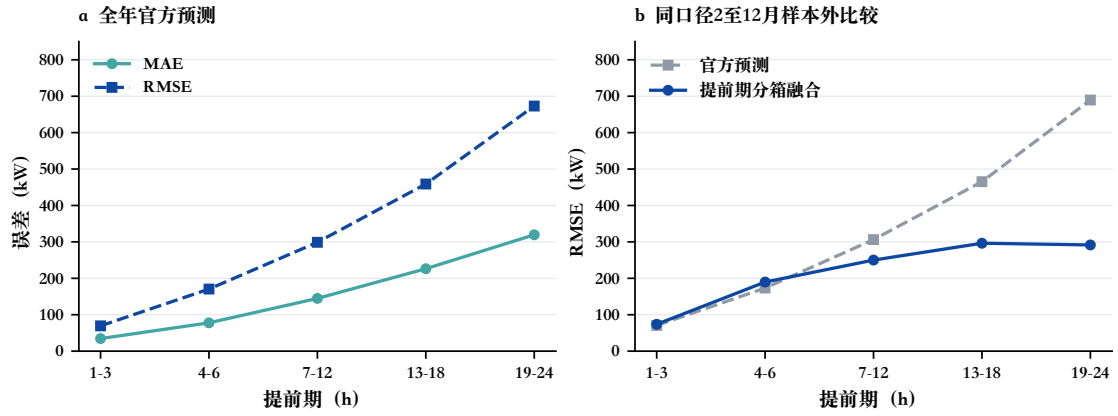


图 6 官方光伏预测误差与提前期分箱融合。**a**，35004 个可匹配小时目标的全年汇总；**b**，2–12 月共同样本的 RMSE。各点为分箱汇总指标，不是独立重复，故不添加误差棒。

7.2 滚动重优化与结算

定义

$$S_0 = \{0\}, \quad S_1 = \{0, 6\}, \quad S_2 = \{0, 6, 12\}, \quad S_3 = \{0, 6, 12, 18\}.$$

时刻 τ 获得新预测后，已经执行的区间冻结，只开放未来区间。实际储能电量作为下一次优化的必需状态，不能以计划电量或固定 6000 kWh 替代。每次更新仍使用 50 条因果成对整日残差场景，并围绕新融合预测重新定中心；风险权重固定 $\lambda_r = 0$ ，使四种安排仅因信息时刻不同。

设更新前后承诺为 $g_t^{\text{old}}, g_t^{\text{new}}$ ， $g_t^{\text{new}} - g_t^{\text{old}} = \Delta g_t^+ - \Delta g_t^-$ ， $\Delta g_t^\pm \geq 0$ 。0:00 采用问题二随机 MILP；后续仅在 $t > \tau$ 上最小化

$$\sum_{t>\tau} p_t \Delta t (1.5 \Delta g_t^+ - 0.5 \Delta g_t^-) + \frac{1}{50} \sum_s \sum_{t>\tau} 5 p_t e_{t,s} \Delta t - V_d(E_{145}).$$

增购按 $1.5p_t$ 支付，减购只返还 $0.5p_t$ ；已经支付的承诺为沉没成本。每次只执行到下一发布时间，实际动作、功率平衡、SOC 递推及跨日传递完全沿用问题二的物理执行规则。主结果按“相对上一承诺”逐次结算；“均相对 0:00 承诺”仅作歧义敏感性分析。

7.3 经济结果与信息价值

334 天的全部滚动日模型均返回最优状态。表7区分实际总费用、紧急购电量和相对 S_0 的信息价值； S_0 使用问题三的官方-季节融合预测，并不等于问题二主结果。

表 7 问题三四种更新时间表的实际结算与信息价值

安排	实际总费用/元	紧急电量/kWh	相对 S0 节费/元
S0	14727970.596431	366345.313	0.000
S1	14632849.794731	332290.996	95120.802
S2	14505500.731613	302254.536	222469.865
S3	14507718.144450	302356.180	220252.452

正式期为 2 月 1 日至 12 月 31 日。S0 使用问题三的官方预报融合信息，不是问题二主方案的复现。

先区分理论信息价值与本文的事后运营信息价值。令 $\mathcal{I}_0 \subseteq \mathcal{I}_6 \subseteq \mathcal{I}_{12} \subseteq \mathcal{I}_{18}$ 为嵌套信息集， $\Pi(\mathcal{I})$ 为可依据信息 $\mathcal{I}$ 制定的策略集合，并定义事前最优期望费用 $V(\mathcal{I}) = \inf_{\pi \in \Pi(\mathcal{I})} \mathbb{E}[C(\pi)]$ 。若新增信息可以被忽略，且使用真实条件分布作事前优化，则 $\mathcal{I}_a \subseteq \mathcal{I}_b$ 蕴含 $\Pi(\mathcal{I}_a) \subseteq \Pi(\mathcal{I}_b)$ ，从而

$$V(\mathcal{I}_b) \leq V(\mathcal{I}_a), \quad \text{VOI}^{\text{th}}(\mathcal{I}_b | \mathcal{I}_a) = V(\mathcal{I}_a) - V(\mathcal{I}_b) \geq 0. \quad (9)$$

证明只需注意：拥有较大信息集的决策者可以忽略新增信息并复用原策略，因此其可行策略集不缩小。

本文实际计算的不是式(9)中的完美信息期望值，而是固定预测器、有限年度真实路径、滚动承诺与非对称结算共同作用下的运营信息价值，定义为

$$\text{VOI}^{\text{op}}(S_i) = C^{\text{real}}(S_0) - C^{\text{real}}(S_i), \quad \Delta \text{VOI}_i^{\text{op}} = C^{\text{real}}(S_{i-1}) - C^{\text{real}}(S_i).$$

四种安排的实际总费用依次为 14727970.596431、14632849.794731、14505500.731613、14507718.144450 元。 $S_2 = \{0, 6, 12\}$ 最低，相对 S_0 节省 222469.864818 元（1.5105%）；紧急购电量由 366345.312534 降至 302254.535714 kWh，下降 17.4946%。图7同时保留费用从零起算的全尺度图和标明“局部放大”的细节图，避免用截断坐标夸大微小差异。

06:00 与 12:00 的增量 VOI 分别为 +95120.801699、+127349.063118 元，18:00 为 -2217.412836 元且未截断。这仅表明在既定预测、结算和储能约束下，该年度路径上额外的不完美预测更新未产生正的边际运营收益，不表示更多信息本身有害，也不表示 18:00 预测本身更差；更多更新不保证经济价值单调增加。两种结算解释都保持 $S_2 < S_3 < S_1 < S_0$ ，同一安排的年度费用最大差异仅 516.002078 元，故主要排序不受该歧义

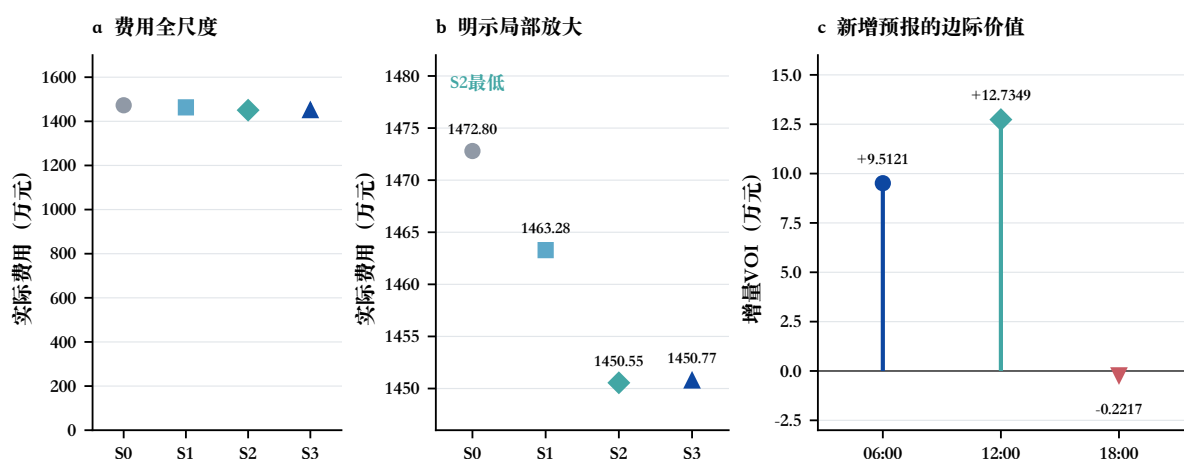


图 7 问题三经济结果。**a**，年度实际费用零起点全尺度；**b**，同一数据的明示局部放大；**c**，新增发布时间的增量信息价值，水平线为零。三者均读取正式结果表，无误差棒。

影响，无需另图。

为检验年度节省是否由少数极端日期驱动，进一步以运营日为配对单位比较 S_0 与 S_2 ，结果见表8。334 个日差的均值和中位数分别为 666.077 元和 376.660 元，84.13% 的日期取得正节省；日均节省的 95% 非参数 bootstrap 区间为 [546.361, 792.543] 元。双侧 Wilcoxon 符号秩检验统计量为 3911， $p = 2.865 \times 10^{-42}$ 。该结果与效应量共同说明优势并非仅由少数日期贡献；但相邻日期可能存在序列相关，推断结果仍应结合年度总量和日差分布解释。

表 8 S_2 相对 S_0 的日级配对经济收益

统计量	数值
配对运营日数	334
日节省均值/元	666.077
日节省中位数/元	376.660
四分位区间/元	[69.917, 804.840]
正节省日期占比	84.13%
Wilcoxon 统计量	3911
双侧 p 值	2.865×10^{-42}
日均节省 95% bootstrap 区间/元	[546.361, 792.543]

统计单位为运营日；双侧 Wilcoxon 符号秩检验采用 wilcox 零差处理。Bootstrap 对 334 个配对日差有放回重采样 10000 次，随机种子为 2026。

物理审计中，最大实际功率平衡和电量递推残差分别不超过 1.82×10^{-12} kW 与 2.28×10^{-13} kWh；跨日电量、已执行区间冻结误差和未吸收放电均为 0。按题意提交的 result3.xlsx 采用包含全部四次发布的 S_3 ，而 S_2 的经济优势作为信息频率比较结论单独报告。

八、 问题四：动态电价下的调度扩展

8.1 动态电价特征与信息口径

附件 4 给出 2025 年 1 月 1 日至 12 月 31 日的 10 分钟外网电价，单位为元/kWh。按区间结束时刻展开后共 52560 条记录，连续覆盖 2025 年 1 月 1 日 00:10 至 2026 年 1 月 1 日 00:00，每日 144 条；不存在缺失、重复、负值或零值，并与负荷和光伏运行网格逐点一致。极端价格是实际市场信号，故仅作核验而不删除或缩尾。

表 9 附件 4 动态电价的描述性统计

指标	均值	标准差	最小值	第一四分位数	中位数	第三四分位数	最大值	样本数
电价/(元/kWh)	0.766	0.341	0.008	0.454	0.738	1.027	1.794	52560

全年电价均值、标准差及范围分别为 0.766、0.341 和 [0.0076, 1.7936] 元/kWh。图8表明价格水平和离散程度随月份变化；全年日内价格极差的日均值为 1.107 元/kWh，相邻 10 分钟价格最大绝对变化为 0.4996 元/kWh。

对 144 个日内时段计算中位数与四分位区间，见图9。按 24 个小时均价的上下四分位数划分，0:00–4:00 和 22:00–24:00 为样本低价小时，8:00–10:00 和 17:00–21:00 为样本高价小时；该划分只是数据描述，不代表预设峰谷电价制度。

净负荷仍定义为 $N_t = L_t - P_t$ 且不截断。电价与负荷的 Pearson、Spearman 系数分别为 0.508 和 0.580，与净负荷的两类系数分别为 0.260 和 0.262；与光伏的线性相关接近于零。这说明价格与系统状态存在有限关联，但不能据此作因果解释。完整相关系数见表10，联合分布见图10。

表 10 动态电价与系统状态的相关性

系统状态	Pearson 系数	Spearman 系数
实际负荷	0.508	0.580
实际光伏	0.028	0.189
净负荷	0.260	0.262

题面要求依据附件 4 在波动电价下重新计算问题二、三，但没有说明 0:00 制定计划或日内更新时能否预先获知余下价格，因此价格信息可得性判为**不明确**。本文将两种口径分开：正式年度结果采用 GIVEN_PRICE，即把附件 4 当日 144 点价格作为题目给定输入；另保留严格因果价格预测接口，用于未来价格未知的解释。因果接口只用历史价格滞后和日历特征，经一月拟合、验证后冻结，样本外指标见图11；该预测路径没有被混入下述正式费用。

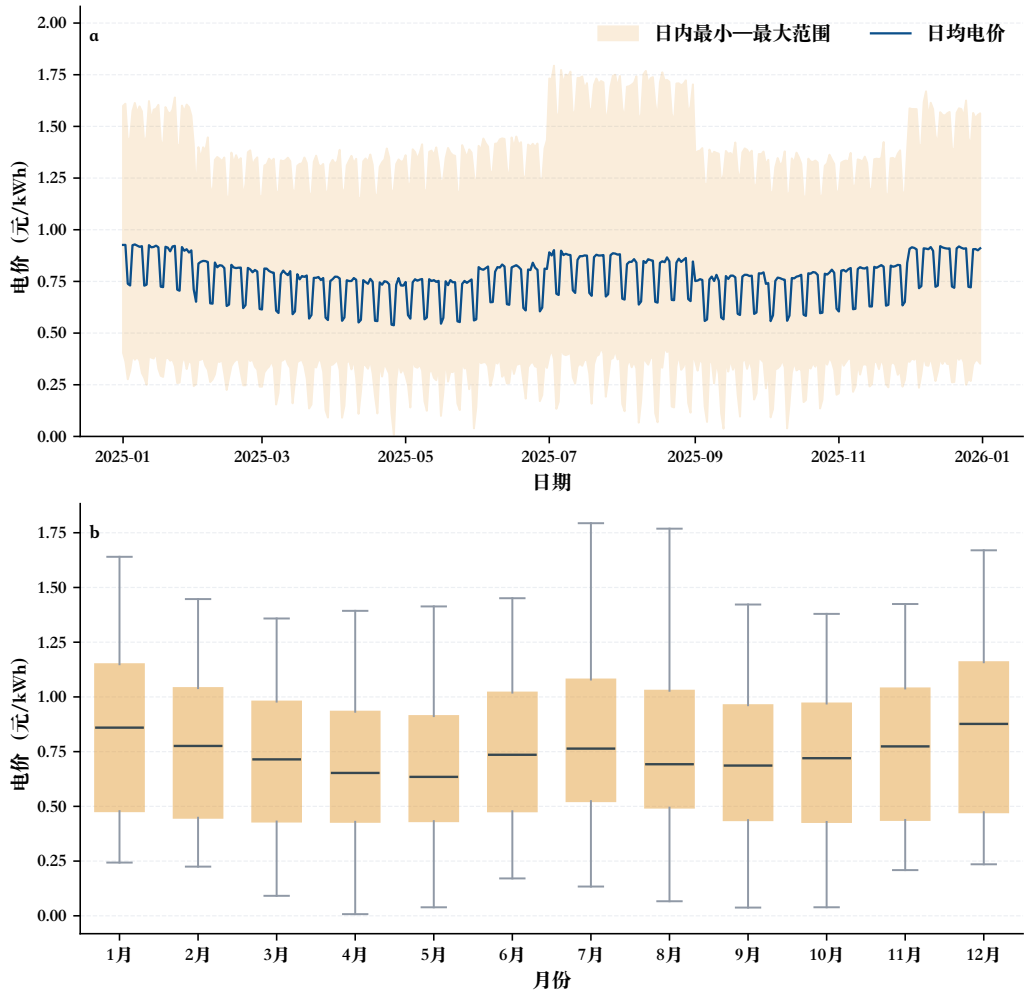


图 8 附件 4 动态电价的全年变化与月度分布。上图为 10 分钟价格序列，下图为各月全部观测的分布；极端值均保留。

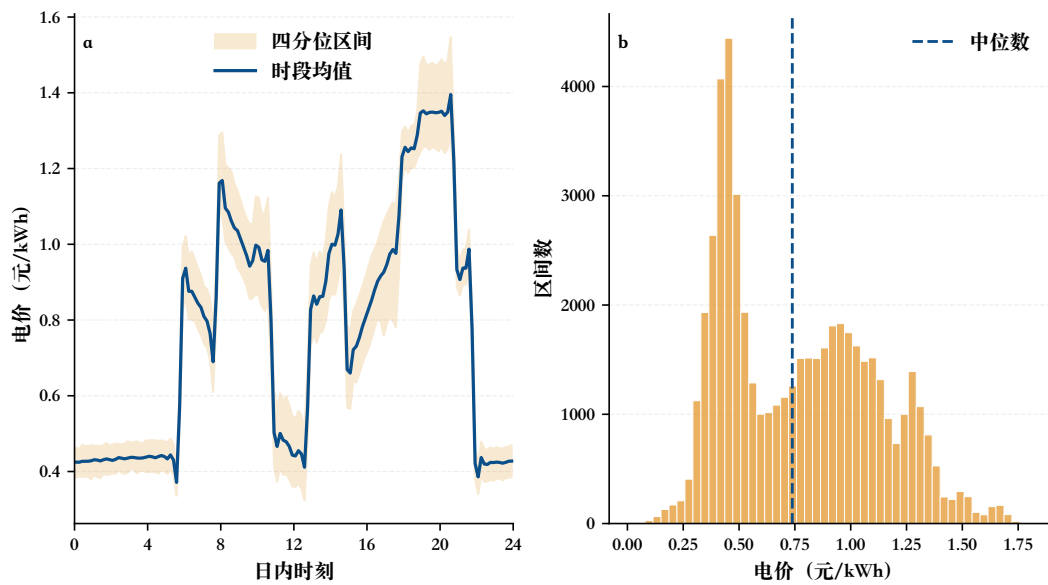


图 9 动态电价的日内轮廓与全年分布。日内主线及阴影分别表示同刻中位数和四分位区间，阴影不是置信区间。

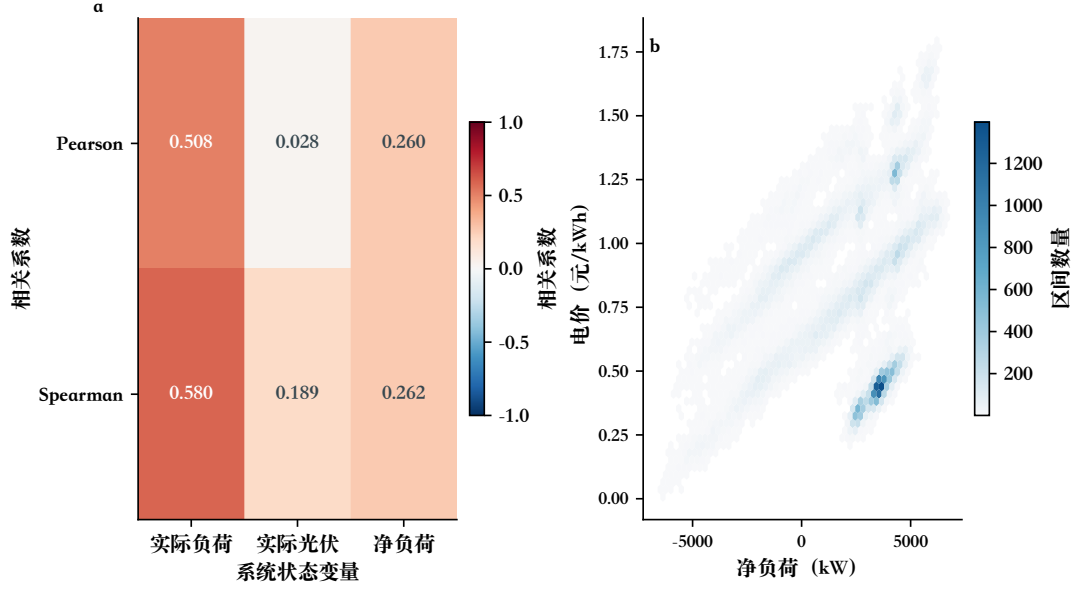


图 10 动态电价与实际负荷、光伏及净负荷的相关结构。相关系数与散点密度均使用全年 **52560** 个同步区间，只描述关联。

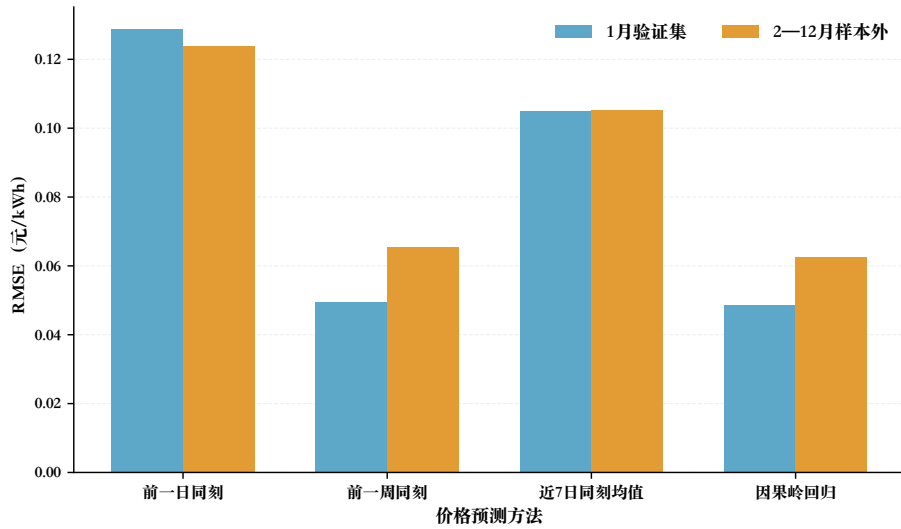


图 11 未来价格未知口径下的轻量因果预测比较。验证期用于选型，**2-12** 月仅作冻结样本外评价；该图不作为 **GIVEN_PRICE** 年度费用的数据来源。

8.2 Q4-2 与 Q4-3 模型

问题四不另建储能模型，而把附件 4 逐时价格 p_t 传入已封板的调度链。Q4-2 完整复用问题二的因果负荷/光伏预测、50 条成对整日残差场景、两阶段随机 MILP、延续价值、实际执行和跨日 SOC；只将式(8)中的固定日价格替换为当日动态价格。因此目标仍为

$$\min \sum_t p_t g_t \Delta t + \frac{1}{50} \sum_s \sum_t 5 p_t e_{t,s} \Delta t - V_d(E_{145}), \quad (10)$$

其余场景功率平衡、互斥、功率和电量边界均不变。实际费用按各区间真实价格核算为计划购电费与紧急购电费之和，紧急购电仍为 $5p_t$ 。

Q4-3 完整复用问题三的因果融合预测、已执行区间冻结、实际 SOC 交接和滚动结算。0:00 按式 (10) 形成初始承诺；在 $\tau \in \{6, 12, 18\}$ 获得新光伏预测后，只对 $t > \tau$ 的未来区间最小化

$$\sum_{t>\tau} p_t \Delta t (1.5 \Delta g_t^+ - 0.5 \Delta g_t^-) + \frac{1}{50} \sum_s \sum_{t>\tau} 5 p_t e_{t,s} \Delta t - V_d(E_{145}). \quad (11)$$

增购、减购和紧急购电均使用其所属 10 分钟区间的 p_t ，不以日均价或人工峰谷价替代。主结果沿用“相对上一承诺”的顺序结算，均相对 0:00 承诺的解释仅作敏感性核验。

8.3 年度调度结果

2025 年 2 月 1 日至 12 月 31 日的正式结果见表 11。Q4-2 共 334/334 日最优，计划购电费为 13926450.778073 元，模型内期望紧急购电费为 1090095.357606 元；实际回放的紧急购电量和费用分别为 396204.272796 kWh 和 1621505.578288 元，实际总费用为 15547956.356361 元。储能充、放电加权平均电价分别为 0.466423 和 1.121601 元/kWh，价差 0.655178 元/kWh，说明优化结果确实把电量由相对低价区间转移至高价区间，而非依赖人为峰谷划分。

表 11 动态电价下问题四的年度实际结算结果

方案	发布时间/h	计划费/万元	调整费/万元	紧急费/万元	总费用/万元	紧急电量/万 kWh
Q4-2	0	1392.645	—	162.151	1554.796	39.620
S0	0	1394.007	0.000	149.122	1543.129	36.530
S1	0,6	1394.939	2.482	135.876	1533.298	33.165
S2	0,6,12	1394.937	0.880	123.875	1519.692	30.215
S3	0,6,12,18	1395.066	1.065	123.885	1520.017	30.175

评价期为 2025 年 2 月 1 日至 12 月 31 日；各方案均为 334/334 日最优。Q4-2 无滚动调整；S0-S3 采用相对上一承诺的主结算口径。所有数值均为单次年度实际回放结果，不附加统计误差棒。

Q4-3 四种更新时间表的实际总费用依次为 15431293.173519、15332979.276497、15196922.472637 和 15200168.117470 元，仍以 $S_2 = \{0, 6, 12\}$ 最低。相对 S_0 ， S_2 节省 234370.700882 元（1.5188%），紧急购电量由 365303.875194 降至 302154.909779 kWh，下降 17.2867%。相对 Q4-2， S_2 少支出 351033.883724 元（2.2577%）。新增 6:00 和 12:00 发布的边际运营信息价值分别为 +98313.897022 和 +136056.803860 元，18:00 为 -3245.644832 元；这仍表示特定年度路径上的边际经济收益，而非“18:00 预测更差”。两种调整结算解释均保持 $S_2 < S_3 < S_1 < S_0$ ，同一方案年度费用最大差异仅 265.457 元。

所有方案均通过动态价格逐点对齐、功率平衡、SOC 递推及边界、跨日实际 SOC、充放电互斥、执行区间冻结和结算重构审计；未吸收放电为 0。按题意导出的两个结果

工作簿分别对应 Q4-2 和采用全部四个发布时间的 S_3 ； S_2 的最低费用作为更新频率比较结论报告。

8.4 价格与预测不确定性鲁棒性

为分离价格形状与平均价格的影响，令

$$p_t^{(\gamma)} = \bar{p} + \gamma(p_t - \bar{p}), \quad \gamma \in \{0, 0.25, 0.50, 0.75, 1.00\}.$$

各情形保持均价、真实负荷/光伏、点预测、残差抽样索引和物理参数不变。图 12 显示， S_2 在全部 γ 点均低于 Q4-2，节费率为 2.04%–2.26%；但费用在 $\gamma = 0$ 至 0.25 间略升，随后下降，故不能声称波动越大费用必然越低。波动增强总体扩大了可利用价差信号，同时改变计划和紧急购电的时点配置。

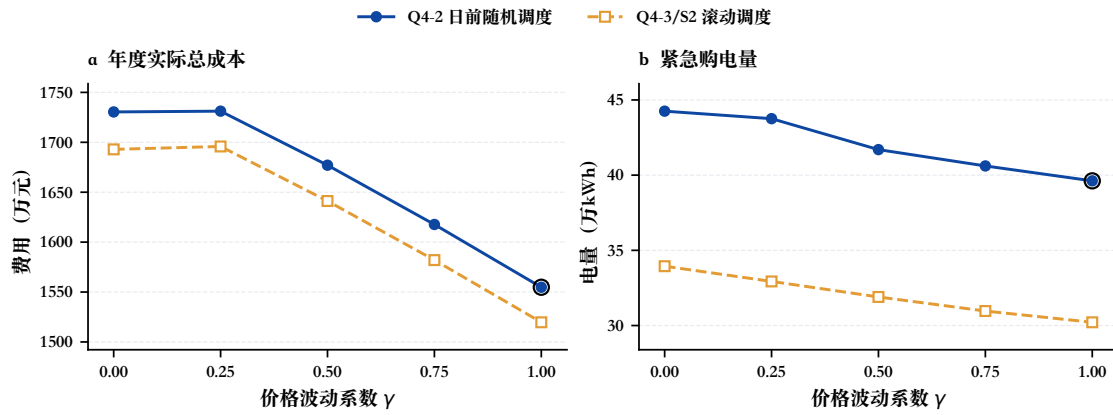


图 12 价格波动强度鲁棒性。各点为固定输入下的一次 334 日时序回放；左为实际总费用，右为紧急购电量。空心圈标记正式 $\gamma = 1$ ，不添加无重复样本依据的误差棒。

再保持点预测中心不变，将同一成对残差日的负荷和光伏误差同步缩放为 $\varepsilon^{(s,\kappa)} = \kappa \varepsilon^{(s)}$ ，其中 $\kappa \in \{0.75, 1.00, 1.25, 1.50\}$ 。图 13 显示，随着 κ 增大，更宽的规划场景促使计划提高预防性覆盖，两种方案的事后紧急购电量均下降；但过度放大残差会增加保守计划或调整代价，实际费用在正式 $\kappa = 1$ 附近较低而非严格单调。 S_2 在全部测试点仍低于 Q4-2，节费率为 1.62%–2.71%。该试验只比较 Q4-2 与既定 S_2 ，没有在每个扰动点重新选择 S_0 – S_3 ，因而不据此宣称 S_2 对任意参数都全局最优。

价格与残差扰动共 8 个表格点（含复用的正式基线）全部通过物理、结算、冻结及价格对齐审计，7 个新扰动点的两类模型均为 334/334 日最优。这说明滚动更新的费用优势在所测范围内较稳定；同时，非单调响应表明动态电价、风险覆盖与调整结算共同决定费用，不能用单一“波动越大套利越多”解释。

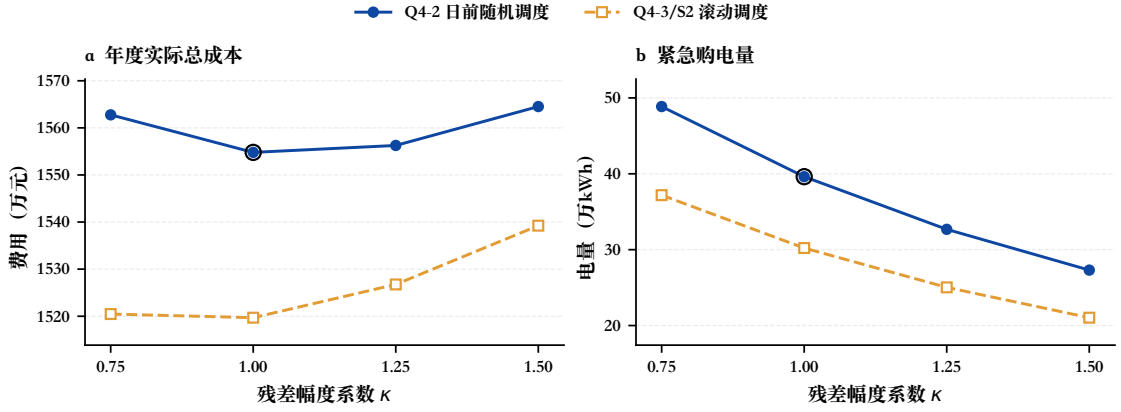


图 13 预测残差幅度鲁棒性。固定随机种子及成对场景索引，只缩放残差幅度；各点为单次年度轨迹，左为实际总费用，右为紧急购电量，不设置误差棒。

九、模型评价与适用范围

9.1 决策导向评价

本文把预测精度视为中间指标，把实际购电费用、紧急购电量、滚动调整费用和信息价值作为决策导向指标。这种评价与 Predict-then-Optimize 关于“预测损失不等于下游决策损失”的区分一致 [12]，但本文没有使用 SPO+ 训练。问题二中，近 7 日同刻均值的冻结样本外表现经日级配对检验优于五种可复现对照；问题三中，6:00 和 12:00 更新具有正增量 VOI，而 18:00 略为负，从经济层面再次说明更多信息或更低预测误差不自动等价于更低决策成本。

物理可靠性由三层证据支撑：全部正式日模型达到最优状态；功率平衡、储能递推、边界、互斥及跨日状态均通过残差审计；Q1 参数单因素分析和 Q3 结算口径敏感性均不改变对应主要结论。不过，Wilcoxon 配对单位虽取运营日以避免把 10 分钟点误当独立样本，连续日期仍可能存在序列相关， p 值应与效应量和日胜率共同解释。残差自助场景也受历史样本与 50 条场景数限制。

9.2 离线全信息下界与可观测成本差距

为量化因果决策与完全预知之间仍有多大差距，本文另建不可实施的离线全信息基准。该基准读取 2025 年 2 月 1 日至 12 月 31 日共 48096 个 10 分钟区间的真实负荷和光伏，沿用问题二、三固定电价、6000 kWh 初值、储能效率、功率和电量边界、禁止售电及弃光规则，并在 334 天上保持一条连续 SOC 轨迹。它仅放松充放电互斥二元结构，不设置每日重置或全年末端等式。

全年 LP 最优费用为 12228166.562875 元，求解中同时充放电区间数为 0，功率平衡与 SOC 递推最大残差分别为 1.82×10^{-12} kW 和 1.03×10^{-12} kWh。由于该 LP 解自身满足互斥及全部物理约束，结合松弛下界关系，它在本实例上同时也是物理完美信息问题

的最优解；这一结论不推广至其他模型。其末端电量降至 1200 kWh，而正式问题未规定全年末端回到初值，因此主下界不附加该条件；但这会带来有限视界末端乐观性，故该值只能作为信息放松下界，不能作为可实施策略。

表 12 因果策略与离线全信息基准的同口径费用比较

方案	费用/元	相对离线基准差距/元
离线全信息基准	12228166.562875	0
问题三 S_2	14505500.731613	2277334.168739
问题二因果日前策略	14839462.002357	2611295.439482

离线基准使用未来真实负荷和光伏，不能实施；其年末电量仅受物理边界约束，故具有有限视界乐观性。

以 C_{LB} 表示该基准，问题二实际费用、问题三 S_2 费用分别为 $C_{Q2} = 14839462.002357$ 元和 $C_{Q3} = 14505500.731613$ 元，满足 $C_{LB} \leq C_{Q3} \leq C_{Q2}$ 。于是问题二相对下界的成本差距为 2611295.439482 元，问题三剩余差距为 2277334.168739 元；问题三相对问题二降低 333961.270744 元，并缩小了

$$\rho = \frac{C_{Q2} - C_{Q3}}{C_{Q2} - C_{LB}} = 12.7891\%$$

的可观测成本差距。该比例同时包含预测更新、调整灵活性和滚动决策的共同作用，不能解释为恢复了同等比例的“纯预测信息价值”。图14采用零起点费用轴展示三者关系。

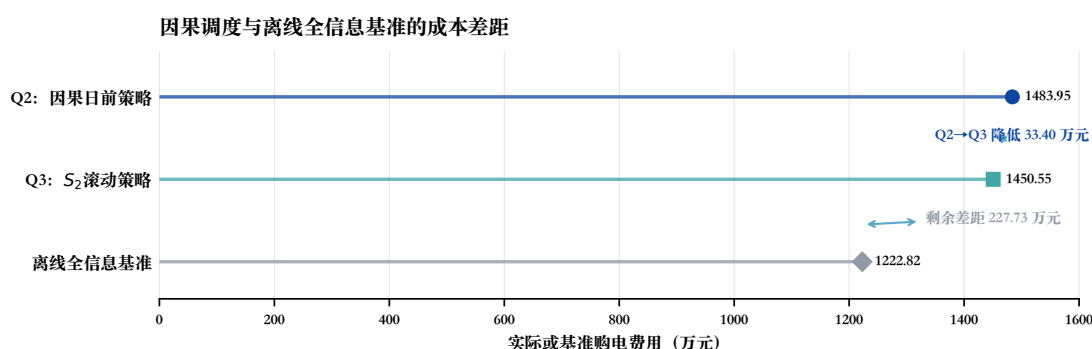


图 14 因果策略与离线全信息基准的费用差距。三点均为 2025 年 2 月 1 日至 12 月 31 日的同口径年度费用，横轴从零开始；离线基准使用未来真实负荷和光伏，且无年末电量等式，不能实施。年度总量为单次确定性核算，不添加统计误差棒。

9.3 因果信息链与物理执行可信度

问题二至四均按“已知信息 → 预测 → 优化 → 执行 → 观测真实值 → 结算与更新”的顺序运行。任一决策时刻只使用此前已经发布或实际发生的信息；未来真实负荷和光伏不进入当期决策，仅在对应时段结束后用于结算、实际 SOC 递推、残差池更新和后续评价。这一逐时评价方式使回测的信息边界与实际运行过程一致，并降低未来信息泄

漏风险。

计划层与物理执行层分别记录计划购电及计划充放电，以及真实负荷、真实光伏、实际充放电、实际 SOC 和紧急购电。已经支付但未被实际系统吸收的计划购电不被解释为物理注入；实际动作同时受计划上限、当期真实供需和实际 SOC 约束，下一日初始状态取上一日实际终端 SOC。正式审计据此检查功率平衡、SOC 递推、跨日衔接、未吸收放电和隐含能量汇，避免把计划轨迹直接作为实际运行轨迹。

9.4 模型的模块化扩展性

四问共享同一储能物理骨架：问题一为确定性 MILP，问题二扩展为基于场景的两阶段随机 MILP，问题三加入多时点滚动重优化与调整结算，问题四进一步引入动态电价。因而，预测、不确定性量化、优化决策和实际执行是可替换但相互校验的模块，而不是四套彼此独立的算法。

若进一步考虑风机、柴油机、多储能单元、线路容量、备用约束或分段线性电池退化成本，可在现有变量-约束结构中增加相应决策变量、物理约束和目标项，而无需改变“预测-场景-优化-滚动执行”的总体结构。精确动态规划在单电池、低维状态问题中能够给出清晰的状态价值函数；但当 SOC、SOH、多储能、机组状态和网络状态共同进入时，状态维数会迅速增加。本文选择 MILP 与滚动优化作为统一骨架，是因为其更适合本题后续逐级接入不确定性、结算规则和工程约束，而非意味着该方法在所有低维问题中均优于动态规划。

全年滚动计算中，相互独立的策略流可并行求解，并定期保存与输入签名绑定的状态和进度快照；异常中断后仅从最近的有效检查点继续。该机制用于减少重复计算并提高结果可复核性，不构成模型创新，也不代表系统已经完成实际电网部署验证。

9.5 适用范围与局限

模型适用于具有 10 分钟运行数据、储能物理边界明确且可顺序获得负荷/光伏信息的微网。若允许售电、计入电池退化成本，或价格、天气与网络约束发生变化，需要扩展目标和约束；进一步工作可以比较决策损失驱动的预测训练，但不能由本文结果声称已实现端到端决策学习。

十、 结论

本文把四问组织为同一调度框架随信息集逐步扩展的过程：问题一确定性优化，问题二引入预测不确定性，问题三随新信息滚动修正，问题四在保持物理和结算契约的基础上引入逐时动态电价。

问题一的 MILP 费用为 35126.948591 元，较无储能对照节省 12925.098 元 (26.898%)，弃光从 6247.963 kWh 降为 0。条件性交换论证解释了有损内部循环为何缺乏经济激励；

附件 1 单日 LP 松弛进一步给出零间隙且无同时充放电的数值证书，但该实例结论不推广到后续模型。问题二选择近 7 日同刻均值，并以 50 条因果成对整日残差场景建立两阶段随机 MILP。334/334 日最优，实际紧急购电量 398161.900954 kWh，总费用 14839462.002357 元；实际 SOC 跨日传递，功率平衡和储能递推通过审计。CVaR 扩展只衡量紧急购电尾部风险，主结果保持 $\lambda_r = 0$ 。

问题三把官方预测与季节预测按提前期分箱融合，并按新发布时间滚动重优化。 $S_2 = \{0, 6, 12\}$ 的年度实际费用最低，为 14505500.731613 元。相对 S_0 节省 222469.864818 元（1.5105%），紧急购电量下降 17.4946%。6:00 和 12:00 增量 VOI 为正，18:00 为 -2217.412836 元；负值不意味着该时刻预测更差，而说明额外信息的经济价值受结算、储能状态和剩余时域共同影响，并不随更新频率单调增加。两种调整结算解释保持 $S_2 < S_3 < S_1 < S_0$ ，主要结论稳定。

问题四按附件 4 价格路径给定口径重新计算前两类调度。Q4-2 实际总费用约为 1554.80 万元；Q4-3 仍以 $S_2 = \{0, 6, 12\}$ 最低，约为 1519.69 万元，较 Q4-2 减少 35.10 万元（2.2577%）。储能充、放电加权价格为 0.466423 和 1.121601 元/kWh，表明动态价差信号被用于跨时段转移电量。价格波动和残差幅度试验中， S_2 在全部测试点均保持费用优势，但总费用与扰动强度均不严格单调；因此，滚动调度的价值来自价格、预测误差、储能状态与调整结算的共同作用。未来电价可得性在题面中仍不明确，故上述年度费用只代表附件价格直接给定口径，因果价格预测接口另行保留而未与正式结果混用。

单时段分析进一步表明，在暂不考虑储能耦合时，同等幅度的净负荷低估所产生的局部费用遗憾为高估的 4 倍，解释了为何对称预测指标不足以替代经济评价。离线全信息基准费用为 12228166.562875 元；以此为参照，问题三 S_2 相对问题二缩小 12.7891% 的可观测成本差距。该下界使用未来真实信息且存在年末视界乐观性，不是可部署方案；实际 18:00 增量运营信息价值为负也不违反嵌套完美信息下理论 VOI 非负的性质。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于问题分析辅助、代码调试、程序结构检查、LaTeX 排版辅助、语言润色和图表排版建议等，详细使用情况见支撑材料。

本参赛队始终主导赛题理解、模型选择、数学推导、参数确定、程序运行、结果判断与论文结论。所有 AI 输出均经过人工阅读、修改、实际运行或与底层结果文件交叉核验，未经审查的输出未直接作为核心建模成果提交。具体工具、使用过程、人工修改和核验情况见支撑材料《AI 工具使用详情》。

参考文献

- [1] Hatziargyriou N, Asano H, Iravani R, Marnay C. Microgrids: An overview of ongoing research, development, and demonstration projects[J]. IEEE Power & Energy Magazine,

- 2007, 5(4): 78–94. DOI: 10.1109/MPAE.2007.376583.
- [2] Inman R H, Pedro H T C, Coimbra C F M. Solar forecasting methods for renewable energy integration[J]. *Progress in Energy and Combustion Science*, 2013, 39(6): 535–576. DOI: 10.1016/j.pecs.2013.06.002.
 - [3] Fritsch F N, Butland J. A method for constructing local monotone piecewise cubic interpolants[J]. *SIAM Journal on Scientific and Statistical Computing*, 1984, 5(2): 300–304. DOI: 10.1137/0905021.
 - [4] Vautard R, Ghil M. Singular spectrum analysis in nonlinear dynamics, with applications to paleoclimatic time series[J]. *Physica D*, 1989, 35(3): 395–424. DOI: 10.1016/0167-2789(89)90077-8.
 - [5] Zeng A, Chen M, Zhang L, Xu Q. Are Transformers Effective for Time Series Forecasting?[C]. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023, 37(9): 11121–11128. DOI: 10.1609/aaai.v37i9.26317.
 - [6] Dawid A P. Present position and potential developments: Some personal views: Statistical theory: The prequential approach[J]. *Journal of the Royal Statistical Society: Series A*, 1984, 147(2): 278–292. DOI: 10.2307/2981683.
 - [7] Hong T, Pinson P, Fan S, Zareipour H, Troccoli A, Hyndman R J. Probabilistic energy forecasting: Global Energy Forecasting Competition 2014 and beyond[J]. *International Journal of Forecasting*, 2016, 32(3): 896–913. DOI: 10.1016/j.ijforecast.2016.02.001.
 - [8] Lujano-Rojas J M, Dufo-López R, Bernal-Agustín J L, Catalão J P S. Optimizing daily operation of battery energy storage systems under real-time pricing schemes[J]. *IEEE Transactions on Smart Grid*, 2017, 8(1): 316–330. DOI: 10.1109/TSG.2016.2602268.
 - [9] Wilcoxon F. Individual comparisons by ranking methods[J]. *Biometrics Bulletin*, 1945, 1(6): 80–83. DOI: 10.2307/3001968.
 - [10] Benjamini Y, Hochberg Y. Controlling the false discovery rate: A practical and powerful approach to multiple testing[J]. *Journal of the Royal Statistical Society: Series B*, 1995, 57(1): 289–300. DOI: 10.1111/j.2517-6161.1995.tb02031.x.
 - [11] Rockafellar R T, Uryasev S. Optimization of conditional value-at-risk[J]. *Journal of Risk*, 2000, 2(3): 21–41. DOI: 10.21314/JOR.2000.038.
 - [12] Elmachtoub A N, Grigas P. Smart “Predict, then Optimize”[J]. *Management Science*, 2022, 68(1): 9–26. DOI: 10.1287/mnsc.2020.3922.

A 补充证明与代码说明

1.1 问题一条件性互补化的交换证明

设 $(g_t, c_t, d_t, w_t, E_t)$ 是命题 1 所述 LP 的一个最优解, 且某时段 t 满足 $c_t > 0, d_t > 0$ 。记 $\alpha = \eta_c \eta_d$, 并定义

$$\delta_t = \min \left\{ c_t, \frac{d_t}{\alpha} \right\}, \quad r_t = (1 - \alpha)\delta_t.$$

由于 $\delta_t > 0$, 令

$$c'_t = c_t - \delta_t, \quad d'_t = d_t - \alpha\delta_t$$

后, $c'_t, d'_t \geq 0$ 且至少一个等于 0; 两个变量只减不增, 故原充、放电功率上限仍成立。该变换对储能增量的影响为

$$\eta_c(c'_t - c_t)\Delta t - \frac{(d'_t - d_t)\Delta t}{\eta_d} = -\eta_c\delta_t\Delta t + \frac{\alpha\delta_t\Delta t}{\eta_d} = 0.$$

因此 E_{t+1} 不变, 继而全部后续储能状态、SOC 上下界及终端条件均保持不变。

在母线功率平衡中, 上述动作产生 r_t 的供给余量。取

$$a_t = \min\{g_t, r_t\}, \quad b_t = r_t - a_t.$$

由条件(4)可得 $0 \leq b_t \leq P_t - w_t$ 。再令

$$g'_t = g_t - a_t, \quad w'_t = w_t + b_t,$$

则 $g'_t \geq 0$ 且 $0 \leq w'_t \leq P_t$ 。代入式(1), 有

$$(g'_t + P_t + d'_t) - (L_t + c'_t + w'_t) = r_t - a_t - b_t = 0,$$

故功率平衡仍成立。其他时段变量保持不变, 因而得到一个消除了时段 t 同时充放电的 LP 可行解。目标函数的变化量为

$$\Delta z = -p_t a_t \Delta t \leq 0.$$

对所有同时充放电时段重复该交换, 得到满足 $c_t d_t = 0$ 的 LP 可行解。由于原解已经最优, 新解的目标值不可能低于 LP 最优值, 故交换后的目标值与原值相同。为该解补充二元变量: 充电时取 $u_t = 1$, 放电时取 $u_t = 0$, 两者均为 0 时任取, 即得到同目标值的 MILP 可行解。因此 $z_{\text{MILP}} \leq z_{\text{LP}}$; 又因 LP 是 MILP 的松弛, $z_{\text{LP}} \leq z_{\text{MILP}}$, 从而 $z_{\text{LP}} = z_{\text{MILP}}$ 。

需要强调的是, 条件(4)保证消去有损循环后产生的供给余量能够通过减少购电或增加弃光吸收。若不能验证这一条件, 上述局部交换并不自动构成一般性的精确松弛证明; 附件 1 实例的零间隙结论仍以表 1 的独立数值交叉验证为准。

B 支撑材料与程序说明

2.1 支撑材料文件列表

编号	文件路径	类型	对应问题	功能	输入	输出
1	data/raw/C 题/附件/附件 1.xlsx	Excel	问题一、二、三	固定电价、典型负荷与光伏	题目附件	确定性与固定价调度输入
2	data/raw/C 题/附件/附件 2.xlsx	Excel	问题二、三、四	全年实际负荷与光伏	题目附件	实际回放输入
3	data/raw/C 题/附件/附件 3.xlsx	Excel	问题三、四	多发布时间光伏预报	题目附件	滚动预测输入
4	data/raw/C 题/附件/附件 4.xlsx	Excel	问题四	10 分钟动态电价	题目附件	动态价格向量
5	src/common/	Python	全部	数据校验、预处理、指标与绘图公共模块	附件 1-4	规范化数据与诊断

编号	文件路径	类型	对应问题	功能	输入	输出
6	src/forecasting/	Python	问题二	因果预测、SSA、场景生成与冻结评价	规范化源荷数据	预测与联合场景
7	src/problem1/	Python	问题一	确定性 MILP、LP 松弛与灵敏度	附件 1	result1 及问题一图表
8	src/problem2/	Python	问题二	两阶段随机 MILP、物理结算与 CVaR	附件 1、2 及预测场景	result2、审计与风险表
9	src/problem3/	Python	问题三	预测融合、滚动重优化、结算与 VOI	附件 1-3	result3、S0-S3 及 VOI
10	src/problem4/	Python	问题四	动态价格接口、年度调度与鲁棒性	附件 2-4	result4-2、result4-3 及鲁棒性结果
11	src/analysis/information_bound.py	Python	综合分析	离线全信息下界与信息差距	全年实际源荷	下界费用与审计
12	src/paper_figures/	Python	全部	论文正式图生成	各问题结果表	paper/figures 中的正式图
13	results/problem1/result1.xlsx	Excel	问题一	题目格式的确定性调度结果	问题一模型	提交表 1、表 2
14	results/problem2/result2.xlsx	Excel	问题二	334 日计划、储能与紧急购电结果	问题二模型	提交表 3
15	results/problem3/result3.xlsx	Excel	问题三	S3 正式滚动购电与调整结果	问题三模型	问题三提交结果
16	results/problem4/result4-2.xlsx	Excel	问题四	动态价格下问题二重算结果	Q4-2 模型	问题 4-2 提交结果
17	results/problem4/result4-3.xlsx	Excel	问题四	动态价格下问题三重算结果	Q4-3 模型	问题 4-3 提交结果
18	results/problem2/Q2_PHYSICAL_SETTLEMENT_AUDIT.md	Markdown	问题二	实际执行与成本口径审计	Q2 逐时结果	物理与费用核验
19	results/problem3/Q3_ROLLING_AUDIT.md	Markdown	问题三	因果性、冻结、结算及 VOI 审计	Q3 年度结果	问题三核验
20	results/problem4/Q4_GIVEN_PRICE_INDEPENDENT_AUDIT.json	JSON	问题四	动态价格年度独立审计	Q4 年度结果	价格、物理与结算核验
21	requirements-lock.txt	文本	全部	Python 依赖精确版本	环境冻结	可复现安装清单
22	paper/AI 工具使用详情.tex	LaTeX	支撑材料	AI 工具使用详情源文件	真实使用记录	AI 工具使用详情.pdf

C 核心源程序

3.1 数据预处理与场景构造

Listing 1 src/common/preprocessing.py

```

1 """Canonical, leakage-safe preprocessing for CUMCM 2026 Problem C."""
2
3 from __future__ import annotations
4
5 from dataclasses import dataclass
6 from pathlib import Path
7 from typing import Any
8
9 import numpy as np
10 import pandas as pd
11 from scipy.interpolate import PchipInterpolator
12
13 from src.common.data_validation import file_sha256
14 from src.common.time_utils import (
15     SLOTS_PER_DAY,
16     TEN_MINUTES,
17     interval_metadata,
18     parse_interval_end_offset,
19     validate_slot_labels,
20     validate_timestamps,
21 )
22
23
24 DT_HOURS = 1.0 / 6.0
25 ONE_DAY_SLOTS = 144
26 SEVEN_DAY_SLOTS = 1008
27
28
29 @dataclass(frozen=True)
30 class CanonicalData:
31     """Frames produced by the Stage 1A data contract."""
32
33     problem1_day: pd.DataFrame

```

```

34 actual_10min: pd.DataFrame
35 pv_forecast_hourly: pd.DataFrame
36 forecast_actual_alignment: pd.DataFrame
37 pv_forecast_10min: pd.DataFrame
38 alignment_report: dict[str, Any]
39 interpolation_report: dict[str, Any]
40
41
42 def _duration_label(offset: pd.Timedelta) -> str:
43     minutes = int(offset / pd.Timedelta(minutes=1))
44     if minutes == 24 * 60:
45         return "24:00"
46     hour, minute = divmod(minutes, 60)
47     return f"{hour:02d}:{minute:02d}"
48
49
50 def load_problem1_day(path: str | Path) -> pd.DataFrame:
51     """Normalize Attachment 1 without statistical scaling."""
52     source = Path(path)
53     raw = pd.read_excel(source, sheet_name="Sheet1")
54     expected = ["时间", "电价", "小区负载", "光伏发电预测功率"]
55     if list(raw.columns) != expected:
56         raise ValueError(f"Unexpected Attachment 1 columns: {list(raw.columns)}")
57     offsets = validate_slot_labels(raw["时间"].tolist())
58     numeric = raw[expected[1:]].apply(pd.to_numeric, errors="coerce")
59     if numeric.isna().any().any():
60         raise ValueError("Attachment 1 contains missing or non-numeric measurements")
61
62     output = pd.DataFrame(
63         {
64             "slot": np.arange(1, SLOTS_PER_DAY + 1, dtype=int),
65             "interval_start": [_duration_label(value - TEN_MINUTES) for value in offsets],
66             "interval_end": [_duration_label(value) for value in offsets],
67             "price_yuan_per_kwh": numeric["电价"].to_numpy(dtype=float),
68             "load_kw": numeric["小区负载"].to_numpy(dtype=float),
69             "pv_forecast_kw": numeric["光伏发电预测功率"].to_numpy(dtype=float),
70         }
71     )
72     output["load_kwh"] = output["load_kw"] * DT_HOURS
73     output["pv_forecast_kwh"] = output["pv_forecast_kw"] * DT_HOURS
74     output["net_load_kw"] = output["load_kw"] - output["pv_forecast_kw"]
75     output["net_load_kwh"] = output["net_load_kw"] * DT_HOURS
76     return output
77
78
79 def wide_sheet_to_long(
80     path: str | Path,
81     *,
82     sheet_name: str,
83     value_name: str,
84 ) -> pd.DataFrame:
85     """Convert one date-by-slot sheet to a timestamped long table."""
86     raw = pd.read_excel(path, sheet_name=sheet_name)
87     if raw.shape[1] != SLOTS_PER_DAY + 1:
88         raise ValueError(f"{Path(path).name}/{sheet_name} must have 144 slot columns")
89     date_column = raw.columns[0]
90     value_columns = list(raw.columns[1:])
91     offsets = validate_slot_labels(value_columns)
92     dates = pd.to_datetime(raw[date_column], errors="coerce").dt.normalize()
93     if dates.isna().any() or dates.duplicated().any():
94         raise ValueError(f"{Path(path).name}/{sheet_name} has invalid or duplicate dates")
95     values = raw[value_columns].apply(pd.to_numeric, errors="coerce").to_numpy(dtype=float)
96
97     operating_date = pd.DatetimeIndex(np.repeat(dates.to_numpy(), SLOTS_PER_DAY))
98     slot = np.tile(np.arange(1, SLOTS_PER_DAY + 1, dtype=int), len(dates))
99     interval_end = pd.DatetimeIndex(

```

```

100     operating_date.to_numpy(dtype="datetime64[ns]")
101     + np.tile(offsets.to_numpy(dtype="timedelta64[ns]"), len(dates))
102 )
103 output = pd.DataFrame(
104     {
105         "operating_date": operating_date,
106         "slot": slot,
107         "interval_start": interval_end - TEN_MINUTES,
108         "interval_end": interval_end,
109         value_name: values.reshape(-1),
110     }
111 )
112 if output[["operating_date", "slot", "interval_end"]].duplicated().any():
113     raise ValueError(f"Duplicate long-table keys in {Path(path).name}/{sheet_name}")
114 return output
115
116
117 def _merge_one_to_one(left: pd.DataFrame, right: pd.DataFrame, keys: list[str]) -> pd.DataFrame:
118     """Outer merge that retains missing values and rejects duplicate keys."""
119     if left.duplicated(keys).any() or right.duplicated(keys).any():
120         raise ValueError(f"Duplicate merge keys: {keys}")
121     return left.merge(right, on=keys, how="outer", validate="one_to_one", sort=True)
122
123
124 def add_causal_history_features(frame: pd.DataFrame) -> pd.DataFrame:
125     """Add fixed historical lags using only rows strictly earlier than each row."""
126     output = frame.sort_values("interval_end", kind="stable").reset_index(drop=True).copy()
127     output["pv_lag_1d"] = output["pv_actual_kw"].shift(ONE_DAY_SLOTS)
128     output["pv_lag_7d"] = output["pv_actual_kw"].shift(SEVEN_DAY_SLOTS)
129     output["load_lag_1d"] = output["load_kw"].shift(ONE_DAY_SLOTS)
130     output["load_lag_7d"] = output["load_kw"].shift(SEVEN_DAY_SLOTS)
131     output["pv_same_slot_7d_mean"] = output.groupby("slot", sort=False)[
132         "pv_actual_kw"
133     ].transform(lambda values: values.shift(1).rolling(7, min_periods=7).mean())
134     return output
135
136
137 def load_actual_10min(attachment2: str | Path, attachment4: str | Path) -> pd.DataFrame:
138     """Build the complete 10-minute actual load, PV and price table."""
139     load = wide_sheet_to_long(attachment2, sheet_name="小区负载", value_name="load_kw")
140     pv = wide_sheet_to_long(
141         attachment2,
142         sheet_name="光伏发电实际功率",
143         value_name="pv_actual_kw",
144     )
145     price = wide_sheet_to_long(
146         attachment4,
147         sheet_name="Sheet1",
148         value_name="price_yuan_per_kwh",
149     )
150     keys = ["operating_date", "slot", "interval_start", "interval_end"]
151     merged = _merge_one_to_one(load, pv, keys)
152     merged = _merge_one_to_one(merged, price, keys)
153     merged = merged.sort_values("interval_end", kind="stable").reset_index(drop=True)
154     initial_check = validate_timestamps(merged["interval_end"], frequency="10min")
155     if initial_check["invalid_count"] or initial_check["duplicate_count"]:
156         raise ValueError("Invalid or duplicate interval_end before frequency completion")
157
158     complete_index = pd.date_range(
159         merged["interval_end"].min(), merged["interval_end"].max(), freq="10min"
160     )
161     values = (
162         merged.set_index("interval_end")[["load_kw", "pv_actual_kw", "price_yuan_per_kwh"]]
163         .reindex(complete_index)
164         .rename_axis("interval_end")
165         .reset_index()

```

```

166 )
167 metadata = interval_metadata(values["interval_end"])
168 output = metadata.merge(values, on="interval_end", validate="one_to_one")
169 output["load_kwh"] = output["load_kw"] * DT_HOURS
170 output["pv_actual_kwh"] = output["pv_actual_kw"] * DT_HOURS
171 output["net_load_kw"] = output["load_kw"] - output["pv_actual_kw"]
172 output["net_load_kwh"] = output["net_load_kw"] * DT_HOURS
173 return add_causal_history_features(output)
174
175
176 def load_pv_forecast_hourly(path: str | Path) -> pd.DataFrame:
177     """Convert Attachment 3 to one row per release and forecast horizon."""
178     raw = pd.read_excel(path, sheet_name="Sheet1")
179     horizon_columns = [f"预报{hour}小时" for hour in range(1, 25)]
180     expected = ["日期", "预报时刻", *horizon_columns]
181     if list(raw.columns) != expected:
182         raise ValueError("Unexpected Attachment 3 columns")
183     release_dates = pd.to_datetime(raw["日期"].ffill(), errors="coerce").dt.normalize()
184     if release_dates.isna().any():
185         raise ValueError("Attachment 3 contains an invalid release date")
186     release_offset = raw["预报时刻"].map(parse_interval_end_offset)
187     release_time = release_dates + pd.to_timedelta(release_offset)
188     releases = raw.assign(release_time=release_time)
189     long = releases.melt(
190         id_vars=["release_time"],
191         value_vars=horizon_columns,
192         var_name="horizon_label",
193         value_name="pv_forecast_kw",
194     )
195     long["horizon_hour"] = long["horizon_label"].str.extract(r"(\d+)").astype(int)
196     long["pv_forecast_kw"] = pd.to_numeric(long["pv_forecast_kw"], errors="coerce")
197     long["target_time"] = long["release_time"] + pd.to_timedelta(
198         long["horizon_hour"], unit="h"
199     )
200     long = long.sort_values(["release_time", "horizon_hour"], kind="stable").reset_index(drop=True)
201     if long.duplicated(["release_time", "horizon_hour"]).any():
202         raise ValueError("Attachment 3 has duplicate release/horizon keys")
203     long.insert(0, "forecast_id", np.arange(1, len(long) + 1, dtype=int))
204     return long[
205         ["forecast_id", "release_time", "horizon_hour", "target_time", "pv_forecast_kw"]
206     ]
207
208
209 def _lookup_actual(
210     actual_lookup: pd.Series,
211     timestamps: pd.Series | pd.DatetimeIndex,
212 ) -> np.ndarray:
213     """Use a verified unique index instead of a many-to-one merge."""
214     return actual_lookup.reindex(pd.DatetimeIndex(timestamps)).to_numpy(dtype=float)
215
216
217 def align_forecast_to_actual(
218     forecast: pd.DataFrame,
219     actual: pd.DataFrame,
220 ) -> tuple[pd.DataFrame, dict[str, Any]]:
221     """Align every forecast record to actual PV at its exact target time."""
222     if actual["interval_end"].duplicated().any():
223         raise ValueError("Actual PV lookup timestamps must be unique")
224     actual_lookup = actual.set_index("interval_end")["pv_actual_kw"].sort_index()
225     aligned = forecast.copy()
226     aligned["pv_actual_kw"] = _lookup_actual(actual_lookup, aligned["target_time"])
227     below = aligned["target_time"] < actual_lookup.index.min()
228     above = aligned["target_time"] > actual_lookup.index.max()
229     missing = aligned["pv_actual_kw"].isna()
230     aligned["alignment_status"] = np.where(missing, "unmatched", "matched")
231     aligned["unmatched_reason"] = ""

```

```

232 aligned.loc[missing & (below | above), "unmatched_reason"] = "out_of_actual_range"
233 aligned.loc[missing & ~(below | above), "unmatched_reason"] = "missing_actual_value"
234 aligned["absolute_error_kw"] = (
235     aligned["pv_forecast_kw"] - aligned["pv_actual_kw"]
236 ).abs()
237
238 candidate_actual: dict[str, np.ndarray] = {}
239 for label, offset in {
240     "target_minus_1h": pd.Timedelta(hours=-1),
241     "target_time": pd.Timedelta(0),
242     "target_plus_1h": pd.Timedelta(hours=1),
243 }.items():
244     candidate_actual[label] = _lookup_actual(
245         actual_lookup, aligned["target_time"] + offset
246     )
247 common = np.isfinite(aligned["pv_forecast_kw"].to_numpy(dtype=float))
248 for values in candidate_actual.values():
249     common &= np.isfinite(values)
250 forecast_values = aligned["pv_forecast_kw"].to_numpy(dtype=float)
251 mae = {
252     label: float(np.mean(np.abs(forecast_values[common] - values[common])))
253     for label, values in candidate_actual.items()
254 }
255 minimum_mae_alignment = min(mae, key=mae.get)
256 report = {
257     "raw_forecast_count": int(len(aligned)),
258     "aligned_count": int((aligned["alignment_status"] == "matched").sum()),
259     "unmatched_count": int((aligned["alignment_status"] == "unmatched").sum()),
260     "unmatched_reasons": {
261         str(key): int(value)
262         for key, value in aligned.loc[
263             aligned["alignment_status"] == "unmatched", "unmatched_reason"
264         ]
265         .value_counts()
266         .items()
267     },
268     "sanity_common_count": int(common.sum()),
269     "mae_kw": mae,
270     "minimum_mae_alignment": minimum_mae_alignment,
271     "selected_alignment": "target_time = release_time + horizon_hour",
272 }
273 return aligned, report
274
275
276 def interpolate_forecast_batches(
277     forecast: pd.DataFrame,
278     actual: pd.DataFrame,
279 ) -> tuple[pd.DataFrame, dict[str, Any]]:
280     """Interpolate each release independently to 10 minutes with causal anchors."""
281     if actual["interval_end"].duplicated().any():
282         raise ValueError("Actual PV lookup timestamps must be unique")
283     actual_lookup = actual.set_index("interval_end")["pv_actual_kw"].sort_index()
284     target_minutes = np.arange(10, 24 * 60 + 1, 10, dtype=int)
285     target_hours = target_minutes / 60.0
286     batches: list[pd.DataFrame] = []
287     method_counts: dict[str, int] = {}
288     anchor_count = 0
289
290     for release_time, batch in forecast.groupby("release_time", sort=True):
291         ordered = batch.sort_values("horizon_hour", kind="stable")
292         hours = ordered["horizon_hour"].to_numpy(dtype=float)
293         hourly = ordered["pv_forecast_kw"].to_numpy(dtype=float)
294         anchor = actual_lookup.get(pd.Timestamp(release_time), np.nan)
295         anchor_used = bool(np.isfinite(anchor))
296         if anchor_used:
297             anchor_count += 1

```

```

298     x = np.concatenate([[0.0], hours])
299     y = np.concatenate([[float(anchor)], hourly])
300 else:
301     x, y = hours.copy(), hourly.copy()
302 valid = np.isfinite(y)
303 x_valid, y_valid = x[valid], y[valid]
304
305 method = "pchip"
306 values = np.full(len(target_hours), np.nan, dtype=float)
307 try:
308     if len(x_valid) < 2:
309         raise ValueError("fewer than two interpolation nodes")
310     values = PchipInterpolator(x_valid, y_valid, extrapolate=False)(target_hours)
311 except (ValueError, FloatingPointError):
312     method = "linear"
313     if len(x_valid) >= 2:
314         values = np.interp(
315             target_hours,
316             x_valid,
317             y_valid,
318             left=np.nan,
319             right=np.nan,
320         )
321 values = np.where(np.isfinite(values), np.maximum(values, 0.0), np.nan)
322
323 hourly_node = target_minutes % 60 == 0
324 hourly_index = target_minutes[hourly_node] // 60 - 1
325 raw_nodes = hourly[hourly_index]
326 values[hourly_node] = np.where(
327     np.isfinite(raw_nodes), np.maximum(raw_nodes, 0.0), np.nan
328 )
329 method_counts[method] = method_counts.get(method, 0) + 1
330 batches.append(
331     pd.DataFrame(
332         {
333             "release_time": pd.Timestamp(release_time),
334             "horizon_step_10min": np.arange(1, SLOTS_PER_DAY + 1, dtype=int),
335             "target_time": pd.Timestamp(release_time)
336               + pd.to_timedelta(target_minutes, unit="m"),
337             "pv_forecast_10min_kw": values,
338             "is_original_hourly_node": hourly_node,
339             "actual_anchor_used": anchor_used,
340             "interpolation_method": method,
341         }
342     )
343 )
344 output = pd.concat(batches, ignore_index=True)
345 node_rows = output[output["is_original_hourly_node"]]
346 hourly_lookup = forecast.set_index(["release_time", "target_time"])["pv_forecast_kw"]
347 original = hourly_lookup.reindex(
348     pd.MultiIndex.from_frame(node_rows[["release_time", "target_time"]])
349 ).to_numpy(dtype=float)
350 preserved = np.isclose(
351     node_rows["pv_forecast_10min_kw"].to_numpy(dtype=float),
352     np.maximum(original, 0.0),
353     equal_nan=True,
354 )
355 report = {
356     "rows": int(len(output)),
357     "release_batches": int(output["release_time"].nunique()),
358     "rows_per_batch": SLOTS_PER_DAY,
359     "missing_values": int(output["pv_forecast_10min_kw"].isna().sum()),
360     "anchor_batches": int(anchor_count),
361     "method_counts": method_counts,
362     "hourly_nodes_checked": int(len(node_rows)),
363     "hourly_nodes_preserved": bool(preserved.all()),

```

```

364     "future_actual_used": False,
365 }
366 return output, report
367
368
369 def build_canonical_data(raw_dir: str | Path) -> CanonicalData:
370     """Read only the four official workbooks and build all Stage 1A frames."""
371     raw = Path(raw_dir)
372     paths = {index: raw / f"附件{index}.xlsx" for index in range(1, 5)}
373     missing = [str(path) for path in paths.values() if not path.exists()]
374     if missing:
375         raise FileNotFoundError("Missing official attachments:\n" + "\n".join(missing))
376     before = {path.name: file_sha256(path) for path in paths.values()}
377     problem1 = load_problem1_day(paths[1])
378     actual = load_actual_10min(paths[2], paths[4])
379     forecast = load_pv_forecast_hourly(paths[3])
380     alignment, alignment_report = align_forecast_to_actual(forecast, actual)
381     interpolated, interpolation_report = interpolate_forecast_batches(forecast, actual)
382     after = {path.name: file_sha256(path) for path in paths.values()}
383     if before != after:
384         raise RuntimeError("An official raw workbook changed during preprocessing")
385     return CanonicalData(
386         problem1_day=problem1,
387         actual_10min=actual,
388         pv_forecast_hourly=forecast,
389         forecast_actual_alignment=alignment,
390         pv_forecast_10min=interpolated,
391         alignment_report=alignment_report,
392         interpolation_report=interpolation_report,
393     )

```

Listing 2 src/forecasting/final_models.py

```

1  """Neutral forecasting components used by the final Stage 2A checkpoint.
2
3  The implementations are deliberately small and independent. They contain no
4  online update, replay, gating, spectral neural layer, or manuscript-specific
5  architecture.
6  """
7
8  from __future__ import annotations
9
10 from dataclasses import dataclass
11 import copy
12
13 import numpy as np
14 from scipy.linalg import svd
15 from scipy.sparse.linalg import svds
16 import torch
17 from torch import nn
18
19 from .model import diagonal_average, polynomial_vandermonde
20
21
22 @dataclass(frozen=True)
23 class CausalScaler:
24     """Scalar z-score fitted only on values supplied by the caller."""
25
26     mean: float
27     std: float
28
29     @classmethod
30     def fit(cls, values: np.ndarray) -> "CausalScaler":
31         array = np.asarray(values, dtype=float)

```

```

32     if not np.isfinite(array).all():
33         raise ValueError("scaler input contains NaN or Inf")
34     scale = float(array.std())
35     return cls(float(array.mean()), scale if scale > 1e-12 else 1.0)
36
37     def transform(self, values: np.ndarray) -> np.ndarray:
38         return (np.asarray(values, dtype=float) - self.mean) / self.std
39
40     def inverse(self, values: np.ndarray) -> np.ndarray:
41         return np.asarray(values, dtype=float) * self.std + self.mean
42
43
44 class MovingAverage(nn.Module):
45     """Centered moving average with the edge padding used by LTSF-Linear."""
46
47     def __init__(self, kernel_size: int) -> None:
48         super().__init__()
49         if kernel_size <= 0 or kernel_size % 2 == 0:
50             raise ValueError("kernel_size must be a positive odd integer")
51         self.kernel_size = int(kernel_size)
52         self.pool = nn.AvgPool1d(kernel_size=self.kernel_size, stride=1)
53
54     def forward(self, x: torch.Tensor) -> torch.Tensor:
55         if x.ndim != 3:
56             raise ValueError("DLinear input must have shape [batch, time, channel]")
57         pad = (self.kernel_size - 1) // 2
58         front = x[:, :1, :].repeat(1, pad, 1)
59         end = x[:, -1:, :].repeat(1, pad, 1)
60         return self.pool(torch.cat([front, x, end], dim=1).permute(0, 2, 1)).permute(0, 2, 1)
61
62
63 class DLinear(nn.Module):
64     """Minimal univariate DLinear: decomposition, two projections, and sum."""
65
66     def __init__(self, lookback: int, horizon: int, kernel_size: int) -> None:
67         super().__init__()
68         self.lookback = int(lookback)
69         self.horizon = int(horizon)
70         self.moving_average = MovingAverage(kernel_size)
71         self.seasonal_projection = nn.Linear(self.lookback, self.horizon)
72         self.trend_projection = nn.Linear(self.lookback, self.horizon)
73
74     def decompose(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
75         trend = self.moving_average(x)
76         return x - trend, trend
77
78     def forward(self, x: torch.Tensor) -> torch.Tensor:
79         seasonal, trend = self.decompose(x)
80         seasonal_out = self.seasonal_projection(seasonal.permute(0, 2, 1))
81         trend_out = self.trend_projection(trend.permute(0, 2, 1))
82         return (seasonal_out + trend_out).permute(0, 2, 1)
83
84
85 class ResidualMLP(nn.Module):
86     """One generic fixed residual learner with no specialist mechanisms."""
87
88     def __init__(self, lookback: int, horizon: int, hidden: int = 16) -> None:
89         super().__init__()
90         self.network = nn.Sequential(
91             nn.Linear(int(lookback), int(hidden)),
92             nn.GELU(),
93             nn.Linear(int(hidden), int(horizon)),
94         )
95
96     def forward(self, x: torch.Tensor) -> torch.Tensor:
97         return self.network(x)

```

```

98
99
100 @dataclass(frozen=True)
101 class FittedTorchModel:
102     model: nn.Module
103     best_epoch: int
104     train_loss: float
105     validation_loss: float
106
107
108 def fit_torch_regressor(
109     model: nn.Module,
110     x: np.ndarray,
111     y: np.ndarray,
112     *,
113     seed: int = 2026,
114     epochs: int = 40,
115     patience: int = 6,
116     learning_rate: float = 1e-3,
117 ) -> FittedTorchModel:
118     """Deterministic chronological early stopping followed by a full-data refit."""
119     torch.manual_seed(seed)
120     torch.use_deterministic_algorithms(True)
121     features = torch.as_tensor(np.asarray(x), dtype=torch.float32)
122     targets = torch.as_tensor(np.asarray(y), dtype=torch.float32)
123     if len(features) < 3 or len(features) != len(targets):
124         raise ValueError("training requires at least three aligned samples")
125     split = max(2, int(np.floor(len(features) * 0.8)))
126     split = min(split, len(features) - 1)
127     initial = copy.deepcopy(model.state_dict())
128     optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)
129     loss_fn = nn.MSELoss()
130     best_state = copy.deepcopy(model.state_dict())
131     best_loss = float("inf")
132     best_epoch = 1
133     stale = 0
134     for epoch in range(1, epochs + 1):
135         model.train()
136         optimizer.zero_grad(set_to_none=True)
137         loss = loss_fn(model(features[:split]), targets[:split])
138         loss.backward()
139         optimizer.step()
140         model.eval()
141         with torch.no_grad():
142             val_loss = float(loss_fn(model(features[split:]), targets[split:]).item())
143         if val_loss < best_loss - 1e-10:
144             best_loss = val_loss
145             best_epoch = epoch
146             best_state = copy.deepcopy(model.state_dict())
147             stale = 0
148         else:
149             stale += 1
150             if stale >= patience:
151                 break
152     # Refit for the selected epoch on every causally legal training sample.
153     model.load_state_dict(initial)
154     optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)
155     for _ in range(best_epoch):
156         model.train()
157         optimizer.zero_grad(set_to_none=True)
158         loss = loss_fn(model(features), targets)
159         loss.backward()
160         optimizer.step()
161     model.eval()
162     with torch.no_grad():
163         train_loss = float(loss_fn(model(features), targets).item())

```

```

164 return FittedTorchModel(model, best_epoch, train_loss, best_loss)
165
166
167 @dataclass(frozen=True)
168 class HankelPolynomialResult:
169     forecast: np.ndarray
170     reconstructed_history: np.ndarray
171
172
173 @dataclass(frozen=True)
174 class HankelPolynomialExpert:
175     """Rank-one Hankel reconstruction plus quadratic time extrapolation."""
176
177     lookahead: int
178     horizon: int = 144
179     hankel_rows: int = 144
180     polynomial_degree: int = 2
181     polynomial_ridge: float = 5.0
182
183     def _projection(self) -> np.ndarray:
184         tin = np.linspace(-1.0, 1.0, self.lookback)
185         tout = np.linspace(
186             1.0 + 2.0 / self.lookback,
187             1.0 + 2.0 * self.horizon / self.lookback,
188             self.horizon,
189         )
190         vin = polynomial_vandermonde(tin, self.polynomial_degree)
191         vout = polynomial_vandermonde(tout, self.polynomial_degree)
192         gram = vin.T @ vin + self.polynomial_ridge * np.eye(self.polynomial_degree + 1)
193         return vout @ np.linalg.solve(gram, vin.T)
194
195     def forecast(self, history: np.ndarray) -> HankelPolynomialResult:
196         values = np.asarray(history, dtype=float).reshape(-1)
197         if values.size != self.lookback or not np.isfinite(values).all():
198             raise ValueError("history does not match the declared lookahead")
199         if not 2 <= self.hankel_rows < self.lookback:
200             raise ValueError("invalid Hankel row count")
201         center = float(values.mean())
202         centered = values - center
203         trajectory = np.lib.stride_tricks.sliding_window_view(centered, self.hankel_rows).T
204         if min(trajectory.shape) > 1:
205             left, _, _ = svds(
206                 trajectory,
207                 k=1,
208                 which="LM",
209                 solver="arpack",
210                 v0=np.ones(min(trajectory.shape), dtype=float),
211             )
212         else:
213             left, _, _ = svd(trajectory, full_matrices=False, check_finite=False)
214             left = left[:, :1]
215             reconstructed = diagonal_average(left @ (left.T @ trajectory))
216             forecast = np.maximum(self._projection() @ reconstructed + center, 0.0)
217             return HankelPolynomialResult(forecast, reconstructed + center)
218
219
220 @dataclass(frozen=True)
221 class StructuralResidualHybrid:
222     """Fixed additive combination of an analytical expert and residual MLP."""
223
224     expert: HankelPolynomialExpert
225     residual_model: ResidualMLP
226     residual_scaler: CausalScaler
227
228     def predict(self, history: np.ndarray) -> np.ndarray:
229         analytical = self.expert.forecast(history)

```

```

230     residual_history = history - analytical.reconstructed_history
231     x = self.residual_scaler.transform(residual_history)[None, :]
232     with torch.no_grad():
233         residual_z = self.residual_model(torch.as_tensor(x, dtype=torch.float32)).numpy()[0]
234         residual_kw = self.residual_scaler.inverse(residual_z)
235         return np.maximum(analytical.forecast + residual_kw, 0.0)
236
237
238 def parameter_count(model: nn.Module) -> int:
239     return int(sum(parameter.numel() for parameter in model.parameters()))

```

Listing 3 `src/forecasting/scenarios.py`

```

1  """Leakage-safe paired whole-day residual scenarios for stochastic dispatch."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6
7  import numpy as np
8  import pandas as pd
9
10
11 @dataclass(frozen=True)
12 class ScenarioBatch:
13     """Joint load/generation scenarios and their paired residual provenance."""
14
15     target_date: pd.Timestamp
16     load_kw: np.ndarray
17     generation_kw: np.ndarray
18     source_residual_dates: pd.DatetimeIndex
19
20
21 class PairedDailyResidualGenerator:
22     """Sample complete paired residual days without splitting time or variables."""
23
24     def __init__(self, *, day_steps: int = 144, seed: int = 2026) -> None:
25         if day_steps <= 0:
26             raise ValueError("day_steps must be positive")
27         self.day_steps = int(day_steps)
28         self.generator = np.random.default_rng(seed)
29         self._dates: list[pd.Timestamp] = []
30         self._load: list[np.ndarray] = []
31         self._generation: list[np.ndarray] = []
32
33     @property
34     def pool_size(self) -> int:
35         return len(self._dates)
36
37     def add_residual_day(
38         self,
39         residual_date: object,
40         load_residual: np.ndarray,
41         generation_residual: np.ndarray,
42     ) -> None:
43         """Add one completed residual pair after its full target day is observed."""
44         day = pd.Timestamp(residual_date).normalize()
45         load = np.asarray(load_residual, dtype=float).reshape(-1)
46         generation = np.asarray(generation_residual, dtype=float).reshape(-1)
47         if load.size != self.day_steps or generation.size != self.day_steps:
48             raise ValueError("each residual day must preserve the complete daily vector")
49         if not np.isfinite(load).all() or not np.isfinite(generation).all():
50             raise ValueError("residual days must be finite")
51         if self._dates and day <= self._dates[-1]:

```

```

52         raise ValueError("residual days must be added once in strictly chronological order")
53     self._dates.append(day)
54     self._load.append(load.copy())
55     self._generation.append(generation.copy())
56
57     def sample(
58         self,
59         target_date: object,
60         forecast_load: np.ndarray,
61         forecast_generation: np.ndarray,
62         *,
63         scenario_count: int,
64         generation_upper_bound: float | None = None,
65     ) -> ScenarioBatch:
66         """Sample prior complete days and apply each paired trajectory as a unit."""
67         target = pd.Timestamp(target_date).normalize()
68         if scenario_count <= 0:
69             raise ValueError("scenario_count must be positive")
70         if not self._dates:
71             raise ValueError("residual pool is empty")
72         eligible = np.array([day < target for day in self._dates], dtype=bool)
73         eligible_indices = np.flatnonzero(eligible)
74         if eligible_indices.size == 0:
75             raise ValueError("residual pool contains no date strictly before the target")
76         load_base = np.asarray(forecast_load, dtype=float).reshape(-1)
77         generation_base = np.asarray(forecast_generation, dtype=float).reshape(-1)
78         if load_base.size != self.day_steps or generation_base.size != self.day_steps:
79             raise ValueError("forecasts must contain one complete day")
80         selected = self._generator.choice(eligible_indices, size=scenario_count, replace=True)
81         load_residuals = np.stack([self._load[index] for index in selected])
82         generation_residuals = np.stack([self._generation[index] for index in selected])
83         load_scenarios = np.maximum(load_base[None, :] + load_residuals, 0.0)
84         generation_scenarios = np.maximum(
85             generation_base[None, :] + generation_residuals, 0.0
86         )
87         if generation_upper_bound is not None:
88             if generation_upper_bound <= 0:
89                 raise ValueError("generation_upper_bound must be positive")
90             generation_scenarios = np.minimum(generation_scenarios, generation_upper_bound)
91         return ScenarioBatch(
92             target_date=target,
93             load_kw=load_scenarios,
94             generation_kw=generation_scenarios,
95             source_residual_dates=pd.DatetimeIndex([self._dates[index] for index in selected]),
96         )
97
98
99     def build_walk_forward_scenarios(
100         *,
101         calibration_dates: pd.DatetimeIndex,
102         calibration_load_residuals: np.ndarray,
103         calibration_generation_residuals: np.ndarray,
104         evaluation_dates: pd.DatetimeIndex,
105         forecast_load: np.ndarray,
106         forecast_generation: np.ndarray,
107         actual_load: np.ndarray,
108         actual_generation: np.ndarray,
109         scenario_count: int,
110         seed: int,
111     ) -> tuple[dict[str, np.ndarray], pd.DataFrame]:
112         """Generate every day before adding that day's newly observed residual pair."""
113         day_steps = forecast_load.shape[1]
114         generator = PairedDailyResidualGenerator(day_steps=day_steps, seed=seed)
115         for position, day in enumerate(calibration_dates):
116             generator.add_residual_day(
117                 day,

```

```

118     calibration_load_residuals[position],
119     calibration_generation_residuals[position],
120 )
121
122 load_batches: list[np.ndarray] = []
123 generation_batches: list[np.ndarray] = []
124 source_dates: list[np.ndarray] = []
125 manifest_rows: list[dict[str, object]] = []
126 for position, target_date in enumerate(evaluation_dates):
127     pool_size_before = generator.pool_size
128     batch = generator.sample(
129         target_date,
130         forecast_load[position],
131         forecast_generation[position],
132         scenario_count=scenario_count,
133         generation_upper_bound=None,
134     )
135     load_batches.append(batch.load_kw.astype(np.float32))
136     generation_batches.append(batch.generation_kw.astype(np.float32))
137     source_dates.append(batch.source_residual_dates.to_numpy(dtype="datetime64[D]"))
138     manifest_rows.append(
139         {
140             "target_date": target_date.date().isoformat(),
141             "residual_pool_size": pool_size_before,
142             "scenario_count": scenario_count,
143             "method": "paired_whole_day_residual_resampling",
144             "day_steps": day_steps,
145             "generation_lower_bound_kw": 0.0,
146             "generation_upper_bound_kw": np.nan,
147             "load_lower_bound_kw": 0.0,
148             "all_sources_strictly_before_target": bool(
149                 (batch.source_residual_dates < target_date).all()
150             ),
151         }
152     )
153     generator.add_residual_day(
154         target_date,
155         actual_load[position] - forecast_load[position],
156         actual_generation[position] - forecast_generation[position],
157     )
158     archive = {
159         "target_dates": evaluation_dates.to_numpy(dtype="datetime64[D]"),
160         "source_residual_dates": np.stack(source_dates),
161         "load_kw": np.stack(load_batches),
162         "generation_kw": np.stack(generation_batches),
163     }
164     return archive, pd.DataFrame(manifest_rows)

```

3.2 问题一与问题二优化模型

Listing 4 src/problem1/model.py

```

1 """Deterministic microgrid dispatch model for Problem 1."""
2
3 from __future__ import annotations
4
5 from dataclasses import dataclass
6 from time import perf_counter
7
8 import pandas as pd
9 import pulp
10
11

```

```

12 DT_HOURS = 1.0 / 6.0
13
14
15 @dataclass(frozen=True)
16 class DispatchParameters:
17     """Physical parameters from Appendix 1 of the problem statement."""
18
19     capacity_kwh: float = 12_000.0
20     initial_energy_kwh: float = 6_000.0
21     minimum_energy_kwh: float = 1_200.0
22     maximum_energy_kwh: float = 10_800.0
23     maximum_charge_kw: float = 5_000.0
24     maximum_discharge_kw: float = 5_000.0
25     charge_efficiency: float = 0.9
26     discharge_efficiency: float = 0.9
27     terminal_energy_kwh: float = 6_000.0
28
29
30 @dataclass(frozen=True)
31 class DispatchResult:
32     """Optimal dispatch and solver metadata."""
33
34     dispatch: pd.DataFrame
35     status: str
36     objective_yuan: float
37     runtime_seconds: float
38     solver_name: str
39
40
41 def solve_deterministic_dispatch(
42     data: pd.DataFrame,
43     parameters: DispatchParameters | None = None,
44 ) -> DispatchResult:
45     """Solve the 144-period deterministic Q1 MILP with PuLP and CBC."""
46     parameters = parameters or DispatchParameters()
47     required = {
48         "slot",
49         "interval_start",
50         "interval_end",
51         "price_yuan_per_kwh",
52         "load_kw",
53         "pv_forecast_kw",
54     }
55     missing = required.difference(data.columns)
56     if missing:
57         raise ValueError(f"Missing required columns: {sorted(missing)}")
58     if len(data) != 144:
59         raise ValueError(f"Q1 requires 144 ten-minute periods, got {len(data)}")
60
61     frame = data.sort_values("slot").reset_index(drop=True).copy()
62     if frame["slot"].tolist() != list(range(1, 145)):
63         raise ValueError("slot must be the consecutive integers 1 through 144")
64     if frame[list(required - {"interval_start", "interval_end"}).isna().any().any():
65         raise ValueError("Q1 input contains missing numeric values")
66
67     periods = range(len(frame))
68     model = pulp.LpProblem("CUMCM2026_Q1_Deterministic_Dispatch", pulp.LpMinimize)
69     grid = pulp.LpVariable.dicts("grid_kw", periods, lowBound=0)
70     charge = pulp.LpVariable.dicts("charge_kw", periods, lowBound=0)
71     discharge = pulp.LpVariable.dicts("discharge_kw", periods, lowBound=0)
72     spill = pulp.LpVariable.dicts("spill_kw", periods, lowBound=0)
73     mode = pulp.LpVariable.dicts("charge_mode", periods, cat=pulp.LpBinary)
74     energy = pulp.LpVariable.dicts(
75         "energy_kwh",
76         range(len(frame) + 1),
77         lowBound=parameters.minimum_energy_kwh,

```

```

78     upBound=parameters.maximum_energy_kwh,
79 )
80
81 model += energy[0] == parameters.initial_energy_kwh, "initial_energy"
82 model += energy[len(frame)] == parameters.terminal_energy_kwh, "terminal_energy"
83
84 for t in periods:
85     load_kw = float(frame.at[t, "load_kw"])
86     pv_kw = float(frame.at[t, "pv_forecast_kw"])
87     model += (
88         grid[t] + pv_kw + discharge[t] == load_kw + charge[t] + spill[t]
89     ), f"power_balance_{t + 1:03d}"
90     model += (
91         energy[t + 1]
92         == energy[t]
93         + parameters.charge_efficiency * charge[t] * DT_HOURS
94         - discharge[t] * DT_HOURS / parameters.discharge_efficiency
95     ), f"energy_transition_{t + 1:03d}"
96     model += (
97         charge[t] <= parameters.maximum_charge_kw * mode[t]
98     ), f"charge_limit_{t + 1:03d}"
99     model += (
100         discharge[t] <= parameters.maximum_discharge_kw * (1 - mode[t])
101     ), f"discharge_limit_{t + 1:03d}"
102     model += spill[t] <= pv_kw, f"spill_limit_{t + 1:03d}"
103
104 model += pulp.lpSum(
105     float(frame.at[t, "price_yuan_per_kwh"]) * grid[t] * DT_HOURS
106     for t in periods
107 )
108
109 solver = pulp.COIN_CMD(
110     path=pulp.apis.PULP_CBC_CMD.pulp_cbc_path,
111     msg=False,
112     threads=1,
113 )
114 started = perf_counter()
115 model.solve(solver)
116 runtime_seconds = perf_counter() - started
117 status = pulp.LpStatus[model.status]
118 if status != "Optimal":
119     raise RuntimeError(f"CBC did not find an optimal Q1 solution: {status}")
120
121 result = frame[
122     [
123         "slot",
124         "interval_start",
125         "interval_end",
126         "price_yuan_per_kwh",
127         "load_kw",
128         "pv_forecast_kw",
129     ]
130 ].copy()
131 result["grid_purchase_kw"] = [pulp.value(grid[t]) for t in periods]
132 result["charge_kw"] = [pulp.value(charge[t]) for t in periods]
133 result["discharge_kw"] = [pulp.value(discharge[t]) for t in periods]
134 result["spill_kw"] = [pulp.value(spill[t]) for t in periods]
135 result["storage_start_kwh"] = [pulp.value(energy[t]) for t in periods]
136 result["storage_end_kwh"] = [pulp.value(energy[t + 1]) for t in periods]
137 result["grid_purchase_kwh"] = result["grid_purchase_kw"] * DT_HOURS
138 result["charge_kwh"] = result["charge_kw"] * DT_HOURS
139 result["discharge_kwh"] = result["discharge_kw"] * DT_HOURS
140 result["spill_kwh"] = result["spill_kw"] * DT_HOURS
141
142 return DispatchResult(
143     dispatch=result,

```

```

144     status=status,
145     objective_yuan=float(pulp.value(model.objective)),
146     runtime_seconds=runtime_seconds,
147     solver_name="PuLP CBC",
148 )

```

Listing 5 src/problem2/model.py

```

1  """Expected-cost stochastic MILP for Problem 2 day-ahead dispatch."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from time import perf_counter
7
8  import numpy as np
9  import pandas as pd
10 import pulp
11
12 from src.problem2.forecast_interface import Q2DayInputs
13
14
15 @dataclass(frozen=True)
16 class Q2DispatchParameters:
17     """Battery, emergency-pricing, and terminal-value parameters."""
18
19     capacity_kwh: float = 12_000.0
20     minimum_energy_kwh: float = 1_200.0
21     maximum_energy_kwh: float = 10_800.0
22     maximum_charge_kw: float = 5_000.0
23     maximum_discharge_kw: float = 5_000.0
24     charge_efficiency: float = 0.9
25     discharge_efficiency: float = 0.9
26     emergency_price_multiplier: float = 5.0
27     terminal_reserve_quantile: float = 0.80
28     terminal_value_price_quantiles: tuple[float, ...] = (0.90, 0.50, 0.10)
29     cvar_alpha: float = 0.90
30     risk_weight: float = 0.0
31
32
33 @dataclass(frozen=True)
34 class Q2DayResult:
35     """Optimal shared plan, scenario recourse, and cost accounting for one day."""
36
37     date: object
38     dispatch: pd.DataFrame
39     emergency_kw: np.ndarray
40     spill_kw: np.ndarray
41     planned_grid_used_kw: np.ndarray
42     unused_planned_grid_kw: np.ndarray
43     scenario_source_dates: np.ndarray
44     status: str
45     runtime_seconds: float
46     solver_name: str
47     planned_purchase_cost_yuan: float
48     expected_emergency_cost_yuan: float
49     operating_cost_yuan: float
50     terminal_value_credit_yuan: float
51     optimization_objective_yuan: float
52     scenario_emergency_cost_yuan: np.ndarray
53     cvar_alpha: float
54     risk_weight: float
55     var_cost_yuan: float
56     cvar_cost_yuan: float

```

```

57 cvar_excess_cost_yuan: np.ndarray
58 lambda_cvar_term_yuan: float
59 terminal_value_breakpoints_kwh: tuple[float, ...]
60 terminal_value_rates_yuan_per_kwh: tuple[float, ...]
61
62 @property
63 def final_energy_kwh(self) -> float:
64     return float(self.dispatch["storage_end_kwh"].iloc[-1])
65
66
67 def calculate_empirical_cvar(
68     scenario_costs_yuan: np.ndarray, alpha: float
69 ) -> tuple[float, float]:
70     """Return a discrete equal-weight VaR representative and exact CVaR value."""
71
72     costs = np.asarray(scenario_costs_yuan, dtype=float).reshape(-1)
73     if costs.size == 0 or not np.isfinite(costs).all():
74         raise ValueError("scenario costs must be a nonempty finite array")
75     if not 0.0 < alpha < 1.0:
76         raise ValueError("CVaR alpha must lie strictly between 0 and 1")
77     candidates = np.unique(costs)
78     objectives = np.array(
79         [zeta + np.maximum(costs - zeta, 0.0).mean() / (1.0 - alpha) for zeta in candidates]
80     )
81     index = int(np.argmin(objectives))
82     return float(candidates[index]), float(objectives[index])
83
84
85 def validate_inputs(inputs: Q2DayInputs, parameters: Q2DispatchParameters) -> None:
86     expected_scenario_shape = (inputs.scenario_count, inputs.horizon)
87     if inputs.horizon != 144:
88         raise ValueError(f"Q2 requires 144 ten-minute periods, got {inputs.horizon}")
89     if inputs.scenario_count != 50:
90         raise ValueError(f"Q2 requires 50 paired scenarios, got {inputs.scenario_count}")
91     if inputs.load_scenarios.shape != expected_scenario_shape:
92         raise ValueError("load scenario array does not match [scenario, time]")
93     if inputs.pv_scenarios.shape != expected_scenario_shape:
94         raise ValueError("PV scenario array does not match [scenario, time]")
95     numeric = np.concatenate(
96         [
97             inputs.load_forecast,
98             inputs.pv_forecast,
99             inputs.load_scenarios.reshape(-1),
100             inputs.pv_scenarios.reshape(-1),
101             inputs.price,
102             np.array([inputs.initial_energy, inputs.dt_hours]),
103         ]
104     )
105     if not np.isfinite(numeric).all():
106         raise ValueError("Q2 optimizer input contains NaN or infinite values")
107     if (inputs.pv_scenarios < 0).any():
108         raise ValueError("PV scenarios must be nonnegative")
109     if not (
110         parameters.minimum_energy_kwh
111         <= inputs.initial_energy
112         <= parameters.maximum_energy_kwh
113     ):
114         raise ValueError("initial energy is outside the permitted battery range")
115
116
117 def solve_expected_cost_dispatch(
118     inputs: Q2DayInputs,
119     parameters: Q2DispatchParameters | None = None,
120 ) -> Q2DayResult:
121     """Solve one Q2 day with shared decisions and scenario-specific recourse.
122

```

```

123 Paid day-ahead energy may be unused in each scenario. PV curtailment is kept
124 as a separate physical quantity. A concave piecewise-linear continuation
125 value rewards useful horizon-end energy with decreasing marginal value; it
126 neither targets the day's initial energy nor imposes a daily reset.
127 """
128
129 parameters = parameters or Q2DispatchParameters()
130 _validate_inputs(inputs, parameters)
131 periods = range(inputs.horizon)
132 scenarios = range(inputs.scenario_count)
133 quantiles = tuple(float(value) for value in parameters.terminal_value_price_quantiles)
134 if len(quantiles) != 3:
135     raise ValueError("terminal value requires three decreasing marginal-value segments")
136 if not 0.0 <= parameters.terminal_reserve_quantile <= 1.0:
137     raise ValueError("terminal reserve quantile must lie in [0, 1]")
138 if any(not 0.0 <= quantile <= 1.0 for quantile in quantiles):
139     raise ValueError("terminal value price quantiles must lie in [0, 1]")
140 if not 0.0 < parameters.cvar_alpha < 1.0:
141     raise ValueError("CVaR alpha must lie strictly between 0 and 1")
142 if parameters.risk_weight < 0.0:
143     raise ValueError("CVaR risk weight must be nonnegative")
144 scenario_net_load = inputs.load_scenarios - inputs.pv_scenarios
145 point_net_load = inputs.load_forecast - inputs.pv_forecast
146 positive_error_energy = np.maximum(
147     scenario_net_load - point_net_load[None, :], 0.0
148 ).sum(axis=1) * inputs.dt_hours
149 uncertainty_reserve = float(
150     np.clip(
151         parameters.minimum_energy_kwh
152         + np.quantile(positive_error_energy, parameters.terminal_reserve_quantile),
153         parameters.minimum_energy_kwh + 1.0,
154         parameters.maximum_energy_kwh - 2.0,
155     )
156 )
157 upper_midpoint = uncertainty_reserve + (
158     parameters.maximum_energy_kwh - uncertainty_reserve
159 ) / 2.0
160 breakpoints = (uncertainty_reserve, upper_midpoint, parameters.maximum_energy_kwh)
161 terminal_value_rates = tuple(
162     parameters.discharge_efficiency * float(np.quantile(inputs.price, quantile))
163     for quantile in quantiles
164 )
165 if any(left < right for left, right in zip(terminal_value_rates, terminal_value_rates[1:])):
166     raise ValueError("terminal marginal values must be nonincreasing")
167
168 model = pulp.LpProblem(
169     f"CUMCM2026_Q2_Expected_Cost_{inputs.date}", pulp.LpMinimize
170 )
171 grid = pulp.LpVariable.dicts("planned_grid_kw", periods, lowBound=0)
172 charge = pulp.LpVariable.dicts("planned_charge_limit_kw", periods, lowBound=0)
173 discharge = pulp.LpVariable.dicts("planned_discharge_limit_kw", periods, lowBound=0)
174 mode = pulp.LpVariable.dicts("charge_mode", periods, cat=pulp.LpBinary)
175 energy = pulp.LpVariable.dicts(
176     "energy_kwh",
177     range(inputs.horizon + 1),
178     lowBound=parameters.minimum_energy_kwh,
179     upBound=parameters.maximum_energy_kwh,
180 )
181 emergency = pulp.LpVariable.dicts(
182     "emergency_kw", (scenarios, periods), lowBound=0
183 )
184 spill = pulp.LpVariable.dicts("spill_kw", (scenarios, periods), lowBound=0)
185 planned_grid_used = pulp.LpVariable.dicts(
186     "planned_grid_used_kw", (scenarios, periods), lowBound=0
187 )
188 terminal_segments = [

```

```

189     pulp.LpVariable(
190         f"terminal_value_segment_{index + 1}_kwh",
191         lowBound=0,
192         upBound=upper - lower,
193     )
194     for index, (lower, upper) in enumerate(
195         zip((parameters.minimum_energy_kwh,) + breakpoints[:-1], breakpoints)
196     )
197 ]
198
199 model += energy[0] == inputs.initial_energy, "carried_initial_energy"
200 model += energy[inputs.horizon] == parameters.minimum_energy_kwh + pulp.lpSum(
201     terminal_segments
202 ), "terminal_value_decomposition"
203 for t in periods:
204     model += (
205         energy[t + 1]
206         == energy[t]
207         + parameters.charge_efficiency * charge[t] * inputs.dt_hours
208         - discharge[t] * inputs.dt_hours / parameters.discharge_efficiency
209     ), f"energy_transition_{t + 1:03d}"
210     model += (
211         charge[t] <= parameters.maximum_charge_kw * mode[t]
212     ), f"charge_limit_{t + 1:03d}"
213     model += (
214         discharge[t] <= parameters.maximum_discharge_kw * (1 - mode[t])
215     ), f"discharge_limit_{t + 1:03d}"
216     for s in scenarios:
217         pv_kw = float(inputs.pv_scenarios[s, t])
218         load_kw = float(inputs.load_scenarios[s, t])
219         model += (
220             planned_grid_used[s][t]
221             + emergency[s][t]
222             + pv_kw
223             + discharge[t]
224             == load_kw + charge[t] + spill[s][t]
225         ), f"power_balance_{s + 1:02d}_{t + 1:03d}"
226         model += (
227             planned_grid_used[s][t] <= grid[t]
228         ), f"planned_grid_use_limit_{s + 1:02d}_{t + 1:03d}"
229         model += spill[s][t] <= pv_kw, f"spill_limit_{s + 1:02d}_{t + 1:03d}"
230
231 planned_cost = pulp.lpSum(
232     float(inputs.price[t]) * grid[t] * inputs.dt_hours for t in periods
233 )
234 scenario_emergency_costs = [
235     pulp.lpSum(
236         parameters.emergency_price_multiplier
237         * float(inputs.price[t])
238         * emergency[s][t]
239         * inputs.dt_hours
240         for t in periods
241     )
242     for s in scenarios
243 ]
244 expected_emergency_cost = pulp.lpSum(scenario_emergency_costs) / inputs.scenario_count
245 terminal_value_credit = pulp.lpSum(
246     rate * segment for rate, segment in zip(terminal_value_rates, terminal_segments)
247 )
248 cvar_zeta = None
249 cvar_excess = None
250 cvar_expression = None
251 if parameters.risk_weight > 0.0:
252     cvar_zeta = pulp.LpVariable("emergency_cost_var_yuan", lowBound=0)
253     cvar_excess = pulp.LpVariable.dicts(
254         "emergency_cost_excess_yuan", scenarios, lowBound=0

```

```

255 )
256 for s in scenarios:
257     model += (
258         cvar_excess[s] >= scenario_emergency_costs[s] - cvar_zeta
259     ), f"cvar_excess_s{s + 1:02d}"
260 cvar_expression = cvar_zeta + pulp.lpSum(cvar_excess) / (
261     (1.0 - parameters.cvar_alpha) * inputs.scenario_count
262 )
263 risk_term = 0.0 if cvar_expression is None else parameters.risk_weight * cvar_expression
264 model += planned_cost + expected_emergency_cost + risk_term - terminal_value_credit
265
266 solver = pulp.HiGHS(msg=False, threads=1)
267 started = perf_counter()
268 model.solve(solver)
269 runtime = perf_counter() - started
270 status = pulp.LpStatus[model.status]
271 if status != "Optimal":
272     raise RuntimeError(
273         f"HiGHS did not find an optimal Q2 solution for {inputs.date}: {status}"
274     )
275
276 emergency_values = np.array(
277     [[pulp.value(emergency[s][t]) for t in periods] for s in scenarios],
278     dtype=float,
279 )
280 spill_values = np.array(
281     [[pulp.value(spill[s][t]) for t in periods] for s in scenarios],
282     dtype=float,
283 )
284 planned_grid_used_values = np.array(
285     [[pulp.value(planned_grid_used[s][t]) for t in periods] for s in scenarios],
286     dtype=float,
287 )
288 grid_values = np.array([pulp.value(grid[t]) for t in periods], dtype=float)
289 unused_planned_grid_values = grid_values[None, :] - planned_grid_used_values
290 dispatch = pd.DataFrame(
291     {
292         "date": str(inputs.date),
293         "slot": np.arange(1, inputs.horizon + 1),
294         "timestamp": inputs.timestamps,
295         "price_yuan_per_kwh": inputs.price,
296         "load_forecast_kw": inputs.load_forecast,
297         "pv_forecast_kw": inputs.pv_forecast,
298         "planned_grid_kw": grid_values,
299         "charge_kw": [pulp.value(charge[t]) for t in periods],
300         "discharge_kw": [pulp.value(discharge[t]) for t in periods],
301         "planned_charge_limit_kw": [pulp.value(charge[t]) for t in periods],
302         "planned_discharge_limit_kw": [pulp.value(discharge[t]) for t in periods],
303         "storage_start_kwh": [pulp.value(energy[t]) for t in periods],
304         "storage_end_kwh": [pulp.value(energy[t + 1]) for t in periods],
305         "expected_emergency_kw": emergency_values.mean(axis=0),
306         "expected_spill_kw": spill_values.mean(axis=0),
307         "expected_planned_grid_used_kw": planned_grid_used_values.mean(axis=0),
308         "expected_unused_planned_grid_kw": unused_planned_grid_values.mean(axis=0),
309     }
310 )
311 for power_column in (
312     "planned_grid",
313     "charge",
314     "discharge",
315     "expected_emergency",
316     "expected_spill",
317     "expected_planned_grid_used",
318     "expected_unused_planned_grid",
319 ):
320     dispatch[f"{power_column}_kwh"] = (

```

```

321     dispatch[f"{power_column}_kw"] * inputs.dt_hours
322 )
323
324 planned_cost_value = float(pulp.value(planned_cost))
325 emergency_cost_value = float(pulp.value(expected_emergency_cost))
326 terminal_value_credit_value = float(pulp.value(terminal_value_credit))
327 scenario_emergency_cost_values = np.array(
328     [float(pulp.value(expression)) for expression in scenario_emergency_costs], dtype=float
329 )
330 empirical_var, empirical_cvar = calculate_empirical_cvar(
331     scenario_emergency_cost_values, parameters.cvar_alpha
332 )
333 if cvar_expression is None:
334     var_cost_value = empirical_var
335     cvar_cost_value = empirical_cvar
336     excess_values = np.maximum(scenario_emergency_cost_values - empirical_var, 0.0)
337 else:
338     var_cost_value = float(pulp.value(cvar_zeta))
339     cvar_cost_value = float(pulp.value(cvar_expression))
340     excess_values = np.array(
341         [float(pulp.value(cvar_excess[s])) for s in scenarios], dtype=float
342     )
343 return Q2DayResult(
344     date=inputs.date,
345     dispatch=dispatch,
346     emergency_kw=emergency_values,
347     spill_kw=spill_values,
348     planned_grid_used_kw=planned_grid_used_values,
349     unused_planned_grid_kw=unused_planned_grid_values,
350     scenario_source_dates=np.array(inputs.scenario_source_dates, copy=True),
351     status=status,
352     runtime_seconds=runtime,
353     solver_name="PuLP HiGHS",
354     planned_purchase_cost_yuan=planned_cost_value,
355     expected_emergency_cost_yuan=emergency_cost_value,
356     operating_cost_yuan=planned_cost_value + emergency_cost_value,
357     terminal_value_credit_yuan=terminal_value_credit_value,
358     optimization_objective_yuan=float(pulp.value(model.objective)),
359     scenario_emergency_cost_yuan=scenario_emergency_cost_values,
360     cvar_alpha=parameters.cvar_alpha,
361     risk_weight=parameters.risk_weight,
362     var_cost_yuan=var_cost_value,
363     cvar_cost_yuan=cvar_cost_value,
364     cvar_excess_cost_yuan=excess_values,
365     lambda_cvar_term_yuan=parameters.risk_weight * cvar_cost_value,
366     terminal_value_breakpoints_kwh=breakpoints,
367     terminal_value_rates_yuan_per_kwh=terminal_value_rates,
368 )

```

Listing 6 src/problem2/forecast_interface.py

```

1 """Read-only Stage 2B interface for the frozen Q2 forecast artifacts.
2
3 The interface deliberately exposes forecasts and paired scenarios, never the
4 future actual columns that are retained in the Stage 2A evaluation table.
5 It performs no forecasting, scenario generation, or dispatch optimization.
6 """
7
8 from __future__ import annotations
9
10 from dataclasses import dataclass
11 from datetime import date as Date
12 from functools import lru_cache
13 import hashlib

```

```

14 import json
15 from pathlib import Path
16
17 import numpy as np
18 import pandas as pd
19
20 from src.common.paths import PROCESSED_DATA_DIR, PROJECT_ROOT, RESULTS_DIR
21
22
23 HORIZON = 144
24 DT_HOURS = 1.0 / 6.0
25 FORMAL_LOAD_FORECASTER = "Last Week"
26 FORMAL_PV_FORECASTER = "7-day same-slot mean"
27
28 FORECAST_DIR = RESULTS_DIR / "tables" / "forecasting"
29 FORMAL_FORECAST_PATH = FORECAST_DIR / "q2_forecast_predictions.csv"
30 SCENARIO_PATH = FORECAST_DIR / "q2_scenarios.npz"
31 SCENARIO_MANIFEST_PATH = FORECAST_DIR / "q2_scenario_manifest.csv"
32 SELECTION_PATH = FORECAST_DIR / "forecast_selection_summary.csv"
33 MODEL_DECISION_PATH = FORECAST_DIR / "q2_model_decision.json"
34 LEAKAGE_AUDIT_PATH = FORECAST_DIR / "q2_leakage_audit.json"
35 FINAL_AUDIT_PATH = FORECAST_DIR / "forecast_final_leakage_audit.json"
36 PRICE_PATH = PROCESSED_DATA_DIR / "C题" / "problem1_day.csv"
37 RAW_ATTACHMENT_DIR = PROJECT_ROOT / "data" / "raw" / "C题" / "附件"
38
39
40 @dataclass(frozen=True)
41 class Q2DayInputs:
42     """Optimizer inputs for one operating day.
43
44     Units are kW for power, kWh for ``initial_energy``, yuan/kWh for
45     ``price``, and hours for ``dt_hours``. ``scenario_source_dates`` is one
46     shared provenance vector: row ``s`` of both scenario arrays comes from
47     that same residual-day draw.
48     """
49
50     date: Date
51     load_forecast: np.ndarray
52     pv_forecast: np.ndarray
53     load_scenarios: np.ndarray
54     pv_scenarios: np.ndarray
55     price: np.ndarray
56     initial_energy: float
57     timestamps: np.ndarray
58     scenario_source_dates: np.ndarray
59     dt_hours: float = DT_HOURS
60
61     @property
62     def horizon(self) -> int:
63         return int(self.load_forecast.shape[0])
64
65     @property
66     def scenario_count(self) -> int:
67         return int(self.load_scenarios.shape[0])
68
69
70 @dataclass(frozen=True)
71 class _FormalArtifacts:
72     forecasts: pd.DataFrame
73     target_dates: np.ndarray
74     source_residual_dates: np.ndarray
75     load_scenarios: np.ndarray
76     pv_scenarios: np.ndarray
77     price: np.ndarray
78     scenario_count: int
79

```

```

80
81 def _sha256(path: Path) -> str:
82     digest = hashlib.sha256()
83     content = path.read_bytes()
84     if path.suffix.lower() in {".csv", ".json"}:
85         # Git may check text artifacts out with CRLF on Windows. The frozen
86         # Stage 2A hashes were recorded from LF bytes, so canonicalize only
87         # line endings while retaining content-level change detection.
88         content = content.replace(b"\r\n", b"\n")
89     digest.update(content)
90     return digest.hexdigest()
91
92
93 def _read_json(path: Path) -> dict[str, object]:
94     return json.loads(path.read_text(encoding="utf-8"))
95
96
97 def _readonly(values: np.ndarray, *, dtype: object | None = None) -> np.ndarray:
98     result = np.array(values, dtype=dtype, copy=True)
99     result.setflags(write=False)
100     return result
101
102
103 @lru_cache(maxsize=1)
104 def _load_formal_artifacts() -> _FormalArtifacts:
105     """Load and validate the frozen Stage 2A artifacts exactly once."""
106     final_audit = _read_json(FINAL_AUDIT_PATH)
107     leakage_audit = _read_json(LEAKAGE_AUDIT_PATH)
108     decision = _read_json(MODEL_DECISION_PATH)
109     selection = pd.read_csv(SELECTION_PATH)
110
111     expected_hash = final_audit["protected_artifact_hashes_before"][SCENARIO_PATH.name]
112     if _sha256(SCENARIO_PATH) != expected_hash:
113         raise AssertionError("q2_scenarios.npz differs from the frozen Stage 2A artifact")
114     if final_audit["protected_artifact_hashes_before"] != final_audit[
115         "protected_artifact_hashes_after"
116     ]:
117         raise AssertionError("the Stage 2A protected-artifact audit is inconsistent")
118
119     if len(selection) != 1:
120         raise AssertionError("forecast_selection_summary.csv must contain one selection row")
121     if selection.iloc[0]["selected_generation_forecaster"] != FORMAL_PV_FORECASTER:
122         raise AssertionError("the formal PV forecaster is no longer 7-day same-slot mean")
123     if decision["selected_load_forecaster"] != FORMAL_LOAD_FORECASTER:
124         raise AssertionError("the formal load forecaster changed")
125     if decision["selected_generation_forecaster"] != "7-day mean":
126         raise AssertionError("the legacy Q2 decision file no longer identifies the 7-day mean")
127     if not bool(final_audit["strict_causal_protocol"]):
128         raise AssertionError("the final forecasting audit does not certify strict causality")
129     if not bool(leakage_audit["scenario_source_dates_strictly_before_target"]):
130         raise AssertionError("the scenario audit does not certify causal source dates")
131
132     forecasts = pd.read_csv(
133         FORMAL_FORECAST_PATH,
134         parse_dates=["operating_date", "datetime"],
135     ).sort_values(["operating_date", "datetime"], kind="stable")
136     required_columns = {
137         "operating_date",
138         "datetime",
139         "forecast_load",
140         "forecast_generation",
141         "load_last_week",
142         "same_slot_7d_mean",
143     }
144     missing = required_columns.difference(forecasts.columns)
145     if missing:

```

```

146     raise AssertionError(f"formal forecast table is missing columns: {sorted(missing)}")
147 if forecasts["datetime"].duplicated().any():
148     raise AssertionError("formal forecast timestamps must be unique")
149 if not np.isfinite(
150     forecasts[["forecast_load", "forecast_generation"]].to_numpy(dtype=float)
151 ).all():
152     raise AssertionError("formal point forecasts must be finite")
153 if not np.allclose(forecasts["forecast_load"], forecasts["load_last_week"]):
154     raise AssertionError("formal load values no longer match the selected Last Week model")
155 if not np.allclose(
156     forecasts["forecast_generation"], forecasts["same_slot_7d_mean"]
157 ):
158     raise AssertionError("formal PV values no longer match the selected 7-day mean")
159
160 with np.load(SCENARIO_PATH, allow_pickle=False) as archive:
161     required_keys = {
162         "target_dates",
163         "source_residual_dates",
164         "load_kw",
165         "generation_kw",
166     }
167     if set(archive.files) != required_keys:
168         raise AssertionError("unexpected q2_scenarios.npz schema")
169     target_dates = archive["target_dates"].astype("datetime64[D]", copy=True)
170     source_dates = archive["source_residual_dates"].astype("datetime64[D]", copy=True)
171     load_scenarios = archive["load_kw"].copy()
172     pv_scenarios = archive["generation_kw"].copy()
173
174 if load_scenarios.shape != pv_scenarios.shape or load_scenarios.ndim != 3:
175     raise AssertionError("load and PV scenarios must share one [day, scenario, time] shape")
176 day_count, scenario_count, horizon = load_scenarios.shape
177 if horizon != HORIZON:
178     raise AssertionError(f"forecast horizon must be {HORIZON}, got {horizon}")
179 if target_dates.shape != (day_count,):
180     raise AssertionError("scenario target-date dimension does not match scenario arrays")
181 if source_dates.shape != (day_count, scenario_count):
182     raise AssertionError("one shared source-date index is required for every paired scenario")
183 if not np.isfinite(load_scenarios).all() or not np.isfinite(pv_scenarios).all():
184     raise AssertionError("scenario arrays contain NaN or infinite values")
185 if (pv_scenarios < 0).any():
186     raise AssertionError("PV scenarios violate the physical lower bound zero")
187 if not (source_dates < target_dates[:, None]).all():
188     raise AssertionError("a residual scenario uses its target day or a future day")
189
190 formal_dates = forecasts["operating_date"].drop_duplicates().to_numpy(dtype="datetime64[D]")
191 if not np.array_equal(formal_dates, target_dates):
192     raise AssertionError("forecast and scenario target dates are not aligned")
193 daily_counts = forecasts.groupby("operating_date", sort=True).size().to_numpy()
194 if not (daily_counts == HORIZON).all():
195     raise AssertionError("every formal forecast day must contain 144 intervals")
196 for target_date, daily in forecasts.groupby("operating_date", sort=True):
197     expected = pd.date_range(
198         pd.Timestamp(target_date) + pd.Timedelta(minutes=10),
199         periods=HORIZON,
200         freq="10min",
201     ).to_numpy(dtype="datetime64[ns]")
202     observed = daily["datetime"].to_numpy(dtype="datetime64[ns]")
203     if not np.array_equal(observed, expected):
204         raise AssertionError(f"forecast timestamps are misaligned for {target_date.date()}")
205
206 manifest = pd.read_csv(SCENARIO_MANIFEST_PATH, parse_dates=["target_date"])
207 manifest_dates = manifest["target_date"].to_numpy(dtype="datetime64[D]")
208 if not np.array_equal(manifest_dates, target_dates):
209     raise AssertionError("scenario manifest dates do not match the archive")
210 if not (manifest["scenario_count"].to_numpy() == scenario_count).all():
211     raise AssertionError("scenario count is inconsistent across the manifest")

```

```

212 if not (manifest["day_steps"].to_numpy() == HORIZON).all():
213     raise AssertionError("scenario manifest horizon is not 144")
214 if not manifest["all_sources_strictly_before_target"].astype(bool).all():
215     raise AssertionError("scenario manifest reports future leakage")
216 if scenario_count != int(leakage_audit["scenario_count_per_day"]):
217     raise AssertionError("scenario count differs from the leakage audit")
218
219 price_frame = pd.read_csv(PRICE_PATH).sort_values("slot", kind="stable")
220 if price_frame["slot"].tolist() != list(range(1, HORIZON + 1)):
221     raise AssertionError("Q2 fixed price profile must use slots 1 through 144")
222 price = price_frame["price_yuan_per_kwh"].to_numpy(dtype=float)
223 if price.shape != (HORIZON,) or not np.isfinite(price).all():
224     raise AssertionError("Q2 price profile must contain 144 finite yuan/kWh values")
225
226 return _FormalArtifacts(
227     forecasts=forecasts,
228     target_dates=target_dates,
229     source_residual_dates=source_dates,
230     load_scenarios=load_scenarios,
231     pv_scenarios=pv_scenarios,
232     price=price,
233     scenario_count=scenario_count,
234 )
235
236
237 def get_q2_day_inputs(date: object, initial_energy: float) -> Q2DayInputs:
238     """Return frozen forecasts and paired scenarios for one formal Q2 day.
239
240     ``initial_energy`` is supplied by the caller. Stage 2B must pass the
241     previous day's terminal battery energy after the first day; this function
242     intentionally does not impose Q1's daily ``6000 -> 6000`` condition.
243     """
244     target = pd.Timestamp(date)
245     if pd.isna(target):
246         raise ValueError("date must be a valid operating date")
247     target = target.normalize()
248     initial = float(initial_energy)
249     if not np.isfinite(initial):
250         raise ValueError("initial_energy must be finite and expressed in kWh")
251
252     artifacts = _load_formal_artifacts()
253     target_day = np.datetime64(target.date(), "D")
254     positions = np.flatnonzero(artifacts.target_dates == target_day)
255     if positions.size != 1:
256         first = str(artifacts.target_dates[0])
257         last = str(artifacts.target_dates[-1])
258         raise KeyError(f"{target.date()} is outside the formal Q2 period {first} to {last}")
259     position = int(positions[0])
260     daily = artifacts.forecasts.loc[
261         artifacts.forecasts["operating_date"] == target
262     ].sort_values("datetime", kind="stable")
263
264     timestamps = daily["datetime"].to_numpy(dtype="datetime64[ns]")
265     load_forecast = daily["forecast_load"].to_numpy(dtype=float)
266     pv_forecast = daily["forecast_generation"].to_numpy(dtype=float)
267     load_scenarios = artifacts.load_scenarios[position]
268     pv_scenarios = artifacts.pv_scenarios[position]
269     source_dates = artifacts.source_residual_dates[position]
270
271     if load_forecast.shape != (HORIZON,) or pv_forecast.shape != (HORIZON,):
272         raise AssertionError("point forecast shape changed after date selection")
273     if load_scenarios.shape != (artifacts.scenario_count, HORIZON):
274         raise AssertionError("load scenario shape changed after date selection")
275     if pv_scenarios.shape != load_scenarios.shape:
276         raise AssertionError("paired load/PV scenario shapes differ")
277     if not np.isfinite(

```

```

278     np.concatenate(
279         [
280             load_forecast,
281             pv_forecast,
282             load_scenarios.reshape(-1),
283             pv_scenarios.reshape(-1),
284             artifacts.price,
285         ]
286     )
287 ).all():
288     raise AssertionError("optimizer input contains NaN or infinite values")
289 if (pv_scenarios < 0).any():
290     raise AssertionError("PV scenario is negative")
291 if not (source_dates < target_day).all():
292     raise AssertionError("scenario source date is not strictly before its target")
293
294 return Q2DayInputs(
295     date=target.date(),
296     load_forecast=_readonly(load_forecast, dtype=float),
297     pv_forecast=_readonly(pv_forecast, dtype=float),
298     load_scenarios=_readonly(load_scenarios),
299     pv_scenarios=_readonly(pv_scenarios),
300     price=_readonly(artifacts.price, dtype=float),
301     initial_energy=initial,
302     timestamps=_readonly(timestamps),
303     scenario_source_dates=_readonly(source_dates),
304 )
305
306
307 def audit_q2_handoff_integrity() -> dict[str, object]:
308     """Recompute the Stage 2A hash and causality checks used by handoff tests."""
309     artifacts = _load_formal_artifacts()
310     final_audit = _read_json(FINAL_AUDIT_PATH)
311     protected_expected = final_audit["protected_artifact_hashes_before"]
312     protected_current = {
313         name: _sha256(FORECAST_DIR / name) for name in protected_expected
314     }
315     raw_expected = final_audit["raw_excel_hashes_before"]
316     raw_current = {
317         name: _sha256(RAW_ATTACHMENT_DIR / name) for name in raw_expected
318     }
319     source_is_causal = bool(
320         (
321             artifacts.source_residual_dates
322             < artifacts.target_dates[:, None]
323         ).all()
324     )
325     return {
326         "formal_load_forecaster": FORMAL_LOAD_FORECASTER,
327         "formal_pv_forecaster": FORMAL_PV_FORECASTER,
328         "formal_start": str(artifacts.target_dates[0]),
329         "formal_end": str(artifacts.target_dates[-1]),
330         "formal_day_count": int(artifacts.target_dates.size),
331         "forecast_horizon": HORIZON,
332         "scenario_count": artifacts.scenario_count,
333         "paired_scenarios": artifacts.load_scenarios.shape
334         == artifacts.pv_scenarios.shape,
335         "leakage_audit": source_is_causal
336         and bool(final_audit["strict_causal_protocol"]),
337         "protected_artifact_hashes_unchanged": protected_current
338         == protected_expected,
339         "q2_scenario_hash_unchanged": protected_current[SCENARIO_PATH.name]
340         == protected_expected[SCENARIO_PATH.name],
341         "raw_data_hashes_unchanged": raw_current == raw_expected,
342     }
343

```

```

344
345 _all_ = [
346     "DT_HOURS",
347     "FORMAL_LOAD_FORECASTER",
348     "FORMAL_PV_FORECASTER",
349     "HORIZON",
350     "Q2DayInputs",
351     "audit_q2_handoff_integrity",
352     "get_q2_day_inputs",
353 ]

```

3.3 问题三滚动优化

Listing 7 src/problem3/forecast_fusion.py

```

1  """Strictly causal Q3 forecast baselines, convex fusion and selection."""
2
3  from __future__ import annotations
4
5  from collections import defaultdict
6
7  import numpy as np
8  import pandas as pd
9  from scipy.stats import wilcoxon
10
11 from src.problem3.forecast_analysis import comparison_metrics, metric_row
12 from src.problem3.forecast_data import LEAD_BIN_LABELS, RELEASE_HOURS
13
14
15 CALIBRATION_START = pd.Timestamp("2025-01-08")
16 CALIBRATION_END = pd.Timestamp("2025-01-16 23:59:59")
17 VALIDATION_START = pd.Timestamp("2025-01-17")
18 VALIDATION_END = pd.Timestamp("2025-01-30 23:59:59")
19 EVALUATION_START = pd.Timestamp("2025-02-01")
20 METHOD_COLUMNS = {
21     "官方预测": "forecast_pv_kw",
22     "昨日同刻": "yesterday_pv_kw",
23     "上周同刻": "last_week_pv_kw",
24     "七日同刻均值": "same_slot_7d_mean_kw",
25     "静态凸融合": "static_fusion_kw",
26     "提前期分箱融合": "lead_aware_fusion_kw",
27     "发布时间分组融合": "release_aware_fusion_kw",
28     "在线因果融合": "online_causal_fusion_kw",
29 }
30
31
32 def fit_convex_weight(actual: np.ndarray, official: np.ndarray, seasonal: np.ndarray) -> float:
33     """Fit the official-model weight for a two-expert convex MSE projection."""
34
35     truth = np.asarray(actual, dtype=float)
36     x = np.asarray(official, dtype=float) - np.asarray(seasonal, dtype=float)
37     y = truth - np.asarray(seasonal, dtype=float)
38     valid = np.isfinite(x) & np.isfinite(y)
39     denominator = float(np.dot(x[valid], x[valid]))
40     if denominator <= 0:
41         return 0.5
42     return float(np.clip(np.dot(x[valid], y[valid]) / denominator, 0.0, 1.0))
43
44
45 def _apply_weight(official: pd.Series, seasonal: pd.Series, weight: np.ndarray | float) -> np.ndarray:
46     result = np.asarray(weight) * official.to_numpy(float) + (1.0 - np.asarray(weight)) * seasonal.to_numpy(float)
47     return np.maximum(result, 0.0)
48

```

```

49
50 def causal_online_fusion(frame: pd.DataFrame, eta: float) -> tuple[np.ndarray, np.ndarray]:
51     """Aggregate official/seasonal experts, updating only after truth is available."""
52
53     ordered = frame.sort_values(["issue_time", "lead_hours"], kind="stable")
54     prediction = pd.Series(np.nan, index=frame.index, dtype=float)
55     official_weight = pd.Series(np.nan, index=frame.index, dtype=float)
56     log_weights = np.zeros(2, dtype=float)
57     pending: list[int] = []
58     for issue_time, rows in ordered.groupby("issue_time", sort=True):
59         still_pending: list[int] = []
60         for index in pending:
61             if frame.at[index, "target_time"] <= issue_time:
62                 experts = frame.loc[index, ["forecast_pv_kw", "same_slot_7d_mean_kw"]].to_numpy(float)
63                 truth = float(frame.at[index, "actual_pv_kw"])
64                 if np.isfinite(experts).all() and np.isfinite(truth):
65                     log_weights -= float(eta) * np.abs(experts - truth) / 6000.0
66             else:
67                 still_pending.append(index)
68         pending = still_pending
69         weights = np.exp(log_weights - np.max(log_weights))
70         weights /= weights.sum()
71         indices = rows.index.to_numpy()
72         experts = rows[["forecast_pv_kw", "same_slot_7d_mean_kw"]].to_numpy(float)
73         valid = np.isfinite(experts).all(axis=1)
74         values = np.full(len(rows), np.nan)
75         values[valid] = experts[valid] @ weights
76         prediction.loc[indices] = np.maximum(values, 0.0)
77         official_weight.loc[indices] = weights[0]
78         pending.extend(indices.tolist())
79     return prediction.to_numpy(float), official_weight.to_numpy(float)
80
81
82 def _period_labels(frame: pd.DataFrame) -> pd.Series:
83     issue = frame["issue_time"]
84     return pd.Series(
85         np.select(
86             [
87                 issue.between(CALIBRATION_START, CALIBRATION_END),
88                 issue.between(VVALIDATION_START, VALIDATION_END),
89                 issue.ge(EVALUATION_START),
90             ],
91             ["calibration", "validation", "evaluation"],
92             default="excluded",
93         ),
94         index=frame.index,
95     )
96
97
98 def _bh_fdr(pvalues: np.ndarray) -> np.ndarray:
99     values = np.asarray(pvalues, dtype=float)
100     order = np.argsort(values)
101     adjusted = np.empty_like(values)
102     ranked = values[order] * len(values) / np.arange(1, len(values) + 1)
103     ranked = np.minimum.accumulate(ranked[::-1])[::-1]
104     adjusted[order] = np.clip(ranked, 0.0, 1.0)
105     return adjusted
106
107
108 def paired_significance(frame: pd.DataFrame, selected_column: str) -> pd.DataFrame:
109     """Compare daily paired MAE/RMSE on the Feb-Dec evaluation period."""
110
111     evaluation = frame.loc[frame["period"].eq("evaluation")].copy()
112     evaluation["target_date"] = evaluation["issue_time"].dt.normalize()
113     rows: list[dict[str, object]] = []
114     for method, column in METHOD_COLUMNS.items():

```

```

115     if column == selected_column:
116         continue
117     for metric in ("MAE", "RMSE"):
118         paired = []
119         for _, day in evaluation.groupby("target_date", sort=True):
120             selected_error = day[selected_column].to_numpy(float) - day["actual_pv_kw"].to_numpy(float)
121             comparator_error = day[column].to_numpy(float) - day["actual_pv_kw"].to_numpy(float)
122             valid = np.isfinite(selected_error) & np.isfinite(comparator_error)
123             if not valid.any():
124                 continue
125             if metric == "MAE":
126                 selected_value = float(np.mean(np.abs(selected_error[valid])))
127                 comparator_value = float(np.mean(np.abs(comparator_error[valid])))
128             else:
129                 selected_value = float(np.sqrt(np.mean(np.square(selected_error[valid]))))
130                 comparator_value = float(np.sqrt(np.mean(np.square(comparator_error[valid]))))
131             paired.append((selected_value, comparator_value))
132         values = np.asarray(paired, dtype=float)
133         difference = values[:, 0] - values[:, 1]
134         if np.allclose(difference, 0.0):
135             statistic, pvalue = 0.0, 1.0
136         else:
137             statistic, pvalue = wilcoxon(values[:, 0], values[:, 1], zero_method="wilcox")
138         rows.append(
139             {
140                 "selected_method": "提前期分箱融合",
141                 "comparator": method,
142                 "pairing_unit": "目标日期",
143                 "metric": metric,
144                 "n_days": int(len(values)),
145                 "median_selected_minus_comparator_kw": float(np.median(difference)),
146                 "wilcoxon_statistic": float(statistic),
147                 "p_value": float(pvalue),
148             }
149         )
150     table = pd.DataFrame(rows)
151     table["p_value_bh_fdr"] = _bh_fdr(table["p_value"].to_numpy(float))
152     return table
153
154
155 def build_fusion_analysis(frame: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, pd.DataFrame, dict[str, object]]:
156     """Fit on early January, select on late January and evaluate Feb-Dec."""
157
158     work = frame.loc[frame["actual_pv_kw"].notna()].copy()
159     work["period"] = _period_labels(work)
160     common = work.loc[
161         work[list(METHOD_COLUMNS.values())[4]].notna().all(axis=1)
162         & work["period"].ne("excluded")
163     ].copy()
164     validation = common.loc[common["period"].eq("validation")]
165     seasonal_columns = {
166         "昨日同刻": "yesterday_pv_kw",
167         "上周同刻": "last_week_pv_kw",
168         "七日同刻均值": "same_slot_7d_mean_kw",
169     }
170     seasonal_scores = {
171         name: metric_row(validation["actual_pv_kw"], validation[column])["rmse_kw"]
172         for name, column in seasonal_columns.items()
173     }
174     selected_seasonal = min(seasonal_scores, key=seasonal_scores.get)
175     seasonal_column = seasonal_columns[selected_seasonal]
176     calibration = common.loc[common["period"].eq("calibration")]
177     global_weight = fit_convex_weight(
178         calibration["actual_pv_kw"], calibration["forecast_pv_kw"], calibration[seasonal_column]
179     )
180     lead_weights = {

```

```

181     label: fit_convex_weight(
182         rows["actual_pv_kw"], rows["forecast_pv_kw"], rows[seasonal_column]
183     )
184     for label, rows in calibration.groupby("lead_bin", observed=True)
185 }
186 release_weights = {
187     int(hour): fit_convex_weight(
188         rows["actual_pv_kw"], rows["forecast_pv_kw"], rows[seasonal_column]
189     )
190     for hour, rows in calibration.groupby("release_hour")
191 }
192 common["static_fusion_kw"] = _apply_weight(
193     common["forecast_pv_kw"], common[seasonal_column], global_weight
194 )
195 common["lead_aware_fusion_kw"] = _apply_weight(
196     common["forecast_pv_kw"],
197     common[seasonal_column],
198     common["lead_bin"].map(lead_weights).to_numpy(float),
199 )
200 common["release_aware_fusion_kw"] = _apply_weight(
201     common["forecast_pv_kw"],
202     common[seasonal_column],
203     common["release_hour"].map(release_weights).to_numpy(float),
204 )
205 eta_candidates = (0.5, 1.0, 2.0, 4.0, 8.0)
206 eta_scores: dict[float, float] = {}
207 online_cache: dict[float, np.ndarray] = {}
208 for eta in eta_candidates:
209     values, _ = causal_online_fusion(common, eta)
210     online_cache[eta] = values
211     mask = common["period"].eq("calibration")
212     eta_scores[eta] = float(metric_row(common.loc[mask, "actual_pv_kw"], values[mask])["rmse_kw"])
213 selected_eta = min(eta_scores, key=eta_scores.get)
214 common["online_causal_fusion_kw"] = online_cache[selected_eta]
215
216 comparison = comparison_metrics(common, METHOD_COLUMNS)
217 validation_overall = comparison.loc[
218     comparison["period"].eq("validation")
219     & comparison["scope_type"].eq("overall")
220 ].sort_values(["rmse_kw", "mae_kw"], kind="stable")
221 selected_method = str(validation_overall.iloc[0]["method"])
222 selected_column = METHOD_COLUMNS[selected_method]
223 significance = paired_significance(common, selected_column)
224 weights = pd.DataFrame(
225     [
226         {"weight_type": "global", "group": "all", "official_weight": global_weight},
227         *[
228             {"weight_type": "lead_bin", "group": label, "official_weight": lead_weights[label]}
229             for label in LEAD_BIN_LABELS
230         ],
231         *[
232             {"weight_type": "release_hour", "group": str(hour), "official_weight": release_weights[hour]}
233             for hour in RELEASE_HOURS
234         ],
235     ]
236 )
237 weights["seasonal_weight"] = 1.0 - weights["official_weight"]
238 validation_row = validation_overall.loc[validation_overall["method"].eq(selected_method)].iloc[0]
239 evaluation_row = comparison.loc[
240     comparison["period"].eq("evaluation")
241     & comparison["scope_type"].eq("overall")
242     & comparison["method"].eq(selected_method)
243 ].iloc[0]
244 selection = {
245     "calibration_period": "2025-01-08/2025-01-16",
246     "validation_period": "2025-01-17/2025-01-30",

```

```

247     "evaluation_period": "2025-02-01/2025-12-31",
248     "selected_seasonal_method": selected_seasonal,
249     "selected_method": selected_method,
250     "selected_column": selected_column,
251     "selected_online_eta": selected_eta,
252     "validation_mae_kw": float(validation_row["mae_kw"]),
253     "validation_rmse_kw": float(validation_row["rmse_kw"]),
254     "evaluation_mae_kw": float(evaluation_row["mae_kw"]),
255     "evaluation_rmse_kw": float(evaluation_row["rmse_kw"]),
256     "lead_aware": selected_method == "提前期分箱融合",
257     "release_aware": selected_method == "发布时间分组融合",
258     "structural_expert_included": False,
259     "structural_expert_reason": "现有结构化专家没有覆盖四个发布批次的同一整点协议，故不作不公平拼接。",
260     "fusion_is_causal": True,
261     "lead_weights": lead_weights,
262     "release_weights": {str(k): v for k, v in release_weights.items()},
263     "global_official_weight": global_weight,
264 }
265 return common, comparison, significance, weights, selection
266
267
268 _all_ = [
269     "EVALUATION_START",
270     "METHOD_COLUMNS",
271     "VALIDATION_START",
272     "build_fusion_analysis",
273     "causal_online_fusion",
274     "fit_convex_weight",
275     "paired_significance",
276 ]

```

Listing 8 src/problem3/rolling_model.py

```

1  """Remaining-horizon Q3 planning MILP using the locked Q2 equations.
2
3  This module owns only the day-ahead re-optimization wrapper. Actual physical
4  settlement is deliberately delegated to ``src.problem2.run.settle_realized_day``.
5  """
6
7  from __future__ import annotations
8
9  from dataclasses import dataclass
10 from time import perf_counter
11
12 import numpy as np
13 import pandas as pd
14 import pulp
15
16 from src.problem2.model import Q2DispatchParameters
17 from src.problem2.forecast_interface import Q2DayInputs
18
19
20 @dataclass(frozen=True)
21 class Q3RollingPlanResult:
22     date: object
23     dispatch: pd.DataFrame
24     emergency_kw: np.ndarray
25     spill_kw: np.ndarray
26     planned_grid_used_kw: np.ndarray
27     unused_planned_grid_kw: np.ndarray
28     scenario_source_dates: np.ndarray
29     status: str
30     runtime_seconds: float
31     solver_name: str

```

```

32 planned_purchase_cost_yuan: float
33 adjustment_cost_yuan: float
34 expected_emergency_cost_yuan: float
35 operating_cost_yuan: float
36 terminal_value_credit_yuan: float
37 optimization_objective_yuan: float
38 terminal_value_breakpoints_kwh: tuple[float, ...]
39 terminal_value_rates_yuan_per_kwh: tuple[float, ...]
40 maximum_scenario_power_balance_residual_kw: float
41
42 @property
43 def final_energy_kwh(self) -> float:
44     return float(self.dispatch["storage_end_kwh"].iloc[-1])
45
46
47 def _terminal_segments(
48     inputs: Q2DayInputs, parameters: Q2DispatchParameters
49 ) -> tuple[tuple[float, ...], tuple[float, ...]]:
50     net = inputs.load_scenarios - inputs.pv_scenarios
51     point = inputs.load_forecast - inputs.pv_forecast
52     positive_error = np.maximum(net - point[None, :], 0.0).sum(axis=1) * inputs.dt_hours
53     reserve = float(
54         np.clip(
55             parameters.minimum_energy_kwh
56             + np.quantile(positive_error, parameters.terminal_reserve_quantile),
57             parameters.minimum_energy_kwh + 1.0,
58             parameters.maximum_energy_kwh - 2.0,
59         )
60     )
61     midpoint = reserve + (parameters.maximum_energy_kwh - reserve) / 2.0
62     return (
63         (reserve, midpoint, parameters.maximum_energy_kwh),
64         tuple(
65             parameters.discharge_efficiency * float(np.quantile(inputs.price, q))
66             for q in parameters.terminal_value_price_quantiles
67         ),
68     )
69
70
71 def solve_remaining_dispatch(
72     inputs: Q2DayInputs,
73     parameters: Q2DispatchParameters | None = None,
74     *,
75     prior_commitment_kw: np.ndarray | None = None,
76 ) -> Q3RollingPlanResult:
77     """Solve one remaining Q3 horizon with the locked Q2 physical equations.
78
79     At 00:00, grid energy is priced at the ordinary tariff. At later legal
80     releases, the already paid commitment is sunk and the decision objective
81     uses the official incremental settlement: 1.5p for an increase and a
82     0.5p credit for a decrease, both relative to the previous commitment.
83     """
84
85     parameters = parameters or Q2DispatchParameters()
86     if inputs.horizon < 1 or inputs.horizon > 144:
87         raise ValueError(f"remaining Q3 horizon must lie in [1, 144], got {inputs.horizon}")
88     if inputs.scenario_count != 50:
89         raise ValueError("Q3 requires the locked 50 paired scenarios")
90     expected_shape = (inputs.scenario_count, inputs.horizon)
91     if inputs.load_scenarios.shape != expected_shape or inputs.pv_scenarios.shape != expected_shape:
92         raise ValueError("Q3 scenario arrays must have shape [50, remaining horizon]")
93     numeric = np.concatenate(
94         [
95             inputs.load_forecast,
96             inputs.pv_forecast,
97             inputs.load_scenarios.reshape(-1),

```

```

98     inputs.pv_scenarios.reshape(-1),
99     inputs.price,
100     np.array([inputs.initial_energy, inputs.dt_hours]),
101 ]
102 )
103 if not np.isfinite(numeric).all():
104     raise ValueError("Q3 rolling input contains NaN or infinite values")
105 if (inputs.pv_scenarios < 0).any():
106     raise ValueError("Q3 PV scenarios must be nonnegative")
107 if not parameters.minimum_energy_kwh <= inputs.initial_energy <= parameters.maximum_energy_kwh:
108     raise ValueError("Q3 initial energy is outside the permitted battery range")
109 if prior_commitment_kw is not None:
110     prior_commitment_kw = np.asarray(prior_commitment_kw, dtype=float)
111     if prior_commitment_kw.shape != (inputs.horizon,):
112         raise ValueError("prior Q3 commitment must match the remaining horizon")
113     if not np.isfinite(prior_commitment_kw).all() or (prior_commitment_kw < -2e-6).any():
114         raise ValueError("prior Q3 commitment must be finite and nonnegative")
115     prior_commitment_kw = np.maximum(prior_commitment_kw, 0.0)
116 breakpoints, rates = _terminal_segments(inputs, parameters)
117 periods = range(inputs.horizon)
118 scenarios = range(inputs.scenario_count)
119 model = pulp.LpProblem(f"CUMCM2026_Q3_Rolling_{inputs.date}", pulp.LpMinimize)
120 grid = pulp.LpVariable.dicts("q3_planned_grid_kw", periods, lowBound=0)
121 charge = pulp.LpVariable.dicts("q3_planned_charge_kw", periods, lowBound=0)
122 discharge = pulp.LpVariable.dicts("q3_planned_discharge_kw", periods, lowBound=0)
123 mode = pulp.LpVariable.dicts("q3_charge_mode", periods, cat=pulp.LpBinary)
124 energy = pulp.LpVariable.dicts(
125     "q3_energy_kwh",
126     range(inputs.horizon + 1),
127     lowBound=parameters.minimum_energy_kwh,
128     upBound=parameters.maximum_energy_kwh,
129 )
130 emergency = pulp.LpVariable.dicts("q3_emergency_kw", (scenarios, periods), lowBound=0)
131 spill = pulp.LpVariable.dicts("q3_spill_kw", (scenarios, periods), lowBound=0)
132 used = pulp.LpVariable.dicts("q3_planned_grid_used_kw", (scenarios, periods), lowBound=0)
133 increase = None
134 decrease = None
135 if prior_commitment_kw is not None:
136     increase = pulp.LpVariable.dicts("q3_adjustment_increase_kw", periods, lowBound=0)
137     decrease = pulp.LpVariable.dicts("q3_adjustment_decrease_kw", periods, lowBound=0)
138 segment_vars = [
139     pulp.LpVariable(
140         f"q3_terminal_segment_{index + 1}_kwh", lowBound=0, upBound=upper - lower
141     )
142     for index, (lower, upper) in enumerate(
143         zip((parameters.minimum_energy_kwh,) + breakpoints[:-1], breakpoints)
144     )
145 ]
146 model += energy[0] == inputs.initial_energy, "q3_initial_energy"
147 model += energy[inputs.horizon] == parameters.minimum_energy_kwh + pulp.lpSum(segment_vars)
148 for t in periods:
149     model += energy[t + 1] == (
150         energy[t]
151         + parameters.charge_efficiency * charge[t] * inputs.dt_hours
152         - discharge[t] * inputs.dt_hours / parameters.discharge_efficiency
153     )
154     model += charge[t] <= parameters.maximum_charge_kw * mode[t]
155     model += discharge[t] <= parameters.maximum_discharge_kw * (1 - mode[t])
156     if prior_commitment_kw is not None:
157         assert increase is not None and decrease is not None
158         model += (
159             grid[t] - float(prior_commitment_kw[t]) == increase[t] - decrease[t]
160         )
161     for s in scenarios:
162         model += (
163             used[s][t]

```

```

164         + emergency[s][t]
165         + float(inputs.pv_scenarios[s, t])
166         + discharge[t]
167         == float(inputs.load_scenarios[s, t]) + charge[t] + spill[s][t]
168     )
169     model += used[s][t] <= grid[t]
170     model += spill[s][t] <= float(inputs.pv_scenarios[s, t])
171     planned_cost = pulp.lpSum(
172         float(inputs.price[t]) * grid[t] * inputs.dt_hours for t in periods
173     )
174     if prior_commitment_kw is None:
175         decision_energy_cost = planned_cost
176     else:
177         assert increase is not None and decrease is not None
178         decision_energy_cost = pulp.lpSum(
179             float(inputs.price[t])
180             * inputs.dt_hours
181             * (1.5 * increase[t] - 0.5 * decrease[t])
182             for t in periods
183         )
184     scenario_costs = [
185         pulp.lpSum(
186             parameters.emergency_price_multiplier
187             * float(inputs.price[t])
188             * emergency[s][t]
189             * inputs.dt_hours
190             for t in periods
191         )
192         for s in scenarios
193     ]
194     expected_emergency = pulp.lpSum(scenario_costs) / inputs.scenario_count
195     terminal_credit = pulp.lpSum(rate * variable for rate, variable in zip(rates, segment_vars))
196     model += decision_energy_cost + expected_emergency - terminal_credit
197     solver = pulp.HiGHS(msg=False, threads=1)
198     started = perf_counter()
199     model.solve(solver)
200     runtime = perf_counter() - started
201     status = pulp.LpStatus[model.status]
202     if status != "Optimal":
203         raise RuntimeError(f"Q3 rolling MILP failed for {inputs.date}: {status}")
204     emergency_values = np.array(
205         [[pulp.value(emergency[s][t]) for t in periods] for s in scenarios], dtype=float
206     )
207     spill_values = np.array(
208         [[pulp.value(spill[s][t]) for t in periods] for s in scenarios], dtype=float
209     )
210     used_values = np.array(
211         [[pulp.value(used[s][t]) for t in periods] for s in scenarios], dtype=float
212     )
213     grid_values = np.array([pulp.value(grid[t]) for t in periods], dtype=float)
214     if (grid_values < -2e-6).any():
215         raise AssertionError("Q3 solver returned materially negative planned purchase")
216     grid_values = np.maximum(grid_values, 0.0)
217     unused_values = grid_values[None, :] - used_values
218     charge_values = np.array([pulp.value(charge[t]) for t in periods], dtype=float)
219     discharge_values = np.array([pulp.value(discharge[t]) for t in periods], dtype=float)
220     scenario_balance = (
221         used_values
222         + emergency_values
223         + inputs.pv_scenarios
224         + discharge_values[None, :]
225         - inputs.load_scenarios
226         - charge_values[None, :]
227         - spill_values
228     )
229     maximum_scenario_balance = float(np.max(np.abs(scenario_balance)))

```

```

230 if maximum_scenario_balance > 2e-3:
231     raise AssertionError(
232         f"Q3 scenario power balance residual is {maximum_scenario_balance:.6g} kW"
233     )
234 frame = pd.DataFrame(
235     {
236         "date": str(inputs.date),
237         "slot": np.arange(1, inputs.horizon + 1),
238         "timestamp": inputs.timestamps,
239         "price_yuan_per_kwh": inputs.price,
240         "load_forecast_kw": inputs.load_forecast,
241         "pv_forecast_kw": inputs.pv_forecast,
242         "planned_grid_kw": grid_values,
243         "charge_kw": charge_values,
244         "discharge_kw": discharge_values,
245         "planned_charge_limit_kw": charge_values,
246         "planned_discharge_limit_kw": discharge_values,
247         "storage_start_kwh": [pulp.value(energy[t]) for t in periods],
248         "storage_end_kwh": [pulp.value(energy[t + 1]) for t in periods],
249         "expected_emergency_kw": emergency_values.mean(axis=0),
250         "expected_spill_kw": spill_values.mean(axis=0),
251         "expected_planned_grid_used_kw": used_values.mean(axis=0),
252         "expected_unused_planned_grid_kw": unused_values.mean(axis=0),
253     }
254 )
255 for column in (
256     "planned_grid", "charge", "discharge", "expected_emergency", "expected_spill",
257     "expected_planned_grid_used", "expected_unused_planned_grid",
258 ):
259     frame[f"{column}_kwh"] = frame[f"{column}_kw"] * inputs.dt_hours
260 planned_value = float(pulp.value(planned_cost))
261 adjustment_value = 0.0 if prior_commitment_kw is None else float(pulp.value(decision_energy_cost))
262 expected_value = float(pulp.value(expected_emergency))
263 terminal_value = float(pulp.value(terminal_credit))
264 return Q3RollingPlanResult(
265     date=inputs.date,
266     dispatch=frame,
267     emergency_kw=emergency_values,
268     spill_kw=spill_values,
269     planned_grid_used_kw=used_values,
270     unused_planned_grid_kw=unused_values,
271     scenario_source_dates=np.array(inputs.scenario_source_dates, copy=True),
272     status=status,
273     runtime_seconds=runtime,
274     solver_name="PuLP HiGHS",
275     planned_purchase_cost_yuan=planned_value,
276     adjustment_cost_yuan=adjustment_value,
277     expected_emergency_cost_yuan=expected_value,
278     operating_cost_yuan=planned_value + expected_value,
279     terminal_value_credit_yuan=terminal_value,
280     optimization_objective_yuan=float(pulp.value(model.objective)),
281     terminal_value_breakpoints_kwh=breakpoints,
282     terminal_value_rates_yuan_per_kwh=rates,
283     maximum_scenario_power_balance_residual_kw=maximum_scenario_balance,
284 )
285
286
287 __all__ = ["Q3RollingPlanResult", "solve_remaining_dispatch"]

```

Listing 9 src/problem3/rolling_dispatch.py

```

1 """Causal Q3 rolling optimization, physical execution, settlement, and VOI."""
2
3 from __future__ import annotations

```

```

4
5 from dataclasses import dataclass, replace
6 import pickle
7 from pathlib import Path
8 from time import perf_counter
9 from typing import Callable
10
11 import numpy as np
12 import pandas as pd
13
14 from src.problem2.forecast_interface import Q2DayInputs, get_q2_day_inputs
15 from src.problem2.model import Q2DispatchParameters
16 from src.problem2.run import FORMAL_FORECAST_PATH, settle_realized_day
17 from src.problem3.information_schedule import (
18     SettlementMode,
19     adjustment_settlement,
20     get_q3_forecast_update,
21     schedule_releases,
22 )
23 from src.problem3.rolling_model import Q3RollingPlanResult, solve_remaining_dispatch
24
25
26 DT_HOURS = 1.0 / 6.0
27 ACTUAL_COLUMNS = (
28     "actual_load_kw",
29     "actual_pv_kw",
30     "actual_charge_kw",
31     "actual_discharge_kw",
32     "actual_storage_start_kwh",
33     "actual_storage_end_kwh",
34     "realized_planned_grid_used_kw",
35     "realized_unused_planned_grid_kw",
36     "realized_emergency_kw",
37     "realized_pv_spill_kw",
38     "actual_charge_kwh",
39     "actual_discharge_kwh",
40     "realized_planned_grid_used_kwh",
41     "realized_unused_planned_grid_kwh",
42     "realized_emergency_kwh",
43     "realized_pv_spill_kwh",
44     "realized_emergency_cost_yuan",
45 )
46
47
48 @dataclass(frozen=True)
49 class Q3ScheduleResult:
50     schedule: str
51     daily_main: pd.DataFrame
52     daily_sensitivity: pd.DataFrame
53     intervals: pd.DataFrame
54     audit: dict[str, object]
55
56
57 PriceProvider = Callable[[object], tuple[np.ndarray, np.ndarray]]
58
59
60 def _validated_full_day_price(
61     values: np.ndarray | None,
62     fallback: np.ndarray,
63     *,
64     label: str,
65 ) -> np.ndarray:
66     price = np.asarray(fallback if values is None else values, dtype=float)
67     if price.shape != (144,) or not np.isfinite(price).all() or (price < 0.0).any():
68         raise ValueError(f"{label} must contain 144 finite nonnegative yuan/kWh values")
69     return np.array(price, copy=True)

```

```

70
71
72 def build_release_inputs(
73     date: object,
74     release_hour: int,
75     current_realized_soc: float,
76     price_yuan_per_kwh: np.ndarray | None = None,
77 ) -> tuple[Q2DayInputs, object]:
78     """Build the locked 50-scenario remaining horizon around Arya's forecast."""
79
80     base = get_q2_day_inputs(date, initial_energy=current_realized_soc)
81     full_day_price = _validated_full_day_price(
82         price_yuan_per_kwh,
83         base.price,
84         label="Q3 decision price",
85     )
86     update = get_q3_forecast_update(date, release_hour, current_realized_soc)
87     start = int(release_hour) * 6
88     expected_future = np.arange(144) >= start
89     if not np.array_equal(update.future_mask, expected_future):
90         raise AssertionError("Q3 release mask does not match the ten-minute grid")
91     center = np.array(base.pv_forecast, copy=True)
92     center[start:] = update.fused_forecast_kw[start:]
93     residual = base.pv_scenarios - base.pv_forecast[None, :]
94     pv_scenarios = np.maximum(center[None, start:] + residual[:, start:], 0.0)
95     inputs = Q2DayInputs(
96         date=base.date,
97         load_forecast=np.array(base.load_forecast[start:], copy=True),
98         pv_forecast=np.array(center[start:], copy=True),
99         load_scenarios=np.array(base.load_scenarios[:, start:], copy=True),
100         pv_scenarios=pv_scenarios,
101         price=np.array(full_day_price[start:], copy=True),
102         initial_energy=float(current_realized_soc),
103         timestamps=np.array(base.timestamps[start:], copy=True),
104         scenario_source_dates=np.array(base.scenario_source_dates, copy=True),
105         dt_hours=base.dt_hours,
106     )
107     return inputs, update
108
109
110 def _solve_release(
111     date: object,
112     release_hour: int,
113     current_realized_soc: float,
114     prior_commitment_kw: np.ndarray | None = None,
115     price_yuan_per_kwh: np.ndarray | None = None,
116 ) -> tuple[Q3RollingPlanResult, object]:
117     inputs, update = build_release_inputs(
118         date,
119         release_hour,
120         current_realized_soc,
121         price_yuan_per_kwh=price_yuan_per_kwh,
122     )
123     result = solve_remaining_dispatch(
124         inputs,
125         Q2DispatchParameters(),
126         prior_commitment_kw=prior_commitment_kw,
127     )
128     frame = result.dispatch.copy()
129     frame["slot"] = np.arange(release_hour * 6 + 1, 145)
130     frame["release_hour"] = release_hour
131     result = replace(result, dispatch=frame)
132     return result, update
133
134
135 def settle_segment_with_locked_q2(

```

```

136 plan_segment: pd.DataFrame,
137 actual_segment: pd.DataFrame,
138 initial_energy_kwh: float,
139 ) -> tuple[pd.DataFrame, float]:
140     """Execute a segment by invoking the unchanged 144-row Q2 settlement."""
141
142     plan = plan_segment.copy().reset_index(drop=True)
143     actual = actual_segment.copy().reset_index(drop=True)
144     count = len(plan)
145     if count < 1 or count > 144 or len(actual) != count:
146         raise ValueError("Q3 execution segment has inconsistent length")
147     if not np.array_equal(
148         pd.to_datetime(plan["timestamp"]).to_numpy(dtype="datetime64[ns]"),
149         pd.to_datetime(actual["datetime"]).to_numpy(dtype="datetime64[ns]"),
150     ):
151         raise AssertionError("Q3 plan and actual segment timestamps are misaligned")
152     minimal = plan[
153         [
154             "slot",
155             "timestamp",
156             "planned_grid_kw",
157             "planned_charge_limit_kw",
158             "planned_discharge_limit_kw",
159             "price_yuan_per_kwh",
160         ]
161     ].copy()
162     minimal["slot"] = np.arange(1, count + 1)
163     actual_minimal = actual[["datetime", "actual_load", "actual_generation"]].copy()
164     if count < 144:
165         padding_count = 144 - count
166         last_time = pd.Timestamp(actual_minimal["datetime"].iloc[-1])
167         pad_times = pd.date_range(last_time + pd.Timedelta(minutes=10), periods=padding_count, freq="10min")
168         plan_pad = pd.DataFrame(
169             {
170                 "slot": np.arange(count + 1, 145),
171                 "timestamp": pad_times,
172                 "planned_grid_kw": 0.0,
173                 "planned_charge_limit_kw": 0.0,
174                 "planned_discharge_limit_kw": 0.0,
175                 "price_yuan_per_kwh": 0.0,
176             }
177         )
178         actual_pad = pd.DataFrame(
179             {"datetime": pad_times, "actual_load": 0.0, "actual_generation": 0.0}
180         )
181         minimal = pd.concat([minimal, plan_pad], ignore_index=True)
182         actual_minimal = pd.concat([actual_minimal, actual_pad], ignore_index=True)
183     settled, final_energy = settle_realized_day(
184         minimal, actual_minimal, float(initial_energy_kwh)
185     )
186     output = plan.copy()
187     for column in ACTUAL_COLUMNS:
188         output[column] = settled[column].iloc[:count].to_numpy()
189     return output, float(final_energy)
190
191 def _settlement_rows(
192     original_grid_kwh: np.ndarray,
193     revisions_kwh: list[np.ndarray],
194     price: np.ndarray,
195 ) -> dict[SettlementMode, dict[str, float]]:
196     if not revisions_kwh:
197         return {
198             mode: {
199                 "adjustment_cost_yuan": 0.0,
200                 "adjustment_energy_kwh": 0.0,

```

```

202     "adjustment_increase_energy_kwh": 0.0,
203     "adjustment_decrease_energy_kwh": 0.0,
204     "adjusted_interval_count": 0,
205 }
206 for mode in SettlementMode
207 }
208 revisions = np.stack(revisions_kwh)
209 results: dict[SettlementMode, dict[str, float]] = {}
210 for mode in SettlementMode:
211     settled = adjustment_settlement(original_grid_kwh, revisions, price, mode)
212     delta_plus = settled["delta_plus_kwh"]
213     delta_minus = settled["delta_minus_kwh"]
214     touched = np.any(delta_plus + delta_minus > 1e-9, axis=0)
215     results[mode] = {
216         "adjustment_cost_yuan": float(settled["adjustment_cost_yuan"].sum()),
217         "adjustment_energy_kwh": float((delta_plus + delta_minus).sum()),
218         "adjustment_increase_energy_kwh": float(delta_plus.sum()),
219         "adjustment_decrease_energy_kwh": float(delta_minus.sum()),
220         "adjusted_interval_count": int(touched.sum()),
221     }
222 return results
223
224
225 def run_schedule_day(
226     date: object,
227     schedule: str,
228     initial_energy_kwh: float,
229     actual_day: pd.DataFrame,
230     *,
231     decision_price_yuan_per_kwh: np.ndarray | None = None,
232     settlement_price_yuan_per_kwh: np.ndarray | None = None,
233 ) -> tuple[pd.DataFrame, list[dict[str, object]], float, dict[str, float]]:
234     releases = schedule_releases(schedule)
235     current_soc = float(initial_energy_kwh)
236     original_grid_kwh: np.ndarray | None = None
237     current_commitment_kw: np.ndarray | None = None
238     revisions_kwh: list[np.ndarray] = []
239     executed: list[pd.DataFrame] = []
240     total_solver_runtime = 0.0
241     freeze_residual = 0.0
242     causal_checks = True
243     initial_expected_cost = np.nan
244     maximum_scenario_balance = 0.0
245     optimized_adjustment_cost = 0.0
246     fixed_q2_price = get_q2_day_inputs(date, initial_energy=initial_energy_kwh).price
247     decision_price = _validated_full_day_price(
248         decision_price_yuan_per_kwh,
249         fixed_q2_price,
250         label="Q3 decision price",
251     )
252     price = _validated_full_day_price(
253         settlement_price_yuan_per_kwh,
254         decision_price,
255         label="Q3 settlement price",
256     )
257     dynamic_price_override = (
258         decision_price_yuan_per_kwh is not None
259         or settlement_price_yuan_per_kwh is not None
260     )
261     for release_index, release_hour in enumerate(releases):
262         start = release_hour * 6
263         prior_future = (
264             None
265             if release_index == 0
266             else np.array(current_commitment_kw[start:], copy=True)
267         )

```

```

268 result, update = _solve_release(
269     date,
270     release_hour,
271     current_soc,
272     prior_commitment_kw=prior_future,
273     price_yuan_per_kwh=decision_price,
274 )
275 total_solver_runtime += result.runtime_seconds
276 maximum_scenario_balance = max(
277     maximum_scenario_balance,
278     result.maximum_scenario_power_balance_residual_kw,
279 )
280 if release_index > 0:
281     optimized_adjustment_cost += result.adjustment_cost_yuan
282 plan = result.dispatch.copy().reset_index(drop=True)
283 if release_index == 0:
284     current_commitment_kw = plan["planned_grid_kw"].to_numpy(copy=True)
285     original_grid_kwh = current_commitment_kw * DT_HOURS
286     initial_expected_cost = result.operating_cost_yuan
287 else:
288     assert current_commitment_kw is not None
289     previous = current_commitment_kw.copy()
290     current_commitment_kw[start:] = plan["planned_grid_kw"].to_numpy()
291     freeze_residual = max(
292         freeze_residual,
293         float(np.max(np.abs(current_commitment_kw[start:] - previous[start:])))
294         if start else 0.0,
295     )
296     revisions_kwh.append(current_commitment_kw.copy() * DT_HOURS)
297 causal_checks = causal_checks and bool(
298     update.actual_history_cutoff == update.issue_time
299     and update.executed_mask[start:].all()
300     and update.future_mask[start:].all()
301     and not np.isfinite(update.fused_forecast_kw[start:]).any()
302     and np.isfinite(update.fused_forecast_kw[start:]).all()
303     and abs(update.current_realized_soc - current_soc) <= 1e-9
304 )
305 end = releases[release_index + 1] * 6 if release_index + 1 < len(releases) else 144
306 segment = plan.iloc[: end - start].copy()
307 if dynamic_price_override:
308     segment["decision_price_yuan_per_kwh"] = segment["price_yuan_per_kwh"]
309     segment["price_yuan_per_kwh"] = price[start:end]
310 actual_segment = actual_day.iloc[start:end].copy()
311 settled, current_soc = settle_segment_with_locked_q2(
312     segment, actual_segment, current_soc
313 )
314 settled["schedule"] = schedule
315 settled["release_hour"] = release_hour
316 settled["operating_date"] = str(pd.Timestamp(date).date())
317 executed.append(settled)
318 intervals = pd.concat(executed, ignore_index=True).sort_values("slot").reset_index(drop=True)
319 if len(intervals) != 144 or intervals["slot"].duplicated().any():
320     raise AssertionError("Q3 rolling execution did not freeze exactly 144 unique intervals")
321 assert original_grid_kwh is not None
322 settlement = _settlement_rows(original_grid_kwh, revisions_kwh, price)
323 decision_settlement = _settlement_rows(
324     original_grid_kwh,
325     revisions_kwh,
326     decision_price,
327 )
328 main_adjustment_cost = settlement[
329     SettlementMode.SEQUENTIAL_PREVIOUS_COMMITMENT
330 ]["adjustment_cost_yuan"]
331 adjustment_objective_residual = abs(
332     decision_settlement[SettlementMode.SEQUENTIAL_PREVIOUS_COMMITMENT][
333         "adjustment_cost_yuan"

```

```

334     ]
335     - optimized_adjustment_cost
336 )
337 if adjustment_objective_residual > 2e-3:
338     raise AssertionError(
339         "Q3 optimized adjustment cost does not match sequential settlement"
340     )
341 emergency_cost = float(intervals["realized_emergency_cost_yuan"].sum())
342 base_cost = float(np.sum(price * original_grid_kwh))
343 rows: list[dict[str, object]] = []
344 for mode, values in settlement.items():
345     rows.append(
346         {
347             "date": str(pd.Timestamp(date).date()),
348             "schedule": schedule,
349             "settlement_mode": mode.value,
350             "solver_status": "Optimal",
351             "solver_success": 1,
352             "solver_runtime_seconds": total_solver_runtime,
353             "release_count": len(releases),
354             "maximum_scenario_power_balance_residual_kw": maximum_scenario_balance,
355             "initial_energy_kwh": initial_energy_kwh,
356             "final_energy_kwh": current_soc,
357             "initial_planned_purchase_energy_kwh": float(original_grid_kwh.sum()),
358             "initial_planned_purchase_cost_yuan": base_cost,
359             "initial_expected_operating_cost_yuan": initial_expected_cost,
360             **values,
361             "emergency_energy_kwh": float(intervals["realized_emergency_kwh"].sum()),
362             "emergency_cost_yuan": emergency_cost,
363             "realized_total_cost_yuan": base_cost + values["adjustment_cost_yuan"] + emergency_cost,
364             "emergency_interval_count": int((intervals["realized_emergency_kw"] > 1e-9).sum()),
365             "battery_throughput_kwh": float(
366                 intervals["actual_charge_kwh"].sum() + intervals["actual_discharge_kwh"].sum()
367             ),
368             "mean_soc_kwh": float(
369                 intervals[["actual_storage_start_kwh", "actual_storage_end_kwh"]].to_numpy().mean()
370             ),
371             "minimum_soc_kwh": float(
372                 intervals[["actual_storage_start_kwh", "actual_storage_end_kwh"]].min().min()
373             ),
374             "maximum_soc_kwh": float(
375                 intervals[["actual_storage_start_kwh", "actual_storage_end_kwh"]].max().max()
376             ),
377         }
378     )
379 checks = {
380     "freeze_residual_kw": freeze_residual,
381     "causal_release_inputs": float(causal_checks),
382     "maximum_scenario_power_balance_residual_kw": maximum_scenario_balance,
383     "adjustment_objective_residual_yuan": adjustment_objective_residual,
384 }
385 intervals["initial_00_grid_kw"] = original_grid_kwh / DT_HOURS
386 return intervals, rows, current_soc, checks
387
388
389 def audit_schedule(
390     daily_main: pd.DataFrame,
391     intervals: pd.DataFrame,
392     freeze_residual_kw: float,
393     causal_checks: bool,
394     adjustment_objective_residual_yuan: float = 0.0,
395 ) -> dict[str, object]:
396     tolerance = 2e-3
397     balance = (
398         intervals["realized_planned_grid_used_kw"]
399         + intervals["realized_emergency_kw"]

```

```

400 + intervals["actual_pv_kw"]
401 + intervals["actual_discharge_kw"]
402 - intervals["actual_load_kw"]
403 - intervals["actual_charge_kw"]
404 - intervals["realized_pv_spill_kw"]
405 )
406 expected_soc = (
407     intervals["actual_storage_start_kwh"]
408     + 0.9 * intervals["actual_charge_kw"] * DT_HOURS
409     - intervals["actual_discharge_kw"] * DT_HOURS / 0.9
410 )
411 soc_error = intervals["actual_storage_end_kwh"] - expected_soc
412 cross = daily_main["initial_energy_kwh"].iloc[1:].to_numpy() - daily_main[
413     "final_energy_kwh"
414 ].iloc[:-1].to_numpy()
415 maximum_cross_day = float(np.max(np.abs(cross))) if len(cross) else 0.0
416 absorbable = np.maximum(
417     intervals["actual_load_kw"] + intervals["actual_charge_kw"] - intervals["actual_pv_kw"],
418     0.0,
419 )
420 report = {
421     "realized_power_balance": bool(np.abs(balance).max() <= tolerance),
422     "maximum_power_balance_residual_kw": float(np.abs(balance).max()),
423     "realized_soc_recurrence": bool(np.abs(soc_error).max() <= tolerance),
424     "maximum_soc_recurrence_residual_kwh": float(np.abs(soc_error).max()),
425     "cross_day_realized_soc": bool(maximum_cross_day <= tolerance),
426     "maximum_cross_day_soc_residual_kwh": maximum_cross_day,
427     "soc_bounds": bool(
428         intervals[["actual_storage_start_kwh", "actual_storage_end_kwh"]].min().min()
429         >= 1200.0 - tolerance
430         and intervals[["actual_storage_start_kwh", "actual_storage_end_kwh"]].max().max()
431         <= 10800.0 + tolerance
432     ),
433     "actual_action_limits": bool(
434         (intervals["actual_charge_kw"] <= intervals["planned_charge_limit_kw"] + tolerance).all()
435         and (intervals["actual_discharge_kw"] <= intervals["planned_discharge_limit_kw"] + tolerance).all()
436     ),
437     "phantom_discharge": bool((intervals["actual_discharge_kw"] <= absorbable + tolerance).all()),
438     "unabsorbed_discharge_energy_kwh": 0.0,
439     "hidden_export": bool((intervals["realized_pv_spill_kw"] <= intervals["actual_pv_kw"] + tolerance).all()),
440     "emergency_nonnegative": bool((intervals["realized_emergency_kw"] >= -tolerance).all()),
441     "executed_interval_freeze": bool(freeze_residual_kw <= tolerance),
442     "maximum_executed_freeze_residual_kw": float(freeze_residual_kw),
443     "causality": bool(causal_checks),
444     "settlement_reconciliation": bool(
445         np.max(
446             np.abs(
447                 daily_main["realized_total_cost_yuan"]
448                 - daily_main["initial_planned_purchase_cost_yuan"]
449                 - daily_main["adjustment_cost_yuan"]
450                 - daily_main["emergency_cost_yuan"]
451             )
452         )
453         <= tolerance
454     ),
455     "adjustment_objective_reconciliation": bool(
456         adjustment_objective_residual_yuan <= tolerance
457     ),
458     "maximum_adjustment_objective_residual_yuan": float(
459         adjustment_objective_residual_yuan
460     ),
461     "time_alignment": bool(
462         len(intervals) == len(daily_main) * 144
463         and intervals.groupby("operating_date")["slot"].nunique().eq(144).all()
464     ),
465 }

```

```

466 if not all(value for value in report.values() if isinstance(value, bool)):
467     raise AssertionError(f"Q3 schedule audit failed: {report}")
468 return report
469
470
471 def run_schedule(
472     schedule: str,
473     dates: pd.DatetimeIndex,
474     *,
475     progress: bool = False,
476     checkpoint_path: Path | None = None,
477     price_provider: PriceProvider | None = None,
478     price_information_mode: str = "FIXED_Q2",
479 ) -> Q3ScheduleResult:
480     actual = pd.read_csv(
481         FORMAL_FORECAST_PATH,
482         usecols=["operating_date", "datetime", "actual_load", "actual_generation"],
483         parse_dates=["datetime"],
484     )
485     carried_soc = 6000.0
486     interval_frames: list[pd.DataFrame] = []
487     rows: list[dict[str, object]] = []
488     max_freeze = 0.0
489     causal = True
490     max_adjustment_objective_residual = 0.0
491     started = perf_counter()
492     start_index = 0
493     if checkpoint_path is not None and checkpoint_path.exists():
494         with checkpoint_path.open("rb") as stream:
495             checkpoint = pickle.load(stream)
496             expected_dates = [str(pd.Timestamp(value).date()) for value in dates]
497             if (
498                 checkpoint["schedule"] != schedule
499                 or checkpoint["dates"] != expected_dates
500                 or checkpoint.get("price_information_mode", "FIXED_Q2")
501                 != price_information_mode
502             ):
503                 raise AssertionError("Q3 checkpoint does not match the requested schedule/dates")
504             start_index = int(checkpoint["next_index"])
505             carried_soc = float(checkpoint["carried_soc"])
506             interval_frames = checkpoint["interval_frames"]
507             rows = checkpoint["rows"]
508             max_freeze = float(checkpoint["max_freeze"])
509             causal = bool(checkpoint["causal"])
510             max_adjustment_objective_residual = float(
511                 checkpoint["max_adjustment_objective_residual"]
512             )
513             if progress:
514                 print(f"Q3 {schedule}: resumed at {start_index}/{len(dates)} days", flush=True)
515     for index, date in enumerate(dates[start_index:], start=start_index + 1):
516         actual_day = actual.loc[
517             actual["operating_date"].astype(str) == str(date.date())
518         ].sort_values("datetime", kind="stable")
519         if len(actual_day) != 144:
520             raise AssertionError(f"Appendix 2 actual day {date.date()} does not contain 144 intervals")
521         if price_provider is None:
522             decision_price = None
523             settlement_price = None
524         else:
525             decision_price, settlement_price = price_provider(date)
526         intervals, day_rows, carried_soc, checks = run_schedule_day(
527             date,
528             schedule,
529             carried_soc,
530             actual_day,
531             decision_price_yuan_per_kwh=decision_price,

```

```

532     settlement_price_yuan_per_kwh=settlement_price,
533 )
534 interval_frames.append(intervals)
535 rows.extend(day_rows)
536 max_freeze = max(max_freeze, checks["freeze_residual_kw"])
537 causal = causal and bool(checks["causal_release_inputs"])
538 max_adjustment_objective_residual = max(
539     max_adjustment_objective_residual,
540     checks["adjustment_objective_residual_yuan"],
541 )
542 if progress and (index == 1 or index % 10 == 0 or index == len(dates)):
543     print(f"Q3 {schedule}: {index}/{len(dates)} days Optimal", flush=True)
544 if checkpoint_path is not None and (index % 10 == 0 or index == len(dates)):
545     checkpoint_path.parent.mkdir(parents=True, exist_ok=True)
546     temporary = checkpoint_path.with_suffix(".tmp")
547     payload = {
548         "schedule": schedule,
549         "dates": [str(pd.Timestamp(value).date()) for value in dates],
550         "next_index": index,
551         "carried_soc": carried_soc,
552         "interval_frames": interval_frames,
553         "rows": rows,
554         "max_freeze": max_freeze,
555         "causal": causal,
556         "max_adjustment_objective_residual": max_adjustment_objective_residual,
557         "price_information_mode": price_information_mode,
558     }
559     with temporary.open("wb") as stream:
560         pickle.dump(payload, stream, protocol=pickle.HIGHEST_PROTOCOL)
561     temporary.replace(checkpoint_path)
562 all_daily = pd.DataFrame(rows)
563 main = all_daily.loc[
564     all_daily["settlement_mode"].eq(SettlementMode.SEQUENTIAL_PREVIOUS_COMMITMENT.value)
565 ].reset_index(drop=True)
566 intervals = pd.concat(interval_frames, ignore_index=True)
567 main["wall_runtime_seconds"] = perf_counter() - started
568 audit = audit_schedule(
569     main,
570     intervals,
571     max_freeze,
572     causal,
573     max_adjustment_objective_residual,
574 )
575 return Q3ScheduleResult(schedule, main, all_daily, intervals, audit)
576
577
578 _all_ = [
579     "Q3ScheduleResult",
580     "PriceProvider",
581     "audit_schedule",
582     "build_release_inputs",
583     "run_schedule",
584     "run_schedule_day",
585     "settle_segment_with_locked_q2",
586 ]

```

3.4 问题四动态电价接口

Listing 10 src/problem4/price_interface.py

```

1 """Dispatch-neutral price contract for future Q4-2 and Q4-3 optimizers."""
2
3 from __future__ import annotations

```

```

4
5 from dataclasses import dataclass
6 from enum import StrEnum
7
8 import numpy as np
9 import pandas as pd
10
11 from src.problem4.price_forecast import PREDICTIONS_PATH, SELECTED_OUTPUT_COLUMN, load_selection
12
13
14 FORMAL_START = pd.Timestamp("2025-02-01")
15 FORMAL_END = pd.Timestamp("2025-12-31")
16 RELEASE_HOURS = (0, 6, 12, 18)
17 HORIZON = 144
18 DT_HOURS = 1.0 / 6.0
19
20
21 class PriceInformationMode(StrEnum):
22     """Two explicit interpretations of the statement's price availability."""
23
24     ORACLE_PERFECT_INFORMATION = "oracle_perfect_information"
25     CAUSAL_FORECAST = "causal_forecast"
26
27
28 @dataclass(frozen=True)
29 class Q4PriceInputs:
30     """Price-only inputs; no battery state or dispatch decision is included."""
31
32     date: np.datetime64
33     issue_time: np.datetime64
34     actual_history_cutoff: np.datetime64
35     interval_end: np.ndarray
36     price_yuan_per_kwh: np.ndarray
37     executed_mask: np.ndarray
38     future_mask: np.ndarray
39     mode: str
40     source_method: str
41     dt_hours: float = DT_HOURS
42
43
44 def _load_predictions() -> pd.DataFrame:
45     frame = pd.read_csv(
46         PREDICTIONS_PATH,
47         parse_dates=["operating_date", "interval_start", "interval_end"],
48     )
49     if frame["interval_end"].duplicated().any():
50         raise AssertionError("Persisted Q4 price timestamps must be unique")
51     return frame
52
53
54 def get_q4_price_inputs(
55     date: str | pd.Timestamp,
56     *,
57     release_hour: int = 0,
58     mode: PriceInformationMode | str,
59 ) -> Q4PriceInputs:
60     """Return a 144-step price path under one explicit information assumption.
61
62     Executed intervals are represented by NaN in ``price_yuan_per_kwh``. The
63     future optimizer must only consume entries where ``future_mask`` is true.
64     Prices are yuan/kWh and the time step is 1/6 hour.
65     The causal path is issued at midnight and reused at later releases; those
66     calls mask executed intervals but do not assimilate intraday price updates.
67     """
68
69     requested = pd.Timestamp(date)

```

```

70 if pd.isna(requested) or requested.tzinfo is not None or requested != requested.normalize():
71     raise ValueError("date must be a timezone-naive midnight operating date")
72 operating_date = requested.normalize()
73 if not FORMAL_START <= operating_date <= FORMAL_END:
74     raise ValueError("Q4 formal date must lie between 2025-02-01 and 2025-12-31")
75 if isinstance(release_hour, bool) or release_hour not in RELEASE_HOURS:
76     raise ValueError(f"release_hour must be one of {RELEASE_HOURS}")
77 information_mode = PricelInformationMode(mode)
78 frame = _load_predictions()
79 day = frame.loc[frame["operating_date"].eq(operating_date)].sort_values("slot")
80 if len(day) != HORIZON or not day["slot"].to_numpy().tolist() == list(range(1, 145)):
81     raise AssertionError("Requested date must contain exactly slots 1..144")
82 issue_time = operating_date + pd.Timedelta(hours=release_hour)
83 interval_end = day["interval_end"].to_numpy(dtype="datetime64[ns]")
84 expected = pd.date_range(operating_date + pd.Timedelta(minutes=10), periods=144, freq="10min")
85 if not np.array_equal(interval_end, expected.to_numpy(dtype="datetime64[ns]")):
86     raise AssertionError("Price timestamps do not match the requested operating day")
87 executed = interval_end <= np.datetime64(issue_time)
88 future = ~executed
89 selection = load_selection()
90 if information_mode is PricelInformationMode.ORACLE_PERFECT_INFORMATION:
91     values = day["price_yuan_per_kwh"].to_numpy(float)
92     source_method = "附件4完美信息价格"
93 else:
94     validation_end = pd.Timestamp(selection["validation_period"][1]) + pd.Timedelta(days=1)
95     if validation_end >= operating_date or not selection["model_frozen_before_evaluation"]:
96         raise AssertionError("Price model selection must precede the requested day")
97     values = day[SELECTED_OUTPUT_COLUMN].to_numpy(float)
98     source_method = str(selection["selected_method"])
99 output = values.copy()
100 output[executed] = np.nan
101 if not np.isfinite(output[future]).all():
102     raise AssertionError("Future Q4 price inputs must all be finite")
103 return Q4PricelInputs(
104     date=operating_date.to_datetime64(),
105     issue_time=issue_time.to_datetime64(),
106     actual_history_cutoff=issue_time.to_datetime64(),
107     interval_end=interval_end,
108     price_yuan_per_kwh=output,
109     executed_mask=executed,
110     future_mask=future,
111     mode=information_mode.value,
112     source_method=source_method,
113 )
114
115
116 def get_q4_2_price_inputs(
117     date: str | pd.Timestamp,
118     *,
119     mode: PricelInformationMode | str,
120 ) -> Q4PricelInputs:
121     """Price contract for future Q4-2 day-ahead dispatch; dispatch is not run."""
122
123     return get_q4_price_inputs(date, release_hour=0, mode=mode)
124
125
126 def get_q4_3_price_inputs(
127     date: str | pd.Timestamp,
128     release_hour: int,
129     *,
130     mode: PricelInformationMode | str,
131 ) -> Q4PricelInputs:
132     """Price contract for future Q4-3 rolling dispatch; dispatch is not run."""
133
134     return get_q4_price_inputs(date, release_hour=release_hour, mode=mode)
135

```

```

136
137 _all_ = [
138     "DT_HOURS",
139     "FORMAL_END",
140     "FORMAL_START",
141     "HORIZON",
142     "PriceInformationMode",
143     "Q4PriceInputs",
144     "RELEASE_HOURS",
145     "get_q4_2_price_inputs",
146     "get_q4_3_price_inputs",
147     "get_q4_price_inputs",
148 ]

```

Listing 11 src/problem4/dispatch_adapter.py

```

1 """Thin dynamic-price adapters around the locked Q2/Q3 dispatch pipelines."""
2
3 from __future__ import annotations
4
5 from dataclasses import dataclass, replace
6 import hashlib
7 from pathlib import Path
8 import pickle
9
10 import numpy as np
11 import pandas as pd
12
13 from src.problem2.evaluate import validate_q2_day
14 from src.problem2.forecast_interface import FORMAL_FORECAST_PATH, get_q2_day_inputs
15 from src.problem2.model import Q2DispatchParameters, solve_expected_cost_dispatch
16 from src.problem2.run import settle_realized_day
17 from src.problem3.rolling_dispatch import Q3ScheduleResult, run_schedule
18 from src.problem4.price_forecast import PREDICTIONS_PATH, SELECTION_PATH
19 from src.problem4.price_interface import PriceInformationMode, get_q4_2_price_inputs
20
21
22 GIVEN_PRICE = "GIVEN_PRICE"
23 CAUSAL_PRICE = "CAUSAL_PRICE"
24 PRICE_MODE_MAP = {
25     GIVEN_PRICE: PriceInformationMode.ORACLE_PERFECT_INFORMATION,
26     CAUSAL_PRICE: PriceInformationMode.CAUSAL_FORECAST,
27 }
28 DT_HOURS = 1.0 / 6.0
29
30
31 @dataclass(frozen=True)
32 class Q42SampleResult:
33     mode: str
34     daily: pd.DataFrame
35     intervals: pd.DataFrame
36     audit: dict[str, object]
37
38
39 def _price_pair(date: object, mode: str) -> tuple[np.ndarray, np.ndarray]:
40     """Return decision and realized settlement prices on the same 144-slot grid."""
41
42     if mode not in PRICE_MODE_MAP:
43         raise ValueError(f"unsupported Q4 price mode: {mode}")
44     decision = get_q4_2_price_inputs(date, mode=PRICE_MODE_MAP[mode])
45     actual = get_q4_2_price_inputs(
46         date, mode=PriceInformationMode.ORACLE_PERFECT_INFORMATION
47     )
48     if not np.array_equal(decision.interval_end, actual.interval_end):

```

```

49     raise AssertionError("decision and settlement price timestamps differ")
50 if decision.executed_mask.any() or actual.executed_mask.any():
51     raise AssertionError("midnight Q4 price input unexpectedly masks an interval")
52 decision_price = np.asarray(decision.price_yuan_per_kwh, dtype=float)
53 settlement_price = np.asarray(actual.price_yuan_per_kwh, dtype=float)
54 if not np.isfinite(decision_price).all() or not np.isfinite(settlement_price).all():
55     raise AssertionError("Q4 price provider returned non-finite prices")
56 return decision_price, settlement_price
57
58
59 def price_input_signature(mode: str) -> str:
60     """Bind resumable checkpoints to the selected persisted price inputs."""
61
62     digest = hashlib.sha256()
63     digest.update(mode.encode("utf-8"))
64     for path in (PREDICTIONS_PATH, SELECTION_PATH):
65         digest.update(path.read_bytes())
66     return digest.hexdigest()
67
68
69 def q43_price_provider(mode: str):
70     """Build the full-day provider consumed by the minimally extended Q3 runner."""
71
72     if mode not in PRICE_MODE_MAP:
73         raise ValueError(f"unsupported Q4 price mode: {mode}")
74
75     def provider(date: object) -> tuple[np.ndarray, np.ndarray]:
76         return _price_pair(date, mode)
77
78     return provider
79
80
81 def _actual_frame() -> pd.DataFrame:
82     return pd.read_csv(
83         FORMAL_FORECAST_PATH,
84         usecols=["operating_date", "datetime", "actual_load", "actual_generation"],
85         parse_dates=["datetime"],
86     ).sort_values(["operating_date", "datetime"], kind="stable")
87
88
89 def _audit_realized_dispatch(daily: pd.DataFrame, intervals: pd.DataFrame) -> dict[str, object]:
90     tolerance = 2e-3
91     balance = (
92         intervals["realized_planned_grid_used_kw"]
93         + intervals["realized_emergency_kw"]
94         + intervals["actual_pv_kw"]
95         + intervals["actual_discharge_kw"]
96         - intervals["actual_load_kw"]
97         - intervals["actual_charge_kw"]
98         - intervals["realized_pv_spill_kw"]
99     )
100     expected_soc = (
101         intervals["actual_storage_start_kwh"]
102         + 0.9 * intervals["actual_charge_kw"] * DT_HOURS
103         - intervals["actual_discharge_kw"] * DT_HOURS / 0.9
104     )
105     soc_residual = intervals["actual_storage_end_kwh"] - expected_soc
106     cross = (
107         daily["initial_energy_kwh"].iloc[1:].to_numpy(float)
108         - daily["actual_final_energy_kwh"].iloc[:-1].to_numpy(float)
109     )
110     absorbable = np.maximum(
111         intervals["actual_load_kw"]
112         + intervals["actual_charge_kw"]
113         - intervals["actual_pv_kw"],
114         0.0,

```

```

115 )
116 price_alignment_residual = np.abs(
117     intervals["realized_emergency_cost_yuan"]
118     - 5.0
119     * intervals["price_yuan_per_kwh"]
120     * intervals["realized_emergency_kwh"]
121 )
122 reconciliation = np.abs(
123     daily["realized_total_cost_yuan"]
124     - daily["planned_purchase_cost_yuan"]
125     - daily["realized_emergency_cost_yuan"]
126 )
127 checks = {
128     "optimal": bool(daily["solver_status"].eq("Optimal").all()),
129     "dynamic_price_alignment": bool(price_alignment_residual.max() <= tolerance),
130     "maximum_price_alignment_residual_yuan": float(price_alignment_residual.max()),
131     "realized_power_balance": bool(np.abs(balance).max() <= tolerance),
132     "maximum_power_balance_residual_kw": float(np.abs(balance).max()),
133     "soc_recurrence": bool(np.abs(soc_residual).max() <= tolerance),
134     "maximum_soc_recurrence_residual_kwh": float(np.abs(soc_residual).max()),
135     "cross_day_soc": bool(not len(cross) or np.abs(cross).max() <= tolerance),
136     "maximum_cross_day_soc_residual_kwh": float(np.abs(cross).max()) if len(cross) else 0.0,
137     "soc_bounds": bool(
138         intervals[["actual_storage_start_kwh", "actual_storage_end_kwh"]]
139         .to_numpy(float)
140         .min()
141         >= 1200.0 - tolerance
142         and intervals[["actual_storage_start_kwh", "actual_storage_end_kwh"]]
143         .to_numpy(float)
144         .max()
145         <= 10800.0 + tolerance
146     ),
147     "no_simultaneous_charge_discharge": bool(
148         ~(
149             (intervals["actual_charge_kw"] > tolerance)
150             & (intervals["actual_discharge_kw"] > tolerance)
151         ).any()
152     ),
153     "phantom_discharge": bool(
154         (intervals["actual_discharge_kw"] <= absorbable + tolerance).all()
155     ),
156     "unabsorbed_discharge_energy_kwh": 0.0,
157     "settlement_reconciliation": bool(reconciliation.max() <= tolerance),
158     "maximum_settlement_residual_yuan": float(reconciliation.max()),
159     "time_alignment": bool(
160         len(intervals) == 144 * len(daily)
161         and intervals.groupby("operating_date")["slot"].nunique().eq(144).all()
162     ),
163 }
164 if not all(value for value in checks.values() if isinstance(value, bool)):
165     raise AssertionError(f"Q4-2 physical audit failed: {checks}")
166 return checks
167
168
169 def run_q42_sample(
170     dates: pd.DatetimeIndex,
171     *,
172     mode: str,
173     checkpoint_path: Path | None = None,
174     progress: bool = False,
175     progress_log_path: Path | None = None,
176 ) -> Q42SampleResult:
177     """Run a short Q4-2 sample without copying or changing the locked Q2 model."""
178
179     actual = _actual_frame()
180     signature = price_input_signature(mode)

```

```

181 carried_soc = 6000.0
182 daily_rows: list[dict[str, object]] = []
183 interval_frames: list[pd.DataFrame] = []
184 start_index = 0
185 if checkpoint_path is not None and checkpoint_path.exists():
186     with checkpoint_path.open("rb") as stream:
187         state = pickle.load(stream)
188         expected_dates = [str(pd.Timestamp(value).date()) for value in dates]
189         if state["dates"] != expected_dates or state["signature"] != signature:
190             raise AssertionError("Q4-2 checkpoint does not match dates or price inputs")
191         start_index = int(state["next_index"])
192         carried_soc = float(state["carried_soc"])
193         daily_rows = state["daily_rows"]
194         interval_frames = state["interval_frames"]
195
196 for index, date in enumerate(dates[start_index:], start=start_index + 1):
197     decision_price, settlement_price = _price_pair(date, mode)
198     base_inputs = get_q2_day_inputs(date, initial_energy=carried_soc)
199     inputs = replace(base_inputs, price=decision_price)
200     solved = solve_expected_cost_dispatch(inputs, Q2DispatchParameters())
201     validation = validate_q2_day(inputs, solved)
202     plan = solved.dispatch.copy()
203     plan["decision_price_yuan_per_kwh"] = plan["price_yuan_per_kwh"]
204     plan["price_yuan_per_kwh"] = settlement_price
205     actual_day = actual.loc[
206         actual["operating_date"].astype(str).eq(str(pd.Timestamp(date).date()))
207     ].sort_values("datetime", kind="stable")
208     initial_soc = carried_soc
209     settled, carried_soc = settle_realized_day(plan, actual_day, carried_soc)
210     settled["operating_date"] = str(pd.Timestamp(date).date())
211     planned_cost = float(
212         np.sum(settlement_price * settled["planned_grid_kwh"].to_numpy(float))
213     )
214     emergency_cost = float(settled["realized_emergency_cost_yuan"].sum())
215     daily_rows.append(
216         {
217             "date": str(pd.Timestamp(date).date()),
218             "solver_status": solved.status,
219             "solver_runtime_seconds": solved.runtime_seconds,
220             "initial_energy_kwh": initial_soc,
221             "planned_final_energy_kwh": solved.final_energy_kwh,
222             "actual_final_energy_kwh": carried_soc,
223             "planned_purchase_cost_yuan": planned_cost,
224             "planned_purchase_energy_kwh": float(settled["planned_grid_kwh"].sum()),
225             "decision_price_planned_cost_yuan": solved.planned_purchase_cost_yuan,
226             "expected_scenario_emergency_cost_yuan": solved.expected_emergency_cost_yuan,
227             "realized_emergency_energy_kwh": float(
228                 settled["realized_emergency_kwh"].sum()
229             ),
230             "realized_emergency_cost_yuan": emergency_cost,
231             "realized_total_cost_yuan": planned_cost + emergency_cost,
232             "maximum_scenario_power_balance_residual_kw": validation.maximum_power_balance_residual_kw,
233         }
234     )
235     interval_frames.append(settled)
236     month_boundary = bool(
237         index == len(dates)
238         or pd.Timestamp(dates[index - 1]).month
239         != pd.Timestamp(dates[index]).month
240     )
241     if progress and (index == 1 or index % 10 == 0 or month_boundary):
242         print(f"Q4-2 {mode}: {index}/{len(dates)} days Optimal", flush=True)
243     if progress_log_path is not None and month_boundary:
244         progress_frame = pd.DataFrame(daily_rows)
245         progress_frame["month"] = pd.to_datetime(progress_frame["date"]).dt.to_period("M").astype(str)
246         monthly = progress_frame.groupby("month", sort=True).agg(

```

```

247     days_solved=("date", "size"),
248     optimal_days=("solver_status", lambda values: int(values.eq("Optimal").sum())),
249     planned_cost_yuan=("planned_purchase_cost_yuan", "sum"),
250     realized_cost_yuan=("realized_total_cost_yuan", "sum"),
251     emergency_energy_kwh=("realized_emergency_energy_kwh", "sum"),
252     emergency_cost_yuan=("realized_emergency_cost_yuan", "sum"),
253     ending_realized_soc_kwh=("actual_final_energy_kwh", "last"),
254 ).reset_index()
255 progress_log_path.parent.mkdir(parents=True, exist_ok=True)
256 temporary_log = progress_log_path.with_suffix(".tmp")
257 monthly.to_csv(temporary_log, index=False, float_format="%10f")
258 temporary_log.replace(progress_log_path)
259 if checkpoint_path is not None and (index % 10 == 0 or month_boundary):
260     checkpoint_path.parent.mkdir(parents=True, exist_ok=True)
261     temporary = checkpoint_path.with_suffix(".tmp")
262     with temporary.open("wb") as stream:
263         pickle.dump(
264             {
265                 "dates": [str(pd.Timestamp(value).date()) for value in dates],
266                 "signature": signature,
267                 "next_index": index,
268                 "carried_soc": carried_soc,
269                 "daily_rows": daily_rows,
270                 "interval_frames": interval_frames,
271             },
272             stream,
273             protocol=pickle.HIGHEST_PROTOCOL,
274         )
275     temporary.replace(checkpoint_path)
276
277 daily = pd.DataFrame(daily_rows)
278 intervals = pd.concat(interval_frames, ignore_index=True)
279 return Q42SampleResult(mode, daily, intervals, _audit_realized_dispatch(daily, intervals))
280
281
282 def run_q43_sample(
283     schedule: str,
284     dates: pd.DatetimeIndex,
285     *,
286     mode: str,
287     checkpoint_path: Path | None = None,
288     progress: bool = False,
289 ) -> Q3ScheduleResult:
290     """Run the locked Q3 rolling pipeline with only the price provider replaced."""
291
292     signature = price_input_signature(mode)
293     return run_schedule(
294         schedule,
295         dates,
296         progress=progress,
297         checkpoint_path=checkpoint_path,
298         price_provider=q43_price_provider(mode),
299         price_information_mode=f"{mode}:{signature}",
300     )
301
302
303 __all__ = [
304     "CAUSAL_PRICE",
305     "GIVEN_PRICE",
306     "PRICE_MODE_MAP",
307     "Q42SampleResult",
308     "price_input_signature",
309     "q43_price_provider",
310     "run_q42_sample",
311     "run_q43_sample",
312 ]

```